



UNIVERSITÉ HASSAN II DE CASABLANCA

Faculté des Sciences Ben M'Sick

Master Intelligence Artificielle / DSDB

Module : MLOps et DataOps

PredictOps

*Pipeline DataOps/MLOps pour la maintenance
prédictive industrielle*

RAPPORT FINAL DU PROJET

RÉALISÉ PAR

MSOUHLI Rahil	<i>Product Owner</i>
EL MAMOUNI ALAOUI Amina	<i>Scrum Master</i>
Fezzaz Imane	<i>Data Engineer</i>
ABRABRI Zineb	<i>Data Engineer</i>
Elkhatib Sara	<i>Analytics / Data Quality Engineer</i>
RABI Maroua	<i>ML Engineer</i>
AIT SI ALI Rachida	<i>MLOps Engineer</i>
SEKKAT Israa	<i>DevOps / API Engineer</i>

ENCADRÉ PAR

Mr. AITDAOUD Mohammed

Mr. AMZIL TAOUFIK

Année universitaire : 2025 – 2026

Remerciements

La réalisation de ce projet n'aurait pas été possible sans le soutien et l'accompagnement de plusieurs personnes que nous souhaitons remercier sincèrement.

Nous adressons tout d'abord nos plus vifs remerciements à Monsieur AITDAOUD Mohammed, notre encadrant du module de MLOps et DataOps, pour sa disponibilité, ses conseils précieux ainsi que ses orientations qui nous ont permis de mener à bien ce travail. Ses remarques constructives et son expérience ont constitué une source d'inspiration tout au long de la réalisation de ce projet.

Nous tenons également à remercier l'ensemble des enseignants de notre formation pour les connaissances théoriques et pratiques qu'ils nous ont transmises durant notre parcours universitaire. Ces acquis ont grandement contribué au développement de nos compétences et à la réalisation de ce pipeline DataOps/MLOps.

Nos remerciements s'adressent également à l'administration de notre faculté ainsi qu'à toutes les personnes ayant participé, directement ou indirectement, à l'élaboration de ce travail, que ce soit par leurs conseils, leurs encouragements ou leur soutien.

Enfin, nous exprimons notre profonde gratitude à nos familles et à nos proches pour leur soutien moral, leur patience, leur confiance et leurs encouragements permanents tout au long de nos études et durant la réalisation de ce projet.

À toutes ces personnes, nous adressons nos sincères remerciements et notre profonde reconnaissance.

Dédicace

Nous dédions ce travail à nos chers parents, qui ont toujours été une source de motivation, de soutien et d'encouragement tout au long de notre parcours académique. Leur amour, leurs sacrifices et leur confiance ont été essentiels à notre réussite et à notre épanouissement.

Nous le dédions également à nos familles et à nos proches qui nous ont accompagnés avec bienveillance, compréhension et soutien dans les moments les plus importants de notre formation.

À nos enseignants, qui ont contribué à notre apprentissage et au développement de nos compétences, nous exprimons notre respect et notre gratitude pour les efforts qu'ils déploient afin de former les générations futures.

Nous souhaitons également dédier ce projet à tous nos collègues et amis qui ont partagé avec nous cette expérience universitaire, marquée par le travail, la persévérance, l'entraide et le partage des connaissances.

Enfin, nous dédions ce travail à toutes les personnes qui croient en l'importance de la donnée, de l'ingénierie logicielle et de l'intelligence artificielle comme leviers de performance industrielle et de création de valeur.

Résumé

Dans un contexte marqué par l'accélération de la transformation numérique de l'industrie et par l'émergence de l'Industrie 4.0, les entreprises industrielles disposent aujourd'hui de volumes considérables de données issues de leurs équipements et systèmes de production. Les capteurs embarqués sur les machines permettent notamment de collecter en continu des informations relatives à la température, aux vibrations, à l'humidité, à la pression ou encore à la consommation énergétique. Ces données représentent une source importante d'information pour anticiper les défaillances, optimiser les opérations de maintenance et améliorer la disponibilité des équipements.

C'est dans cette perspective que s'inscrit le projet **PredictOps**, dont l'objectif principal est de concevoir et de déployer une plateforme complète de maintenance prédictive reposant sur les principes du **DataOps** et du **MLOps**. L'ambition du projet est de mettre en œuvre une chaîne de traitement industrialisée permettant de transformer des données brutes issues de capteurs industriels en prédictions exploitables par les équipes chargées de la maintenance.

L'étude est réalisée à partir d'un jeu de données composé de **100 000 relevés de capteurs** provenant d'un parc de **cinquante machines industrielles**. Les observations regroupent plusieurs variables représentatives de l'état de fonctionnement des équipements, notamment la température, les vibrations, l'humidité, la pression et la consommation énergétique.

L'analyse exploratoire et statistique des données constitue une première étape essentielle afin d'identifier les caractéristiques du parc industriel, de détecter d'éventuelles anomalies, d'étudier la distribution des variables et de mettre en évidence les facteurs susceptibles d'être associés aux besoins de maintenance. Les résultats de cette analyse montrent que **19,7% des relevés correspondent à une situation nécessitant une intervention de maintenance**. Cette proportion met en évidence l'intérêt d'un système automatisé permettant de détecter et de prioriser les situations nécessitant une attention particulière, plutôt que de s'appuyer exclusivement sur une surveillance manuelle et exhaustive des équipements.

Sur le plan de l'architecture DataOps, PredictOps repose sur une organisation structurée en plusieurs étapes complémentaires. L'ingestion automatisée des données est assurée par **dlt**, permettant de récupérer et de charger les données dans le système de stockage tout en favorisant la reproductibilité des traitements. Les données sont ensuite organisées selon une architecture analytique en couches reposant sur **DuckDB**, afin de séparer les données brutes des données transformées et préparées pour l'analyse.

La transformation, la modélisation et les contrôles de qualité sont réalisés avec **dbt**. Cette étape permet notamment de structurer les données, de documenter les transformations et de mettre en place des tests destinés à vérifier leur cohérence et leur fiabilité avant leur utilisation par les modèles de Machine Learning.

L'ensemble des différentes étapes du pipeline est ensuite coordonné à l'aide de **Dagster**, qui assure l'orchestration des traitements et permet de représenter explicitement les dépendances entre les différentes tâches du système. Cette orchestration contribue à rendre le pipeline plus fiable, observable et facilement reproductible.

La composante Machine Learning constitue une autre partie centrale du projet. Les données préparées sont utilisées pour construire et évaluer des modèles capables d'identifier les situations susceptibles de nécessiter une opération de maintenance. Le suivi des expérimentations et la gestion du cycle de vie des modèles sont assurés par **MLflow**. Cette approche permet de conserver une traçabilité des expériences réalisées, des paramètres utilisés, des métriques obtenues ainsi que des différentes versions des modèles entraînés.

Une fois le modèle retenu, celui-ci est exposé sous la forme d'un service accessible via une API **FastAPI**. Cette API constitue l'interface entre le modèle de Machine Learning et les applications susceptibles de l'utiliser. Elle permet notamment de transmettre de nouvelles observations issues des capteurs et d'obtenir en retour une prédiction pouvant contribuer à la prise de décision des équipes de maintenance.

Le service est conteneurisé afin d'assurer une meilleure portabilité et une reproductibilité de l'environnement d'exécution. Cette approche facilite également son intégration dans une infrastructure de déploiement automatisée.

La dimension DevOps du projet est assurée par l'intégration d'une chaîne **CI/CD basée sur GitHub Actions**. Les différentes étapes d'intégration, de validation et de déploiement sont ainsi automatisées afin de limiter les interventions manuelles et de garantir une meilleure cohérence entre les environnements de développement et de production. Le déploiement de la solution est réalisé sur l'infrastructure **Komodo**, permettant de disposer d'un environnement adapté à l'exécution des différents composants conteneurisés de la plateforme.

Sur le plan méthodologique, le développement de PredictOps a été conduit selon une organisation **Agile Scrum** structurée en trois sprints. Cette organisation a permis de répartir progressivement les différentes fonctionnalités du projet, depuis la préparation et la compréhension des données jusqu'à l'industrialisation du pipeline et au déploiement du service de prédiction.

Ainsi, PredictOps ne se limite pas à la conception d'un modèle prédictif. Le projet propose une approche intégrée dans laquelle la qualité et la gouvernance des données, l'automatisation des traitements, la traçabilité des expérimentations, l'orchestration des workflows, le déploiement du modèle et l'intégration continue sont considérés comme des composantes interdépendantes d'une même chaîne de valeur.

En définitive, le projet PredictOps montre comment l'exploitation structurée et gouvernée des données issues des équipements industriels peut contribuer à faire évoluer les stratégies de maintenance d'une approche principalement réactive vers une approche davantage **anticipative et orientée par les données**. En automatisant la collecte, la préparation, l'analyse et l'exploitation des données, la solution fournit une base technique permettant d'identifier plus rapidement les situations présentant un risque de maintenance et de faciliter leur priorisation.

Cette démarche peut ainsi contribuer à limiter les arrêts non planifiés, à améliorer la disponibilité des équipements et à optimiser l'utilisation des ressources de maintenance. Le projet met également en évidence l'importance d'une architecture reproductible et automatisée pour assurer la pérennité des solutions de Machine Learning dans des environnements industriels soumis à des exigences élevées en matière de fiabilité, de traçabilité et de disponibilité.

Table des matières

Remerciements	1
Dédicace	2
Résumé	3
Introduction Générale	13
Fiche synthétique du projet	14
1 Vision du projet, Data Strategy et Agile	15
1.1 Contexte général	15
1.2 Problématique	15
1.2.1 Pourquoi la maintenance prédictive est-elle un enjeu stratégique ? .	15
1.2.2 Quel problème industriel cette étude cherche-t-elle à résoudre ? . . .	16
1.2.3 Quels risques sont associés aux défaillances machines ?	16
1.3 Objectifs du projet	16
1.4 Utilisateurs cibles	17
1.5 Valeur métier	17
1.6 Data Strategy	18
1.6.1 Source et nature des données	18
1.6.2 Architecture de la stratégie data	19
1.6.3 Gouvernance et qualité des données	20
1.7 Organisation Agile du projet	20
1.7.1 Rôles Scrum de l'équipe	20
1.7.2 Planning des sprints	21
1.7.3 Cérémonies Agile	21
1.7.4 Board Agile	22
1.8 Product Backlog et User Stories	22
1.8.1 Product Backlog	22
1.8.2 Tableau des User Stories détaillées	23
1.8.3 Illustration — Product Backlog	27
1.8.4 Illustration — Board Agile	27
1.8.5 Illustration — Tableau des User Stories	28
1.8.6 Illustration — Analyse exploratoire du jeu de données	28

1.9	Conclusion du chapitre	29
2	Ingestion des données avec dlt	31
2.1	Introduction	31
2.2	Présentation du jeu de données	31
2.3	Pipeline d'ingestion	32
2.4	Implémentation des scripts d'ingestion	33
2.4.1	Définition de la source de données (<code>sources.py</code>)	33
2.4.2	Exécution du pipeline d'ingestion (<code>load_data.py</code>)	34
2.5	Vérification de l'ingestion	34
2.6	Conclusion	35
3	Stockage avec DuckDB	37
3.1	Création des tables	38
3.2	Contrôle du nombre de lignes	39
3.3	Recherche des valeurs manquantes	40
3.4	Création des variables dérivées	41
3.5	Statistiques descriptives	42
3.6	Distribution de la variable cible	42
3.7	Validation automatique	44
4	Transformations dbt et qualité des données	48
	Introduction	48
4.1	Objectif des transformations	48
4.2	Architecture dbt	49
4.3	Modèles dbt créés	51
4.4	Nettoyage des données	53
4.4.1	Vue d'ensemble des données brutes	54
4.4.2	Vérifications structurelles	54
4.4.3	Anomalie détectée : vibrations négatives	54
4.4.4	Analyse des autres variables numériques	55
4.4.5	Analyse des variables catégorielles et binaires	56
4.4.6	Analyse de <code>downtime_risk</code> et <code>predicted_remaining_life</code>	56
4.4.7	Remarques sur le jeu de données	57
4.5	Tests qualité	57
4.5.1	Tests sur le modèle <code>clean_machine_data</code>	58
4.5.2	Tests sur le modèle <code>ml_features</code>	59
4.5.3	Vue d'ensemble des résultats de tests	60
4.6	Data Contract	60
4.7	Documentation des données	61

4.8	Résultats des tests	62
	Conclusion	63
5	Orchestration avec Dagster	64
	Introduction	64
5.1	Rôle de Dagster dans le projet	64
5.2	Architecture du pipeline orchestré	65
5.3	Description des assets et jobs	65
5.4	Dépendances entre les étapes	66
5.5	Exécution du pipeline	67
5.6	Logs et suivi des erreurs	68
	Conclusion	68
6	Machine Learning et prédiction des pannes	70
	Introduction	70
6.1	Objectif du modèle	70
6.2	Données utilisées	70
6.3	Analyse du risque de fuite d'information	71
6.4	Préparation des données	71
6.5	Feature Engineering temporel	72
6.6	Séparation des données	72
6.7	Modèles testés	72
6.8	Optimisation du seuil de décision	73
6.9	Résultats obtenus	73
6.10	Comparaison des modèles	74
6.11	Interprétation des résultats	74
6.12	Importance des variables	74
6.13	Limites du modèle	75
6.14	Conclusion	75
	Conclusion	76
7	MLflow, versionnement et monitoring	77
7.1	Rôle de MLflow dans le projet	77
7.2	Experiment Tracking	77
7.3	Paramètres suivis	78
7.4	Métriques suivies	79
7.5	Artefacts sauvegardés	80
7.6	Model Registry	81
7.7	Versionnement du modèle	81
7.8	Monitoring ML	82

7.9	Détection simple de dérive	83
7.9.1	Méthodologie	83
7.9.2	Cas stable	84
7.9.3	Cas simulé	85
7.9.4	Synthèse et limites	85
7.10	Conclusion	86
8	FastAPI, Docker, CI/CD et déploiement	87
8.1	Objectif du déploiement	87
8.2	Architecture de l'API FastAPI	87
8.3	Endpoint GET <code>/health</code>	88
8.4	Endpoint POST <code>/predict</code>	89
8.5	Tests de l'API	90
8.6	Conteneurisation Docker	91
8.7	Docker Compose	91
8.8	Pipeline CI/CD avec GitHub Actions	91
8.9	Déploiement sur l'infrastructure Komodo	92
8.10	Monitoring du service	94
8.11	Conclusion	95
	Conclusion Générale	97

Table des figures

1.1	Vue consolidée du Product Backlog	27
1.2	Board Agile — suivi du Sprint 1	27
1.3	Tableau consolidé des User Stories	28
1.4	Résultats de l’analyse exploratoire du jeu de données	29
2.1	Exécution du pipeline d’ingestion avec dlt.	35
2.2	Vérification des données chargées dans la base DuckDB.	35
3.1	Architecture du Data Warehouse DuckDB.	37
3.2	Tables créées dans DuckDB.	39
3.3	Nombre de lignes des différentes tables.	39
3.4	Recherche des valeurs manquantes.	40
3.5	Création des variables dérivées.	41
3.6	Statistiques descriptives des principales variables.	42
3.7	Distribution de la variable cible.	43
3.8	Pipeline de validation automatique.	43
3.9	Exécution du script de validation.	45
3.10	Architecture globale du pipeline de données.	46
3.11	Cycle complet du projet de maintenance prédictive.	47
4.1	Graphe de lineage du pipeline dbt (raw → staging → clean → ml_features)	49
4.2	Structure du projet dans l’arborescence de fichiers	50
4.3	Résultat de la commande <code>dbt debug</code>	51
4.4	Code source du modèle <code>stg_machine_data.sql</code>	52
4.5	Code source du modèle <code>clean_machine_data.sql</code>	52
4.6	Code source du modèle <code>ml_features.sql</code>	53
4.7	Résultat de la commande <code>dbt run</code>	53
4.8	Extrait des données brutes (source <code>raw.raw_sensor_data</code>)	54
4.9	Vérification de l’absence de doublons	54
4.10	Vérification des valeurs manquantes	54
4.11	Quantification de l’anomalie de vibration négative	54
4.12	Répartition de la variable cible parmi les lignes concernées par l’anomalie	55
4.13	Volume total après application du filtre de nettoyage	55
4.14	Confirmation de l’absence de vibration négative après nettoyage	55
4.15	Distribution de la variable température par tranches de 10 degrés	55

4.16	Répartition des types de panne (<code>failure_type</code>) parmi les machines en état de panne (<code>machine_status = 2</code>)	56
4.17	Détail chiffré de l'incohérence entre <code>failure_type</code> et <code>machine_status</code>	56
4.18	Distribution de <code>downtime_risk</code> parmi les observations où <code>anomaly_flag = 1</code>	56
4.19	Moyenne de <code>predicted_remaining_life</code> selon l'état de la machine	57
4.20	Fichier de configuration des tests <code>clean_machine_data.yml</code>	58
4.21	Résultat de l'exécution des tests sur <code>clean_machine_data</code>	58
4.22	Fichier de configuration des tests et du contrat <code>ml_features.yml</code>	59
4.23	Résultat de l'exécution des tests sur <code>ml_features</code>	59
4.24	Résultat global de l'ensemble des tests du projet	60
4.25	Page d'accueil de la documentation dbt	62
4.26	Page de documentation de la source <code>raw.raw_sensor_data</code>	62
4.27	Page de documentation du modèle <code>ml_features</code>	62
4.28	Synthèse finale de l'exécution des tests du projet	63
5.1	Vue globale du lineage des assets dans Dagster	65
5.2	Détail d'une exécution réussie du pipeline (6 assets, 18 checks, statut Success)	67
5.3	Historique des exécutions du pipeline	68
7.1	Liste des runs enregistrés dans l'expérience <code>predictive_maintenance</code>	78
7.2	Paramètres enregistrés pour le run <code>gradient_boosting</code>	79
7.3	Métriques enregistrées pour le run <code>gradient_boosting</code>	80
7.4	Artefacts sauvegardés pour le run <code>gradient_boosting</code>	80
7.5	Modèle enregistré dans le Model Registry	81
7.6	Détail de la version 1 du modèle enregistré	82
7.7	Suivi de l'évolution de l'accuracy à travers les runs enregistrés	83
7.8	Résultat de la détection de dérive — cas stable (aucune alerte)	84
7.9	Résultat de la détection de dérive — cas simulé (température +30%, vibration +40%)	85
8.1	Réponse de l'endpoint <code>GET /health</code>	89
8.2	Test de l'endpoint <code>POST /predict</code> via Swagger UI	90
8.3	Pipeline CI/CD GitHub Actions en succès (jobs <code>test</code> et <code>build</code>)	92
8.4	Configuration du Stack Komodo – source Git et variables d'environnement	93
8.5	Stack Komodo à l'état <code>RUNNING</code> après correction	94
8.6	Journal Komodo – requêtes de health check internes (HTTP 200)	95

Liste des tableaux

2.1	Description des variables du jeu de données	32
4.1	Data Contract de la table <code>ml_features</code>	61
5.1	Assets Dagster du pipeline orchestré	66
6.1	Performances finales du meilleur modèle	73
6.2	Comparaison des modèles testés	74

Introduction Générale

La transformation numérique du secteur industriel, communément désignée sous le terme d'Industrie 4.0, repose sur l'interconnexion croissante des équipements de production par le biais de capteurs et de dispositifs IoT. Cette instrumentation généralisée des lignes de production génère des volumes de données considérables qui constituent aujourd'hui un levier stratégique pour repenser les politiques de maintenance industrielle. Face aux limites des approches correctives et préventives systématiques, la maintenance prédictive s'appuie sur l'analyse continue des données capteurs et sur des modèles de Machine Learning afin d'anticiper la dégradation d'un équipement avant que la panne ne survienne.

C'est dans cette perspective que s'inscrit notre projet, intitulé **PredictOps**, qui vise à concevoir un pipeline complet de bout en bout — de l'ingestion des données brutes jusqu'à la mise à disposition d'un modèle prédictif sous forme de service exploitable — en appliquant rigoureusement les principes DataOps et MLOps : reproductibilité, traçabilité, gouvernance de la donnée et automatisation.

Ce rapport présente successivement la vision du projet, la stratégie de gestion des données et l'organisation Agile mise en place (chapitre 1), l'ingestion des données avec **dlt** (chapitre 2), leur stockage analytique avec **DuckDB** (chapitre 3), les transformations et le contrôle qualité réalisés avec **dbt** (chapitre 4), l'orchestration du pipeline avec **Dagster** (chapitre 5), la modélisation par Machine Learning pour la prédiction des pannes (chapitre 6), le suivi des expériences et le monitoring avec **MLflow** (chapitre 7), et enfin la mise à disposition du modèle retenu via une API **FastAPI** conteneurisée et déployée en continu (chapitre 8).

À travers ce projet, nous souhaitons démontrer qu'une gouvernance rigoureuse de la donnée, associée à une chaîne d'outils DataOps et MLOps cohérente, permet de transformer un flux brut de relevés capteurs en un service de décision fiable, traçable et opérationnel pour les équipes de maintenance industrielle.

Fiche synthétique du projet

Nom du projet	PredictOps — Pipeline DataOps/MLOps pour la maintenance prédictive
Module	MLOps et DataOps
Encadrant(e)	Mr AITDAOUD Mohammed
Année universitaire	2025–2026
Date de dépôt	17 septembre 2026
Établissement	Faculté des Sciences Ben M’Sick — Université Hassan II de Casablanca
Équipe	Rahil (Product Owner), Amina (Scrum Master), Imane & Zineb (Data Engineers), Sara (Analytics / Data Quality Engineer), Maroua (ML Engineer), Raxida (MLOps Engineer), Israa (DevOps / API Engineer)
Problématique	Déterminer, à partir de relevés capteurs multivariés issus d’un parc de cinquante machines industrielles, si une machine présente un risque de défaillance imminente nécessitant une intervention de maintenance.
Objectifs	Prédire les défaillances, réduire les arrêts non planifiés, outiller les équipes de maintenance via un service exploitable (API).
Jeu de données	smart_manufacturing_data.csv — 100 000 relevés capteurs, 50 machines, janvier–mars 2025
Stack technique	dlt, DuckDB, dbt, Dagster, scikit-learn, MLflow, FastAPI, Docker, GitHub Actions, Komodo
Résultat clé	19,7 % des relevés analysés correspondent à un besoin de maintenance avéré ; modèle retenu : Gradient Boosting Classifier ; service déployé et accessible publiquement
Organisation	Méthodologie Scrum — 3 sprints d’une semaine (Sprint Planning, Sprint Review, Sprint Retrospective)

Chapitre 1: Vision du projet, Data Strategy et Agile

1.1 Contexte général

La transformation numérique du secteur industriel, communément désignée sous le terme d'Industrie 4.0, repose sur l'interconnexion croissante des équipements de production par le biais de capteurs et de dispositifs IoT (température, vibration, pression, humidité, consommation énergétique). Cette instrumentation généralisée des lignes de production génère des volumes de données considérables, jusqu'alors inexploités, qui constituent aujourd'hui un levier stratégique pour repenser les politiques de maintenance industrielle.

Historiquement, les stratégies de maintenance se sont succédé selon trois grands paradigmes. La **maintenance corrective** consiste à intervenir après la survenue d'une panne, ce qui expose l'entreprise à des arrêts de production non planifiés et à des coûts de réparation élevés. La **maintenance préventive systématique** propose de réviser les équipements à intervalles fixes, indépendamment de leur état réel, ce qui conduit fréquemment à des interventions superflues ou, à l'inverse, insuffisantes. La **maintenance prédictive**, objet du présent travail, s'appuie sur l'analyse continue des données capteurs et sur des modèles de Machine Learning afin d'anticiper la dégradation d'un équipement avant que la panne ne survienne, permettant une intervention ciblée, au moment opportun.

C'est dans cette perspective que s'inscrit le présent chapitre, qui pose les fondations fonctionnelles et stratégiques de la solution développée : la problématique traitée, les objectifs poursuivis, les utilisateurs concernés, la valeur métier attendue, ainsi que la stratégie de gestion des données et l'organisation Agile mise en place pour piloter les travaux.

1.2 Problématique

1.2.1 Pourquoi la maintenance prédictive est-elle un enjeu stratégique ?

Un arrêt de production non planifié affecte directement plusieurs indicateurs de performance industrielle, dont le taux de rendement synthétique (TRS) et le temps moyen entre deux défaillances (MTBF). Au-delà du coût direct de la réparation, l'indisponibilité d'une machine engendre des retards en cascade sur l'ensemble de la chaîne de production,

une mobilisation d'urgence des équipes techniques et, dans certains cas, des risques pour la sécurité des opérateurs. La maintenance prédictive répond à cet enjeu en substituant à une logique réactive une logique anticipative, fondée sur l'exploitation systématique des données de fonctionnement des équipements.

1.2.2 Quel problème industriel cette étude cherche-t-elle à résoudre ?

La problématique centrale consiste à déterminer, à partir de relevés capteurs multivariés issus d'un parc de cinquante machines industrielles (température, vibration, humidité, pression, consommation énergétique), si une machine présente un risque de défaillance imminente nécessitant une intervention de maintenance. Il s'agit donc de transformer un flux continu de données brutes, hétérogènes et volumineux, en une information de décision directement exploitable par les équipes opérationnelles.

1.2.3 Quels risques sont associés aux défaillances machines ?

- Interruptions de production non planifiées, avec répercussions sur les délais de livraison et les engagements contractuels.
- Surcoûts liés aux interventions d'urgence et à l'approvisionnement non anticipé de pièces de rechange.
- Risques accrus pour la sécurité des opérateurs en cas de défaillance mécanique, thermique ou électrique.
- Effets de propagation : la défaillance non détectée d'un composant peut entraîner la dégradation accélérée d'autres sous-systèmes.
- Érosion de la confiance des équipes de production envers la fiabilité de l'outil industriel.

1.3 Objectifs du projet

Les objectifs poursuivis se déclinent en trois axes opérationnels, cohérents avec les besoins identifiés auprès des équipes de maintenance et de production.

- **Prédire les défaillances** : concevoir un modèle de classification supervisée capable de déterminer, à partir des données capteurs, si une machine nécessite une intervention de maintenance, avec un niveau de confiance quantifié.
- **Réduire les arrêts non planifiés** : identifier les signaux précurseurs d'une défaillance suffisamment en amont pour permettre une planification de l'intervention plutôt qu'une réaction dans l'urgence.

- **Outiller les équipes de maintenance** : mettre à disposition une information exploitable — probabilité de défaillance, type de panne le plus probable — au travers d'un service accessible (API), plutôt qu'un modèle statistique isolé et difficilement actionnable.

1.4 Utilisateurs cibles

L'analyse fonctionnelle a permis d'identifier quatre profils d'utilisateurs directement concernés par la solution, dont les besoins et les attentes diffèrent sensiblement.

Utilisateur	Besoin principal	Attente vis-à-vis du système
Responsable maintenance	Anticiper les défaillances afin d'organiser les interventions de manière optimale	Fiabilité des alertes, probabilité de panne explicite, délai d'anticipation suffisant
Technicien industriel	Identifier la machine à inspecter en priorité et comprendre la nature du risque	Interface simple d'utilisation, indication du type de panne probable
Responsable production	Limiter les interruptions non planifiées qui affectent la continuité de production	Visibilité sur la disponibilité prévisionnelle des équipements
Équipe data	Disposer d'un pipeline de données fiable, versionné et supervisable dans la durée	Qualité des données garantie, traçabilité complète, modèle réévaluable

1.5 Valeur métier

La valeur métier générée par la solution s'apprécie selon quatre dimensions complémentaires, directement liées aux enjeux de performance industrielle évoqués en introduction.

- **Réduction du temps d'arrêt** : en anticipant les défaillances, les interventions sont planifiées plutôt que subies, ce qui réduit mécaniquement la durée cumulée d'indisponibilité des équipements.
- **Optimisation de la planification de maintenance** : les équipes peuvent prioriser leurs interventions selon un niveau de risque réel et quantifié, plutôt que selon un calendrier fixe déconnecté de l'état effectif des machines.

- **Réduction des coûts opérationnels** : diminution des réparations d’urgence, des commandes de pièces en flux tendu et des pertes de production associées aux arrêts non planifiés.
- **Amélioration de la disponibilité du parc machines** : l’analyse exploratoire du jeu de données réalisée dans le cadre de cette étude révèle que **19,7 %** des 100 000 relevés analysés correspondent à un besoin de maintenance avéré — une proportion qui justifie pleinement la mise en place d’un dispositif de priorisation automatisé, plutôt qu’un contrôle manuel exhaustif du parc de cinquante machines.

1.6 Data Strategy

1.6.1 Source et nature des données

L’étude s’appuie sur un jeu de données de supervision industrielle (`smart_manufacturing_data.csv`) constitué de 100 000 relevés capteurs, collectés à une fréquence d’une minute entre janvier et mars 2025, sur un parc de cinquante machines instrumentées. Chaque observation correspond à un relevé multi-capteurs instantané pour une machine donnée, associé à un indicateur de besoin de maintenance.

Variable	Type	Description
timestamp	date	Horodatage du relevé capteur (résolution : la minute).
machine_id	entier	Identifiant de la machine (50 machines suivies).
temperature	continu	Température mesurée par le capteur.
vibration	continu	Amplitude de vibration mesurée. Des valeurs négatives, physiquement improbables, ont été identifiées et devront être traitées.
humidity	continu	Taux d’humidité ambiante.
pressure	continu	Pression mesurée.
energy_consumption	continu	Consommation énergétique instantanée de la machine.
machine_status	catégoriel (0/1/2)	État opérationnel de la machine.

Variable	Type	Description
<code>anomaly_flag</code>	binaire	Anomalie détectée par le système de supervision.
<code>predicted_remaining_life</code>	discret	Durée de vie résiduelle estimée.
<code>failure_type</code>	catégoriel	Type de défaillance : Normal, Vibration Issue, Overheating, Pressure Drop et Electrical Fault.
<code>downtime_risk</code>	binaire	Indicateur de risque d'arrêt machine.
<code>maintenance_required</code>	binaire (cible)	Variable cible du modèle : intervention nécessaire (1) ou fonctionnement normal (0).

1.6.2 Architecture de la stratégie data

La stratégie data adoptée repose sur une architecture DataOps/MLOps structurée en couches successives, garantissant reproductibilité, traçabilité et qualité à chaque étape du cycle de vie de la donnée :

- **Ingestion (dlt)** : automatisation du chargement des données brutes vers une couche *raw*, garantissant la reproductibilité de l'intégration.
- **Stockage (DuckDB)** : base analytique organisée en couches *raw*, *cleaned*, *features* et *ML*, assurant une séparation claire des responsabilités de transformation.
- **Transformation (dbt)** : nettoyage, typage et construction de modèles de données versionnés et documentés.
- **Qualité et gouvernance (tests dbt, Data Contract)** : contrôle systématique de la non-nullité, de l'unicité et de la validité des valeurs, avec une attention particulière portée à la variable cible.
- **Orchestration (Dagster)** : automatisation de bout en bout de la chaîne ingestion-stockage-transformation-modélisation.
- **Modélisation (Machine Learning)** : entraînement et évaluation comparative de plusieurs modèles de classification binaire.
- **Suivi des expériences (MLflow)** : traçabilité des paramètres, des métriques et des artefacts, versionnement du modèle retenu.
- **Mise à disposition (FastAPI, Docker, CI/CD)** : exposition du modèle sous la forme d'un service testable, conteneurisé et intégré à une chaîne d'intégration

continue.

1.6.3 Gouvernance et qualité des données

La variable cible retenue, `maintenance_required`, présente l'avantage d'être directement corroborée par la variable `failure_type`, qui détaille la nature de la défaillance (Vibration Issue, Overheating, Pressure Drop, Electrical Fault) lorsqu'une intervention est requise, offrant ainsi une double lecture — binaire et multi-classes — du phénomène étudié. L'analyse exploratoire a par ailleurs mis en évidence un point de vigilance : la présence de valeurs négatives sur la variable `vibration`, physiquement incohérentes, qui devra faire l'objet d'un test de validité formalisé dans le Data Contract, conformément aux principes de gouvernance de la donnée adoptés pour cette étude.

1.7 Organisation Agile du projet

Le pilotage des travaux repose sur une organisation Scrum, instrumentée via l'outil Jira, et structurée en trois sprints d'une semaine chacun, intégrant les cérémonies Agile classiques : Sprint Planning, Sprint Review et Sprint Retrospective.

1.7.1 Rôles Scrum de l'équipe

Rôle Scrum	Membre(s)	Responsabilité principale
Product Owner	Rahil	Vision produit, priorisation du backlog, relation avec les parties prenantes métier
Scrum Master	Amina	Animation des cérémonies Agile, suivi de l'avancement, résolution des blocages
Data Engineer	Imane, Zineb	Ingestion (dlt) et stockage des données (DuckDB)
Analytics / Data Quality Engineer	Sara	Transformations dbt, tests qualité, Data Contract
ML Engineer	Maroua	Feature engineering, entraînement et évaluation des modèles
MLOps Engineer	Raxida	Suivi des expériences (MLflow), Model Registry, monitoring

Rôle Scrum	Membre(s)	Responsabilité principale
DevOps / API Engineer	Israe	API FastAPI, conteneurisation, intégration continue, déploiement

1.7.2 Planning des sprints

Sprint	Dates	Objectif	Contenu principal	Charge
Sprint 1	29 juin – 6 juillet	Préparation et données	Vision, backlog, analyse exploratoire, ingestion (dlt), stockage (DuckDB), premières transformations (dbt), architecture et configuration GitHub, sélection des modèles candidats, préparation de l'environnement MLflow/Docker	8 tickets / 35 pts
Sprint 2	6 – 13 juillet	Pipeline DataOps et Machine Learning	Tests qualité et Data Contract, orchestration (Dagster), entraînement et évaluation du modèle, suivi des expériences (MLflow), première version de l'API	5 tickets / 31 pts
Sprint 3	12 – 19 juillet	Industrialisation et restitution	Model Registry et monitoring, conteneurisation, intégration continue, exposition des points d'accès <code>/predict</code> et <code>/health</code> , rapport final et démonstration	3 tickets / 18 pts

1.7.3 Cérémonies Agile

- **Sprint Planning** : sélection des User Stories du backlog pour le sprint considéré, estimation en points et répartition des responsabilités.
- **Sprint Review** : présentation des livrables produits au cours du sprint (pipeline, modèle, service) à l'ensemble de l'équipe.

- **Sprint Retrospective** : bilan critique du sprint écoulé et définition d’actions d’amélioration pour le sprint suivant.

1.7.4 Board Agile

Le suivi opérationnel des tâches est assuré au moyen d’un tableau Kanban structuré en trois colonnes — *à faire, en cours, terminé* —, dont une illustration est proposée en section 1.8.

1.8 Product Backlog et User Stories

Le Product Backlog rassemble dix-neuf User Stories réparties sur les trois sprints décrits précédemment, hiérarchisées selon la méthode MoSCoW (*Must have, Should have*).

1.8.1 Product Backlog

ID	Épopée	User Story	Prio.	Sprin	Owner	Pts
US-01	Vision & Cadrage	Comprendre l’objectif de l’étude	Must	S1	Rahil	3
US-02	Vision & Cadrage	Définir le Product Backlog et les priorités	Must	S1	Rahil	3
US-03	Vision & Cadrage	Réaliser une première analyse exploratoire du jeu de données	Must	S1	Rahil	5
US-04	Ingestion (dlt)	Identifier et connecter la source de données	Must	S1	Imane	3
US-05	Ingestion (dlt)	Développer le pipeline d’ingestion avec dlt	Must	S1	Imane	8
US-06	Stockage (DuckDB)	Mettre en place la base DuckDB	Must	S1	Zineb	5
US-07	Transformation (dbt)	Réaliser les premières transformations dbt	Must	S1	Sara	5
US-08	Architecture	Définir l’architecture globale et configurer GitHub	Must	S1	Amina	3

ID	Épopée	User Story	Prio.	Sprin	Owner	Pts
US-09	Machine Learning	Sélectionner et comparer les modèles candidats	Should	S1	Maroua	3
US-10	MLOps (MLflow)	Préparer l'environnement MLflow et Docker	Should	S1	Raxida	3
US-11	Déploiement (Docker)	Préparer un environnement Docker de base	Should	S1	Israe	3
US-12	Qualité des données	Développer les tests qualité et le Data Contract	Must	S2	Sara	5
US-13	Orchestration (Dagster)	Orchestrer le pipeline avec Dagster	Must	S2	Amina	8
US-14	Machine Learning	Entraîner et évaluer le modèle	Must	S2	Maroua	8
US-15	MLOps (MLflow)	Assurer le suivi des expériences avec MLflow	Must	S2	Raxida	5
US-16	Déploiement (API)	Développer une première version de l'API	Must	S2	Israe	5
US-17	MLOps (MLflow)	Mettre en place le Model Registry et le monitoring	Must	S3	Raxida	5
US-18	Déploiement (API/Docker)	Conteneuriser et automatiser le déploiement (CI/CD)	Must	S3	Israe	8
US-19	Livrable final	Finaliser le rapport, la documentation et la démonstration	Must	S3	Équipe	5

Légende de priorité : **Must** = priorité indispensable — **Should** = priorité importante.

1.8.2 Tableau des User Stories détaillées

Chaque User Story est formalisée selon le gabarit standard « En tant que [rôle], je veux [besoin], afin de [bénéfice] », accompagné d'un critère d'acceptation vérifiable.

ID	En tant que ... je veux ... afin de ...	Critère d'acceptation	Pts
US-01	En tant que responsable maintenance, je veux comprendre la finalité de la démarche, afin de savoir comment le système contribuera à la planification des interventions. (Rahil)	Document de vision rédigé et validé par l'équipe	3
US-02	En tant que Product Owner, je veux disposer d'un Product Backlog priorisé, afin que l'équipe sache quoi développer en priorité. (Rahil)	Backlog complet, priorisé (MoSCoW) et partagé dans Jira	3
US-03	En tant que membre de l'équipe data, je veux réaliser une première analyse exploratoire du jeu de données, afin d'en évaluer la qualité, la structure et la variable cible avant ingestion. (Rahil)	Rapport d'analyse exploratoire : 100 000 observations, 13 variables, absence de valeurs manquantes, variable cible binaire identifiée	5
US-04	En tant que data engineer, je veux identifier et connecter la source de données, afin de disposer d'une source fiable pour le pipeline. (Imane)	Source de données validée et accessible	3
US-05	En tant que data engineer, je veux développer un pipeline dlt, afin d'automatiser le chargement des données brutes dans DuckDB. (Imane)	Script dlt fonctionnel, données visibles dans la couche raw	8
US-06	En tant que data engineer, je veux mettre en place la base DuckDB, afin de structurer le stockage local des données. (Zineb)	Base DuckDB créée et accessible en lecture/écriture	5
US-07	En tant qu'analytics engineer, je veux réaliser les premières transformations dbt, afin de préparer un jeu de données exploitable. (Sara)	Modèles dbt de type <i>staging</i> exécutés sans erreur	5

ID	En tant que ... je veux ... afin de ...	Critère d'acceptation	Pts
US-08	En tant que DataOps engineer, je veux définir l'architecture globale et configurer le dépôt GitHub, afin de garantir une base de collaboration structurée. (Amina)	Dépôt GitHub structuré, documentation initiale, stratégie de branches définie	3
US-09	En tant que ML engineer, je veux sélectionner et comparer des modèles candidats, afin de préparer la phase d'entraînement. (Maroua)	Liste de modèles candidats documentée (régression logistique, forêts aléatoires, XGBoost)	3
US-10	En tant que MLOps engineer, je veux préparer l'environnement MLflow et Docker, afin d'être en mesure de suivre les expériences dès le sprint suivant. (Raxida)	Environnement MLflow installé et validé	3
US-11	En tant que DevOps engineer, je veux préparer un environnement Docker de base, afin de faciliter l'intégration ultérieure du service. (Israe)	Dockerfile de base fonctionnel	3
US-12	En tant que data quality engineer, je veux développer des tests qualité et un Data Contract, afin de garantir la fiabilité des données en amont de la modélisation. (Sara)	Tests dbt validés, Data Contract documenté	5
US-13	En tant que DataOps engineer, je veux orchestrer le pipeline complet avec Dagster, afin d'automatiser la chaîne ingestion-stockage-transformation. (Amina)	Pipeline exécutable et visible dans l'interface Dagster	8
US-14	En tant que ML engineer, je veux entraîner et évaluer le modèle, afin d'obtenir un système fiable de prédiction des défaillances. (Maroua)	Précision, rappel, F1-score et AUC-ROC calculés pour plusieurs modèles	8

ID	En tant que ... je veux ... afin de ...	Critère d'acceptation	Pts
US-15	En tant que MLOps engineer, je veux assurer le suivi des expériences avec MLflow, afin de comparer les modèles et leurs performances. (Raxida)	Paramètres, métriques et artefacts consignés pour chaque exécution	5
US-16	En tant que DevOps engineer, je veux développer une première version de l'API, afin d'exposer le modèle sous forme de service testable. (Israe)	Point d'accès <code>/predict</code> fonctionnel en environnement local	5
US-17	En tant que MLOps engineer, je veux enregistrer le modèle dans le Model Registry et mettre en place un monitoring, afin de garantir la traçabilité et la détection d'une dérive. (Raxida)	Modèle versionné, comparaison de distributions documentée	5
US-18	En tant que DevOps engineer, je veux conteneuriser l'API et automatiser son déploiement, afin de garantir un service fiable et reproductible. (Israe)	Intégration continue opérationnelle, points d'accès <code>/predict</code> et <code>/health</code> répondant correctement	8
US-19	En tant qu'équipe projet, je veux finaliser le rapport et la démonstration, afin de présenter un système complet et fonctionnel lors de la restitution. (Équipe)	Rapport, documentation et démonstration validés	5

1.8.3 Illustration — Product Backlog

La figure suivante présente une vue consolidée du Product Backlog, organisée selon les trois sprints décrits précédemment (Sprint 1 : 8 tickets, 35 points ; Sprint 2 : 5 tickets, 31 points ; Sprint 3 : 3 tickets, 18 points).

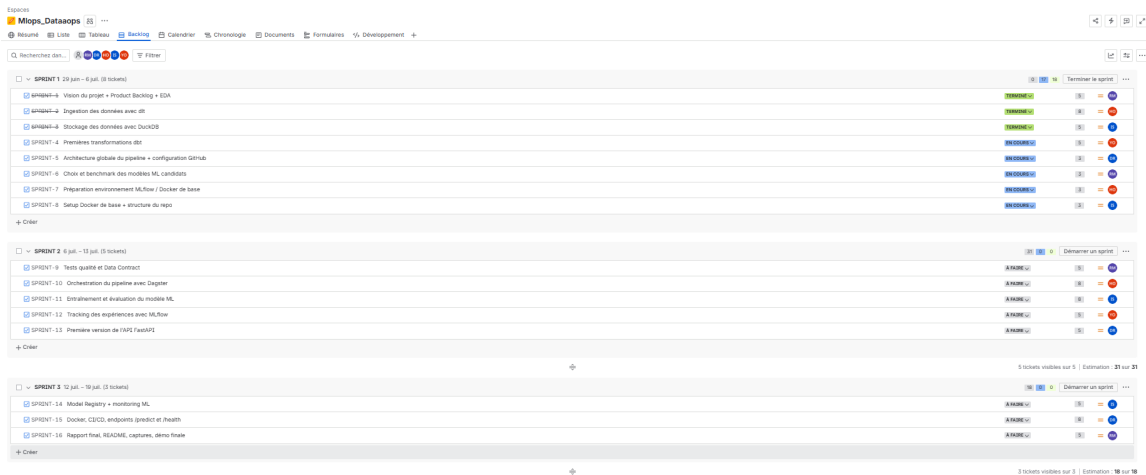


FIGURE 1.1 – Vue consolidée du Product Backlog

1.8.4 Illustration — Board Agile

Le tableau Kanban ci-dessous illustre l'état d'avancement du premier sprint, structuré selon les colonnes *à faire*, *en cours* et *terminé*.

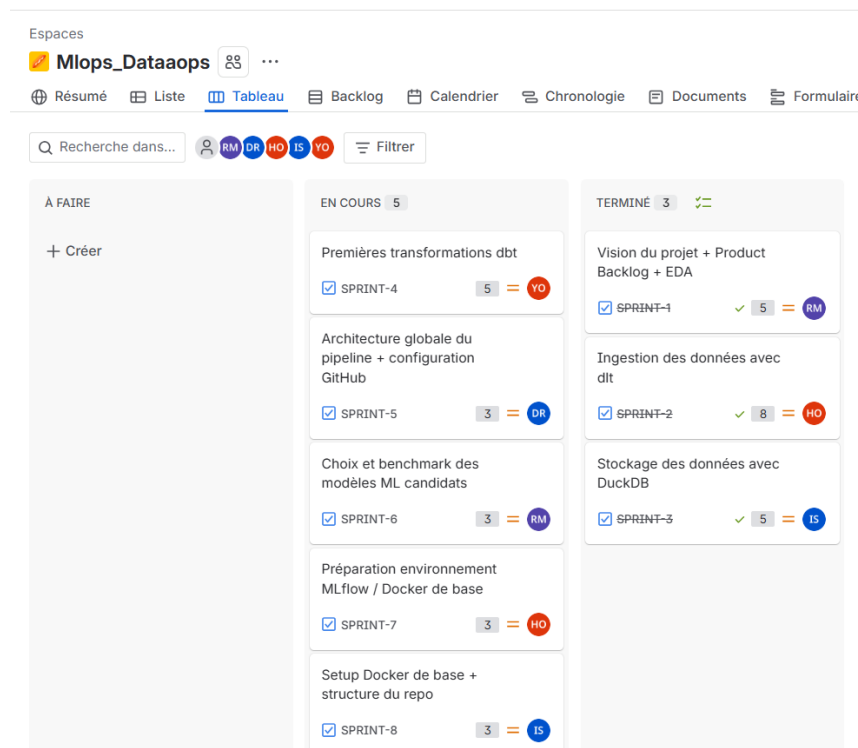
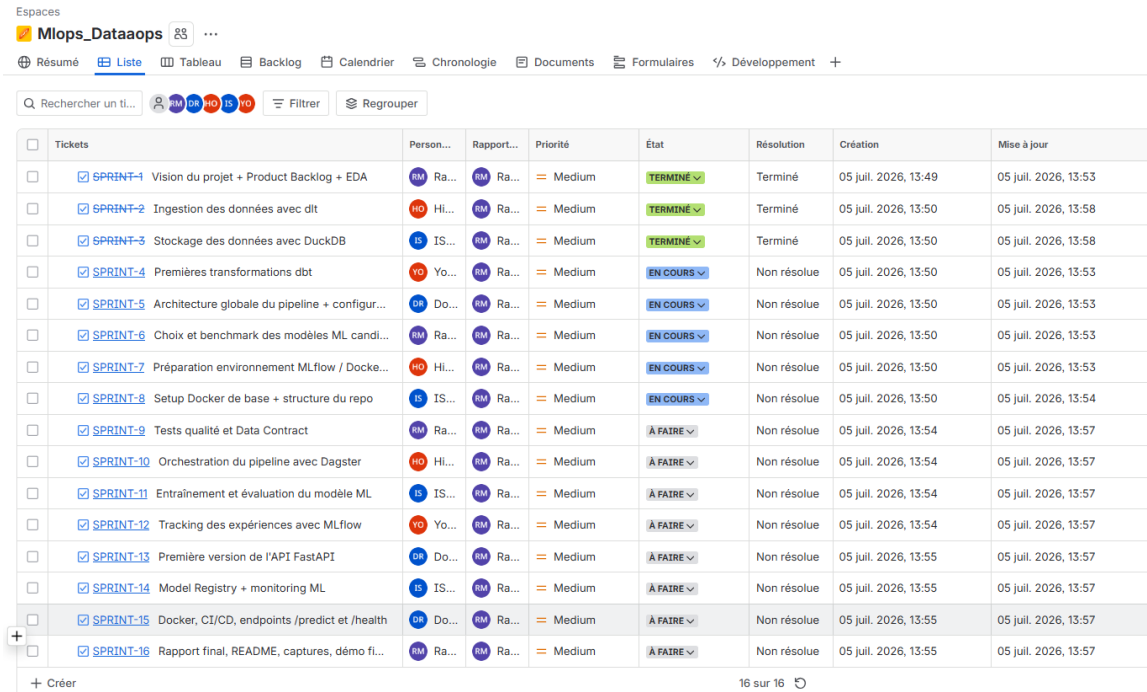


FIGURE 1.2 – Board Agile — suivi du Sprint 1

1.8.5 Illustration — Tableau des User Stories

La figure suivante présente l'ensemble des User Stories, avec leur affectation, leur priorité et leur statut d'avancement.



Tickets	Person...	Rapport...	Priorité	État	Résolution	Création	Mise à jour
☐ SPRINT-1 Vision du projet + Product Backlog + EDA	Ra...	Ra...	Medium	TERMINÉ	Terminé	05 juil. 2026, 13:49	05 juil. 2026, 13:53
☐ SPRINT-2 Ingestion des données avec dlt	Hi...	Ra...	Medium	TERMINÉ	Terminé	05 juil. 2026, 13:50	05 juil. 2026, 13:58
☐ SPRINT-3 Stockage des données avec DuckDB	IS...	Ra...	Medium	TERMINÉ	Terminé	05 juil. 2026, 13:50	05 juil. 2026, 13:58
☐ SPRINT-4 Premières transformations dbt	Yo...	Ra...	Medium	EN COURS	Non résolue	05 juil. 2026, 13:50	05 juil. 2026, 13:53
☐ SPRINT-5 Architecture globale du pipeline + configur...	Do...	Ra...	Medium	EN COURS	Non résolue	05 juil. 2026, 13:50	05 juil. 2026, 13:53
☐ SPRINT-6 Choix et benchmark des modèles ML candi...	Ra...	Ra...	Medium	EN COURS	Non résolue	05 juil. 2026, 13:50	05 juil. 2026, 13:53
☐ SPRINT-7 Préparation environnement MLflow / Docke...	Hi...	Ra...	Medium	EN COURS	Non résolue	05 juil. 2026, 13:50	05 juil. 2026, 13:53
☐ SPRINT-8 Setup Docker de base + structure du repo	IS...	Ra...	Medium	EN COURS	Non résolue	05 juil. 2026, 13:50	05 juil. 2026, 13:54
☐ SPRINT-9 Tests qualité et Data Contract	Ra...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:54	05 juil. 2026, 13:57
☐ SPRINT-10 Orchestration du pipeline avec Dagster	Hi...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:54	05 juil. 2026, 13:57
☐ SPRINT-11 Entraînement et évaluation du modèle ML	IS...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:54	05 juil. 2026, 13:57
☐ SPRINT-12 Tracking des expériences avec MLflow	Yo...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:54	05 juil. 2026, 13:57
☐ SPRINT-13 Première version de l'API FastAPI	Do...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:55	05 juil. 2026, 13:57
☐ SPRINT-14 Model Registry + monitoring ML	IS...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:55	05 juil. 2026, 13:57
☐ SPRINT-15 Docker, CI/CD, endpoints /predict et /health	Do...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:55	05 juil. 2026, 13:57
☐ SPRINT-16 Rapport final, README, captures, démo fi...	Ra...	Ra...	Medium	À FAIRE	Non résolue	05 juil. 2026, 13:55	05 juil. 2026, 13:57

FIGURE 1.3 – Tableau consolidé des User Stories

1.8.6 Illustration — Analyse exploratoire du jeu de données

La figure ci-dessous restitue les résultats de l'analyse exploratoire menée sur le jeu de données (100 000 observations, 13 variables), confirmant l'absence de valeurs manquantes ainsi que la distribution de la variable cible.

```
In [1]: import pandas as pd
df = pd.read_csv("smart_manufacturing_data.csv")

print("Nombre de lignes et colonnes :", df.shape)
print()
print("Valeurs manquantes par colonne :")
print(df.isnull().sum())
print()
print("Distribution de la variable cible :")
print(df["maintenance_required"].value_counts())

Out[1]: Nombre de lignes et colonnes : (100000, 13)

Valeurs manquantes par colonne :
timestamp      0
machine_id      0
temperature     0
vibration       0
humidity        0
pressure        0
energy_consumption  0
machine_status  0
anomaly_flag    0
predicted_remaining_life  0
failure_type    0
downtime_risk   0
maintenance_required  0
dtype: int64

Distribution de la variable cible :
maintenance_required
0    88383
1    19697
Name: count, dtype: int64
```

FIGURE 1.4 – Résultats de l’analyse exploratoire du jeu de données

1.9 Conclusion du chapitre

Ce premier chapitre a permis de poser les fondations fonctionnelles et stratégiques du travail mené. La problématique a été clairement circonscrite : passer d’une logique de maintenance réactive ou systématique à une logique prédictive, fondée sur l’exploitation des données capteurs issues d’un parc de cinquante machines industrielles. Les objectifs poursuivis — prédire les défaillances, réduire les arrêts non planifiés et outiller les équipes de maintenance — ont été définis en cohérence avec les besoins des quatre profils d’utilisateurs identifiés.

L’analyse de la valeur métier a confirmé la pertinence économique de la démarche, la proportion de relevés nécessitant une intervention (19,7 %) justifiant la mise en place d’un dispositif de priorisation automatisé. La stratégie data définie — articulée autour d’une architecture DataOps/MLOps complète (ingestion, stockage, transformation, orchestration, modélisation, suivi des expériences et déploiement) — constitue le socle technique sur lequel s’appuieront l’ensemble des travaux ultérieurs.

Enfin, l’organisation Agile mise en place, structurée en trois sprints d’une semaine et formalisée au travers d’un Product Backlog de dix-neuf User Stories, garantit un pilotage itératif et transparent de l’ensemble de l’équipe. Cette vision d’ensemble constitue le point

de départ des chapitres suivants, qui détailleront successivement chacune des briques techniques de la solution : l'ingestion des données (chapitre 2), leur stockage et leur modélisation (chapitre 3), leur transformation et le contrôle de leur qualité (chapitre 4), l'orchestration du pipeline (chapitre 5), la modélisation Machine Learning (chapitre 6), le suivi et le versionnement du modèle (chapitre 7), ainsi que son déploiement (chapitre 8).

Chapitre 2: Ingestion des données avec dlt

2.1 Introduction

L’ingestion des données constitue la première étape du pipeline MLOps. Elle consiste à collecter les données brutes depuis leur source et à les charger dans un système de stockage afin de les rendre disponibles pour les étapes de transformation, d’analyse et de modélisation. Dans ce projet, cette étape est réalisée à l’aide de dlt (Data Load Tool), qui permet d’automatiser le chargement du jeu de données dans une base de données DuckDB. Ce chapitre présente le jeu de données utilisé, l’implémentation du pipeline d’ingestion ainsi que la vérification du chargement des données dans la couche Raw.

2.2 Présentation du jeu de données

Le jeu de données utilisé dans ce projet provient de la plateforme Kaggle et est intitulé *Smart Manufacturing IoT Cloud Monitoring Dataset*. Il regroupe des données simulant le fonctionnement de machines industrielles connectées dans un environnement de fabrication intelligente (*Smart Manufacturing*). Ce jeu de données contient 100 000 observations collectées sur 50 machines industrielles. Chaque observation correspond à un instant précis et décrit l’état de fonctionnement d’une machine à travers plusieurs mesures issues de capteurs ainsi que des informations relatives à la maintenance. Le dataset est composé de **13 variables**, dont une variable cible utilisée pour prédire si une intervention de maintenance est nécessaire.

La variable cible est :

`maintenance_required`

Cette variable est binaire :

- **0** : aucune maintenance n’est requise ;
- **1** : une intervention de maintenance est nécessaire.

Les différentes variables décrivant le fonctionnement des machines sont présentées dans le Tableau [2.1](#).

TABLE 2.1 – Description des variables du jeu de données

Variable	Description
timestamp	Date et heure de l'enregistrement des données.
machine_id	Identifiant unique de la machine industrielle.
temperature	Température mesurée par les capteurs de la machine (°C).
vibration	Niveau de vibration de la machine durant son fonctionnement.
humidity	Taux d'humidité de l'environnement.
pressure	Pression mesurée pendant le fonctionnement de la machine.
energy_consumption	Consommation énergétique de la machine.
machine_status	État de fonctionnement de la machine (Idle, Running ou Failure).
anomaly_flag	Indicateur signalant la présence ou non d'une anomalie.
predicted_remaining_life	Estimation de la durée de vie restante de la machine.
failure_type	Type de panne détectée lorsque celle-ci est présente.
downtime_risk	Niveau de risque d'arrêt de la machine.
maintenance_required	Variable cible indiquant si une maintenance est nécessaire (0 ou 1).

Grâce à la diversité des mesures collectées, ce jeu de données est particulièrement adapté à la mise en œuvre d'un système de maintenance prédictive. Avant d'être exploité dans les étapes suivantes du pipeline MLOps, il est chargé dans la couche Raw de la base de données DuckDB à l'aide du pipeline d'ingestion développé avec dlt.

2.3 Pipeline d'ingestion

Le pipeline d'ingestion développé dans ce projet permet d'automatiser le chargement des données brutes depuis un fichier CSV vers une base de données DuckDB. Cette étape constitue la première phase du pipeline MLOps et garantit que les données sont stockées

de manière fiable et reproductible avant les étapes de transformation et de modélisation. L'architecture générale du pipeline suit les étapes suivantes :

Fichier CSV $\longrightarrow$ `sources.py` $\longrightarrow$ Pipeline `dlt` $\longrightarrow$ DuckDB

Le fonctionnement du pipeline est décrit comme suit :

- **Fichier CSV** : Les données brutes sont stockées dans le fichier `smart_manufacturing_data.csv`, qui contient les différentes mesures collectées par les capteurs des machines industrielles.
- **Script `sources.py`** : Ce script est responsable de la lecture du fichier CSV. Il définit une ressource `dlt` permettant de charger les données par lots (*chunks*) afin d'améliorer les performances et de limiter l'utilisation de la mémoire lors du traitement de grands volumes de données.
- **Pipeline `dlt`** : Le pipeline créé avec `dlt` orchestre le processus d'ingestion. Il détecte automatiquement le schéma des données, prépare les tables nécessaires et assure le chargement des enregistrements vers la base de données de destination.
- **Base de données DuckDB** : Les données sont finalement stockées dans la base DuckDB, au sein du schéma `raw`. Cette couche représente les données brutes du projet et constitue le point de départ des étapes suivantes, notamment le nettoyage, la transformation des données et l'entraînement des modèles de maintenance prédictive.

Grâce à cette architecture, le pipeline d'ingestion offre une solution automatisée, reproductible et efficace pour le chargement des données. Il facilite la gestion des jeux de données industriels et assure leur disponibilité pour l'ensemble des étapes du workflow MLOps.

2.4 Implémentation des scripts d'ingestion

Cette section présente les deux scripts principaux développés pour réaliser le pipeline d'ingestion des données. Le premier script, `sources.py`, est chargé de lire le jeu de données et de le préparer pour l'ingestion. Le second, `load_data.py`, configure le pipeline `dlt` et orchestre le chargement des données dans la base de données DuckDB.

2.4.1 Définition de la source de données (`sources.py`)

Le fichier `sources.py` constitue la première composante du pipeline d'ingestion. Son rôle est de définir la source de données qui sera exploitée par le pipeline `dlt`. Il est chargé de localiser le fichier CSV contenant les données brutes, de vérifier son existence, puis de lire son contenu. Afin d'optimiser les performances et de limiter l'utilisation de la

mémoire, les données sont lues de manière progressive, par lots (*chunks*), plutôt qu'en une seule opération. Chaque lot est ensuite converti dans un format compatible avec dlt, ce qui permet son intégration automatique dans le pipeline d'ingestion. Enfin, le script déclare une ressource dlt représentant le jeu de données. Cette ressource constitue le point d'entrée du pipeline et permet de transmettre les données au processus de chargement de manière structurée et reproductible.

2.4.2 Exécution du pipeline d'ingestion (`load_data.py`)

Le fichier `load_data.py` assure l'exécution complète du pipeline d'ingestion. Il configure un pipeline dlt en précisant son nom, la destination de stockage ainsi que le schéma de destination correspondant à la couche *Raw*. Une fois le pipeline configuré, le script lance automatiquement le chargement des données définies dans le fichier `sources.py`. Les données sont alors importées dans la base de données DuckDB, où elles sont stockées sans modification afin de préserver leur intégrité. À l'issue de l'exécution, le script affiche un résumé de l'opération d'ingestion, comprenant notamment le nombre de données chargées, l'état du pipeline ainsi que les éventuelles erreurs détectées. Cette étape permet de vérifier que le processus d'ingestion s'est déroulé correctement avant de passer aux phases suivantes du pipeline MLOps.

2.5 Vérification de l'ingestion

Après le développement du pipeline d'ingestion, une phase de vérification a été réalisée afin de s'assurer que les données ont été correctement chargées dans la base de données DuckDB. Cette étape permet de valider le bon fonctionnement de l'ensemble du processus avant de passer aux phases suivantes du pipeline MLOps. L'exécution du pipeline est effectuée depuis le terminal à l'aide de la commande suivante :

```
python ingestion/load_data.py
```

L'exécution du script confirme que le pipeline dlt s'est déroulé avec succès. Les données ont été automatiquement importées dans la base de données DuckDB, au sein du schéma `raw`, sans qu'aucune erreur ne soit détectée. Le message affiché dans le terminal indique également que la base de données `maintenance_predictive_pipeline.duckdb` a été créée avec succès et que l'ensemble des 100 000 enregistrements du jeu de données a été chargé dans la table `raw_sensor_data`. Afin de confirmer l'intégrité des données importées, un script de vérification (`check_data.py`) a ensuite été exécuté. Ce script permet notamment de contrôler le nombre d'enregistrements présents dans la base ainsi que d'afficher un aperçu des premières observations chargées.

Les résultats obtenus confirment que l'intégralité du jeu de données a été importée avec succès, garantissant ainsi une base de données fiable pour les étapes de transformation, de validation et de modélisation qui seront présentées dans les chapitres suivants.

```
PS C:\Users\lenovo\Downloads\predictive-maintenance> python ingestion/load_data.py
Démarrage de l'ingestion des données capteurs...

Ingestion terminée avec succès.
Pipeline maintenance_predictive_pipeline load step completed in 20.64 seconds
1 load package(s) were loaded to destination duckdb and into dataset raw
The duckdb destination used duckdb:///C:\Users\lenovo\Downloads\predictive-maintenance\maintenance_predictive_pipeline.duckdb location to store data
Load package 1783173733.7282221 is LOADED and contains no failed jobs
PS C:\Users\lenovo\Downloads\predictive-maintenance>
```

FIGURE 2.1 – Exécution du pipeline d'ingestion avec dlt.

La Figure 2.1 présente le résultat de l'exécution du pipeline d'ingestion développé avec dlt. Les messages affichés dans le terminal indiquent que le pipeline s'est exécuté avec succès et que les données du fichier CSV ont été chargées dans la base de données DuckDB. Les informations retournées précisent également le nom du pipeline, la destination de stockage, le schéma *raw* ainsi que l'absence d'erreurs lors du chargement, confirmant ainsi le bon fonctionnement du processus d'ingestion.

```
PS C:\Users\lenovo\Downloads\predictive-maintenance> python check_data.py
=====
count_star()
0
100000
=====
APERÇU DES DONNÉES
=====
```

	timestamp	machine_id	temperature	vibration	humidity	pressure	energy_consumption	machine_status	anomaly_flag	predicted_remaining_life	failure_type	downtime_risk	maintenance_required	dlt_load_id	dlt_id	
0	2025-01-01 01:00:00+01:00	39	78.61	28.65	79.96	3.73	2.16	1	0	186	Normal	0.0	0.0	0	1783173733.7282221	v3NoCvzzGPh5g
1	2025-01-01 01:01:00+01:00	29	68.19	57.28	35.94	3.64	0.69	1	0	300	Normal	0.0	0.0	0	1783173733.7282221	W82T0V1LSu54w
2	2025-01-01 01:02:00+01:00	15	98.94	50.20	72.00	1.00	2.40	1	1	19	Normal	1.0	1.0	1	1783173733.7282221	3-w82GzPM8Ld5g
3	2025-01-01 01:03:00+01:00	43	90.91	37.65	30.34	3.15	4.96	1	1	10	Normal	1.0	1.0	1	1783173733.7282221	Wp80au564gTA
4	2025-01-01 01:04:00+01:00	8	72.32	40.69	56.71	2.68	0.63	2	0	65	Vibration Issue	0.0	0.0	1	1783173733.7282221	UPLd9GLdXOL6rA

FIGURE 2.2 – Vérification des données chargées dans la base DuckDB.

La Figure 2.2 illustre les résultats du script de vérification exécuté après l'ingestion des données. Celui-ci confirme que les 100 000 enregistrements du jeu de données ont été correctement importés dans la couche *raw* de la base DuckDB. En complément, un aperçu des premières lignes est affiché, permettant de vérifier la présence des différentes variables du dataset ainsi que la cohérence des données chargées. Cette étape valide l'intégrité des données avant leur utilisation dans les phases de transformation et de préparation des caractéristiques.

2.6 Conclusion

Dans ce chapitre, nous avons présenté la mise en œuvre du pipeline d'ingestion des données à l'aide de l'outil dlt. Après avoir décrit le jeu de données utilisé et l'architecture générale du pipeline, nous avons expliqué le rôle des scripts développés pour lire le fichier CSV et charger automatiquement les données dans une base DuckDB au sein de la couche *raw*. Les tests réalisés ont confirmé le bon déroulement de l'ingestion ainsi que le chargement complet des 100 000 enregistrements du dataset. Cette première étape constitue une

base fiable pour les traitements qui seront réalisés dans les chapitres suivants, notamment la validation des données, leur transformation et la préparation des variables destinées au modèle de maintenance prédictive.

Chapitre 3: Stockage avec DuckDB

Après l'étape d'ingestion réalisée à l'aide de **dlt**, les données sont stockées dans une base DuckDB afin de constituer un Data Warehouse local. Cette couche permet de structurer les données, de vérifier leur qualité et de préparer les jeux de données qui seront utilisés dans les étapes de transformation et de Machine Learning. Le choix de DuckDB pour cette phase du projet s'explique par sa légèreté, son absence de dépendance à un serveur externe et sa capacité à exécuter des requêtes analytiques directement sur des fichiers, ce qui en fait un outil particulièrement adapté à un environnement de développement local avant un éventuel passage à l'échelle vers un entrepôt de données plus robuste.

La base est organisée selon une architecture en plusieurs niveaux permettant de séparer les données brutes, les données nettoyées, les variables dérivées et le jeu de données final. Cette séparation en couches successives, inspirée des architectures de type medalion couramment utilisées en ingénierie des données, garantit une traçabilité complète du cheminement de l'information : chaque transformation appliquée aux données est isolée dans un schéma dédié, ce qui facilite le débogage, la reproductibilité des traitements et la maintenance du pipeline dans le temps.

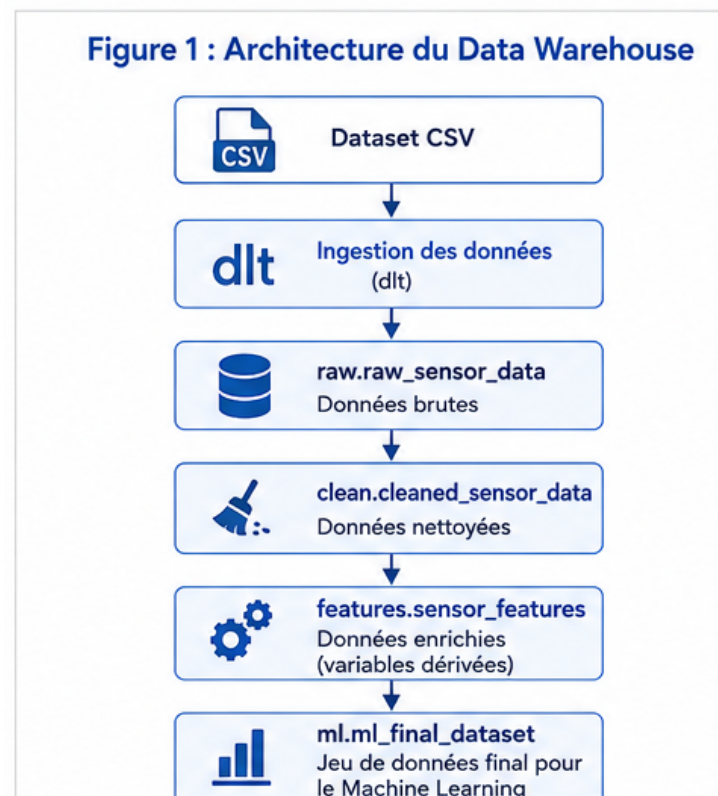


FIGURE 3.1 – Architecture du Data Warehouse DuckDB.

La figure 3.1 représente sous forme d'un schéma vertical l'ensemble du cheminement suivi par les données depuis leur source jusqu'à leur exploitation finale. Le flux part du fichier CSV d'origine, traité par le connecteur **dlt** qui assure l'ingestion, puis alimente successivement la table `raw.raw_sensor_data` contenant les données brutes, la table `clean.cleaned_sensor_data` regroupant les données nettoyées, la table `features.sensor_features` rassemblant les variables enrichies et dérivées, et enfin la table `ml.ml_final_dataset` qui constitue le jeu de données prêt pour l'entraînement des modèles de Machine Learning. Chaque étape est représentée par un bloc distinct relié au suivant par une flèche, illustrant la progression linéaire et unidirectionnelle de la donnée à travers les quatre schémas de la base.

Les données transitent successivement par quatre couches :

- Raw : données brutes provenant du pipeline d'ingestion, conservées sans aucune modification afin de disposer à tout moment d'une copie fidèle de la source d'origine.
- Clean : données nettoyées, débarrassées des incohérences, des doublons et des formats non conformes détectés lors du contrôle qualité.
- Features : création des variables dérivées, obtenues par combinaison ou transformation des variables existantes en vue d'enrichir le pouvoir prédictif du futur modèle.
- ML : jeu de données final destiné à l'entraînement des modèles, structuré de manière à être directement exploitable par les algorithmes de Machine Learning.

Cette organisation en couches successives permet non seulement de garantir la qualité et la cohérence des données à chaque étape, mais aussi de faciliter la réutilisation de chaque niveau intermédiaire pour d'autres besoins analytiques éventuels, sans avoir à rejouer l'ensemble du pipeline depuis la source brute.

3.1 Création des tables

Le stockage repose sur quatre schémas DuckDB permettant de séparer les différentes étapes de préparation des données. Chaque schéma correspond directement à l'une des couches présentées précédemment, ce qui permet d'isoler physiquement les responsabilités : les opérations de nettoyage n'affectent jamais les données brutes, et la génération des variables dérivées n'altère pas les données nettoyées qui les ont produites.

Les tables sont créées automatiquement à partir du script SQL `create_tables.sql`, qui définit pour chaque schéma la structure des colonnes, les types de données ainsi que les contraintes nécessaires à la cohérence du modèle. L'automatisation de cette étape via un script unique garantit que l'ensemble de l'architecture peut être reconstruit de manière identique et reproductible, que ce soit en environnement de développement ou lors d'une nouvelle exécution du pipeline.

```

PS C:\Users\zineb\Downloads\predictive-maintenance-mlops-main (2)\predictive-maintenance-mlops-main\predictive-maintenance-mlops-main> py -3.13 warehouse\check_database.py
=====
TABLES DISPONIBLES
=====

```

	database	schema	...	column_types	temporary
0	maintenance_predictive_pipeline	clean	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
1	maintenance_predictive_pipeline	features	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
2	maintenance_predictive_pipeline	ml	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
3	maintenance_predictive_pipeline	raw	...	[VARCHAR, VARCHAR, BIGINT, TIMESTAMP WITH TIME...	False
4	maintenance_predictive_pipeline	raw	...	[BIGINT, BIGINT, VARCHAR, VARCHAR, TIMESTAMP W...	False
5	maintenance_predictive_pipeline	raw	...	[BIGINT, BIGINT, TIMESTAMP WITH TIME ZONE, VAR...	False
6	maintenance_predictive_pipeline	raw	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
7	maintenance_predictive_pipeline	raw	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False

```

[8 rows x 6 columns]

```

FIGURE 3.2 – Tables créées dans DuckDB.

La figure 3.2 présente la sortie de la commande listant l'ensemble des tables enregistrées dans la base DuckDB après exécution du script `create_tables.sql`. On y retrouve les quatre tables réparties dans leurs schémas respectifs : `raw.raw_sensor_data`, `clean.cleaned_sensor_data`, `features.sensor_features` et `ml.ml_final_dataset`, accompagnées de leur nom de schéma, ce qui confirme visuellement que l'architecture en quatre couches a bien été instanciée avant toute opération de transformation.

3.2 Contrôle du nombre de lignes

Une première vérification consiste à contrôler que les transformations n'ont provoqué aucune perte de données. Ce contrôle est essentiel dans un pipeline en plusieurs étapes, car une jointure mal formulée, un filtre trop restrictif ou une erreur de logique lors du passage d'une couche à l'autre peut entraîner une réduction silencieuse du volume de données, susceptible de fausser les analyses et les résultats du modèle sans qu'aucune erreur explicite ne soit levée.

Les résultats montrent que chacune des quatre tables contient exactement 100 000 lignes, ce qui confirme qu'aucune donnée n'a été perdue ou dupliquée au cours des différentes transformations appliquées entre les couches raw, clean, features et ml.

```

=====
NOMBRE DE LIGNES PAR TABLE
=====

```

	table_name	total_rows
0	raw	100000
1	clean	100000
2	features	100000
3	ml	100000

FIGURE 3.3 – Nombre de lignes des différentes tables.

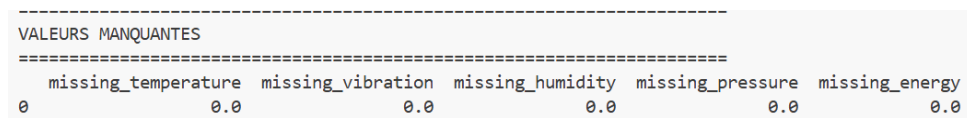
La figure 3.3 affiche le résultat de la requête SQL de comptage exécutée sur chacun des quatre schémas. Le résultat est présenté sous forme d'un tableau associant à chaque table son nombre de lignes, permettant de vérifier en un coup d'œil que les valeurs restent

identiques d'une couche à l'autre et qu'aucune ligne n'a été perdue lors des transformations successives appliquées entre les schémas `raw`, `clean`, `features` et `ml`.

3.3 Recherche des valeurs manquantes

Une deuxième série de contrôles permet de vérifier que les principales variables ne contiennent aucune valeur manquante. La présence de valeurs nulles non détectées pourrait en effet compromettre la fiabilité des statistiques descriptives ainsi que la qualité de l'entraînement des modèles de Machine Learning, certains algorithmes étant particulièrement sensibles à ce type d'incomplétude.

Cette vérification est réalisée systématiquement sur l'ensemble des colonnes jugées critiques pour la suite du projet, en s'appuyant sur des requêtes SQL dédiées permettant de comptabiliser, pour chaque variable, le nombre d'occurrences nulles ou manquantes.



```
-----
VALEURS MANQUANTES
-----
missing_temperature missing_vibration missing_humidity missing_pressure missing_energy
0                  0.0                0.0                0.0                0.0                0.0
```

FIGURE 3.4 – Recherche des valeurs manquantes.

La figure 3.4 montre le résultat de la requête de comptage des valeurs nulles exécutée sur les principales colonnes des tables. Le résultat se présente sous la forme d'une liste de variables, chacune associée à un compteur de valeurs manquantes égal à zéro, ce qui atteste de la complétude du jeu de données à ce stade du pipeline.

Figure 3 : Schéma de Feature Engineering

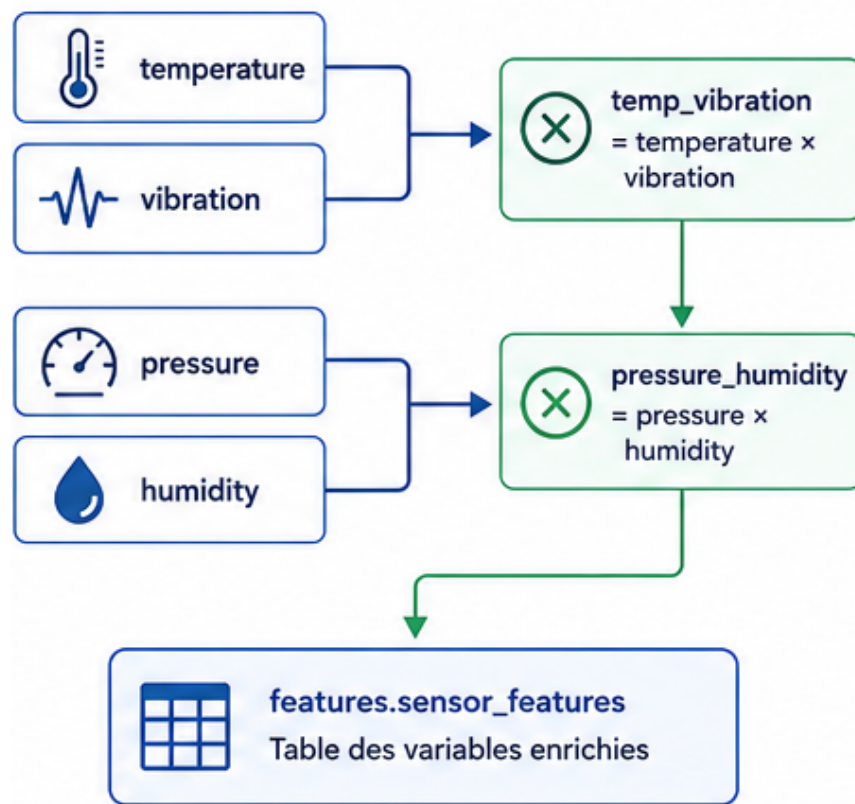


FIGURE 3.5 – Création des variables dérivées.

La figure 3.5 détaille le schéma de construction des deux variables issues du Feature Engineering. Les variables `temperature` et `vibration` sont combinées par multiplication pour produire la variable `temp_vibration`, tandis que les variables `pressure` et `humidity` sont combinées de la même manière pour produire la variable `pressure_humidity`. Le schéma représente ces deux transformations par des blocs reliés par des flèches convergeant vers la table `features.sensor_features`, qui centralise l'ensemble des variables enrichies destinées aux étapes ultérieures du pipeline.

3.4 Création des variables dérivées

Deux nouvelles variables ont été créées afin d'illustrer une étape simple de Feature Engineering :

- `temp_vibration` : variable obtenue par combinaison de la température et du niveau de vibration mesurés sur l'équipement, permettant de capturer une interaction potentiellement significative entre ces deux signaux physiques.

- `pressure_humidity` : variable dérivée croisant la pression et le taux d'humidité, susceptible de révéler des schémas de dégradation non visibles lorsque ces deux mesures sont considérées isolément.

Ces variables sont stockées dans le schéma **features**, distinct des données nettoyées d'origine, de manière à conserver une frontière claire entre les données mesurées directement et celles issues d'un traitement analytique supplémentaire. Cette approche permet également de faire évoluer ou d'ajouter de nouvelles variables dérivées sans avoir à modifier la structure des couches précédentes.

3.5 Statistiques descriptives

Les statistiques descriptives permettent d'obtenir une première analyse des données avant leur exploitation par les modèles de Machine Learning. Elles offrent un aperçu synthétique de la distribution de chaque variable numérique, à travers des indicateurs tels que la moyenne, l'écart-type, les valeurs minimales et maximales ainsi que les principaux quantiles.

Cette étape exploratoire permet notamment de détecter d'éventuelles valeurs aberrantes, d'identifier des échelles de grandeur très différentes entre variables, et d'orienter les choix de prétraitement à effectuer avant l'entraînement des modèles, tels que la normalisation ou la standardisation.

STATISTIQUES DESCRIPTIVES					
	avg_temperature	avg_vibration	avg_humidity	avg_pressure	avg_energy
0	75.015625	50.01227	54.995401	3.000405	2.747064

FIGURE 3.6 – Statistiques descriptives des principales variables.

La figure 3.6 présente, sous forme de tableau, les indicateurs statistiques calculés pour chacune des variables numériques du jeu de données : moyenne, écart-type, valeurs minimale et maximale, ainsi que les quantiles. Cette lecture synthétique permet de comparer rapidement les ordres de grandeur des différentes variables et de repérer une éventuelle dispersion anormale ou des valeurs extrêmes à surveiller avant la phase de modélisation.

3.6 Distribution de la variable cible

La distribution de la variable `maintenance_required` permet de visualiser la répartition des observations entre les deux classes. Cette analyse revêt une importance particulière dans un contexte de classification, dans la mesure où un déséquilibre marqué entre

les classes pourrait biaiser l'apprentissage du modèle en faveur de la classe majoritaire, au détriment de la détection des cas de maintenance réellement nécessaires.

L'observation de cette répartition permet ainsi d'anticiper d'éventuelles stratégies de rééquilibrage à mettre en œuvre lors de la phase de modélisation, et constitue un préalable indispensable à une évaluation pertinente des performances futures du modèle.

DISTRIBUTION DE LA VARIABLE CIBLE		
=====		
	<code>maintenance_required</code>	<code>total</code>
0	0	80303
1	1	19697
=====		

FIGURE 3.7 – Distribution de la variable cible.

La figure 3.7 illustre, sous forme de graphique, la proportion des observations appartenant à chacune des deux classes de la variable `maintenance_required`. La visualisation permet d'évaluer directement le degré d'équilibre entre les cas nécessitant une maintenance et ceux n'en nécessitant pas, information déterminante pour anticiper le comportement du futur modèle de classification.

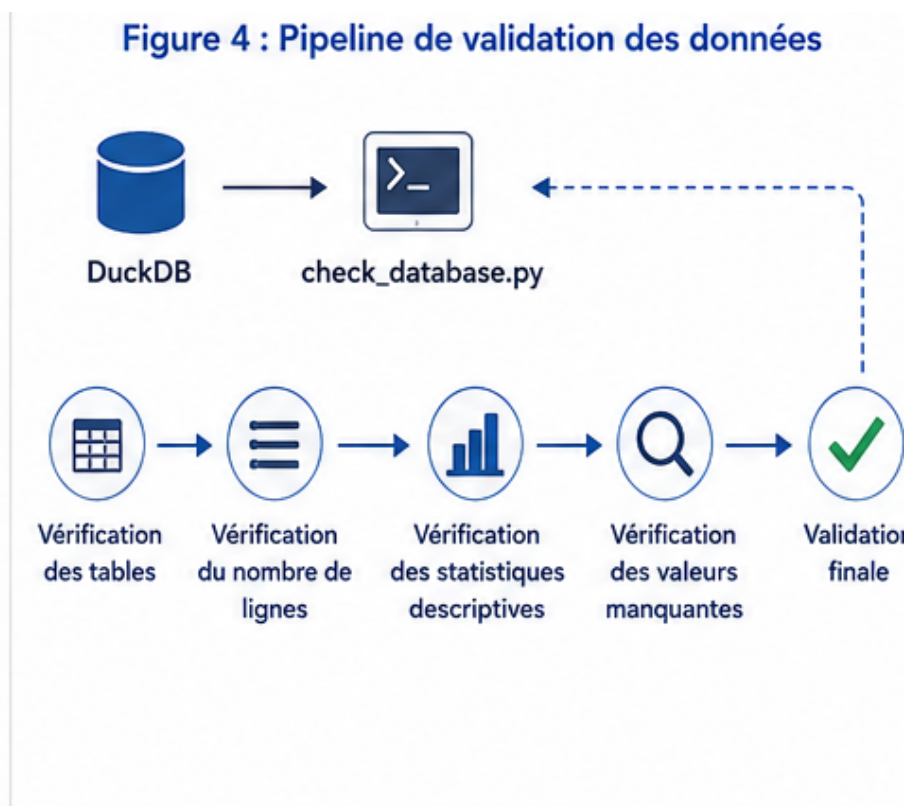


FIGURE 3.8 – Pipeline de validation automatique.

La figure 3.8 schématise le fonctionnement du script `check_database.py` : la base DuckDB est interrogée par ce script, qui exécute successivement la vérification des tables, le contrôle du nombre de lignes, le calcul des statistiques descriptives et la recherche des valeurs manquantes, avant d’aboutir à une étape finale de validation. Les différentes vérifications sont représentées sous forme de cercles reliés entre eux par des flèches, formant une chaîne de contrôle qui se referme sur le script lui-même, symbolisant le caractère cyclique et automatisable de la procédure.

3.7 Validation automatique

Afin d’automatiser les contrôles, un script Python nommé `check_database.py` a été développé. Ce script centralise l’ensemble des vérifications présentées précédemment au sein d’une seule exécution, évitant ainsi de devoir relancer manuellement chaque requête SQL après toute modification du pipeline ou des données sources.

Ce script exécute automatiquement :

- l’affichage des tables ;
- le contrôle du nombre de lignes ;
- la recherche des valeurs manquantes ;
- le calcul des statistiques descriptives ;
- la distribution de la variable cible.

Cette automatisation constitue un gain de temps important et réduit le risque d’erreur humaine lors des phases de vérification, tout en assurant une traçabilité homogène des contrôles effectués à chaque exécution du pipeline.

```

=====
TABLES DISPONIBLES
=====

```

	database	schema	...	column_types	temporary
0	maintenance_predictive_pipeline	clean	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
1	maintenance_predictive_pipeline	features	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
2	maintenance_predictive_pipeline	ml	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
3	maintenance_predictive_pipeline	raw	...	[VARCHAR, VARCHAR, BIGINT, TIMESTAMP WITH TIME...	False
4	maintenance_predictive_pipeline	raw	...	[BIGINT, BIGINT, VARCHAR, VARCHAR, TIMESTAMP W...	False
5	maintenance_predictive_pipeline	raw	...	[BIGINT, BIGINT, TIMESTAMP WITH TIME ZONE, VAR...	False
6	maintenance_predictive_pipeline	raw	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False
7	maintenance_predictive_pipeline	raw	...	[TIMESTAMP WITH TIME ZONE, BIGINT, DOUBLE, DOU...	False

```

[8 rows x 6 columns]

=====
NOMBRE DE LIGNES PAR TABLE
=====

```

table_name	total_rows
0 raw	100000
1 clean	100000
2 features	100000
3 ml	100000

```

=====
DISTRIBUTION DE LA VARIABLE CIBLE
=====

```

maintenance_required	total
0	80303
1	19697

```

=====
STATISTIQUES DESCRIPTIVES
=====

```

	avg_temperature	avg_vibration	avg_humidity	avg_pressure	avg_energy
0	75.015625	50.01227	54.995401	3.000405	2.747064

```

=====
VALEURS MANQUANTES
=====

```

missing_temperature	missing_vibration	missing_humidity	missing_pressure	missing_energy
0	0.0	0.0	0.0	0.0

FIGURE 3.9 – Exécution du script de validation.

La figure 3.9 montre la trace d'exécution du script `check_database.py` dans le terminal, avec l'affichage successif des résultats de chaque contrôle : liste des tables, comptage des lignes, statistiques descriptives, recherche des valeurs manquantes et distribution de la variable cible. Cet enchaînement de sorties confirme que l'ensemble des vérifications s'exécute correctement et sans erreur en une seule commande.

Figure 5 : Architecture globale du pipeline de données

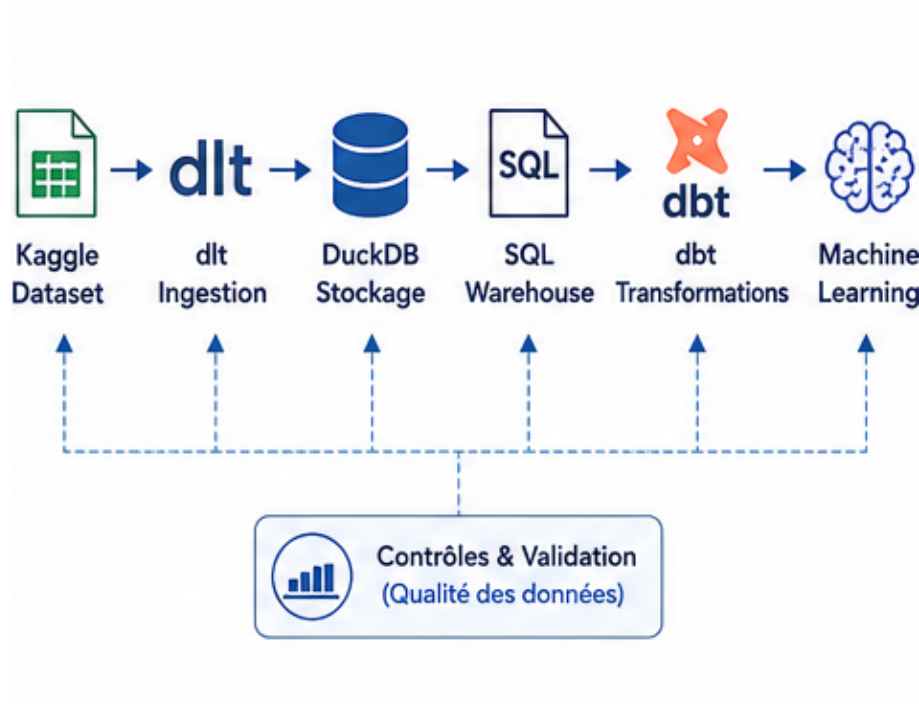


FIGURE 3.10 – Architecture globale du pipeline de données.

La figure 3.10 synthétise l'ensemble de la chaîne de traitement sous forme d'une ligne horizontale reliant successivement le jeu de données Kaggle, l'ingestion via **dlt**, le stockage dans DuckDB, les requêtes SQL de l'entrepôt, les transformations réalisées avec **dbt**, puis l'étape finale de Machine Learning. Un bloc « Contrôles & Validation (Qualité des données) » est positionné en dessous de cette chaîne et relié par des flèches verticales à chacune de ses étapes, illustrant le fait que la qualité des données est vérifiée de manière transversale à chaque point du pipeline, et non uniquement à son issue.

Cette architecture illustre le cheminement complet des données depuis leur ingestion jusqu'à la préparation du jeu de données destiné au Machine Learning. Chaque étape du pipeline, depuis la collecte via **dlt** jusqu'à la constitution du jeu de données final dans le schéma **ml**, s'articule autour de contrôles systématiques garantissant l'intégrité de l'information à chaque transition entre les couches.

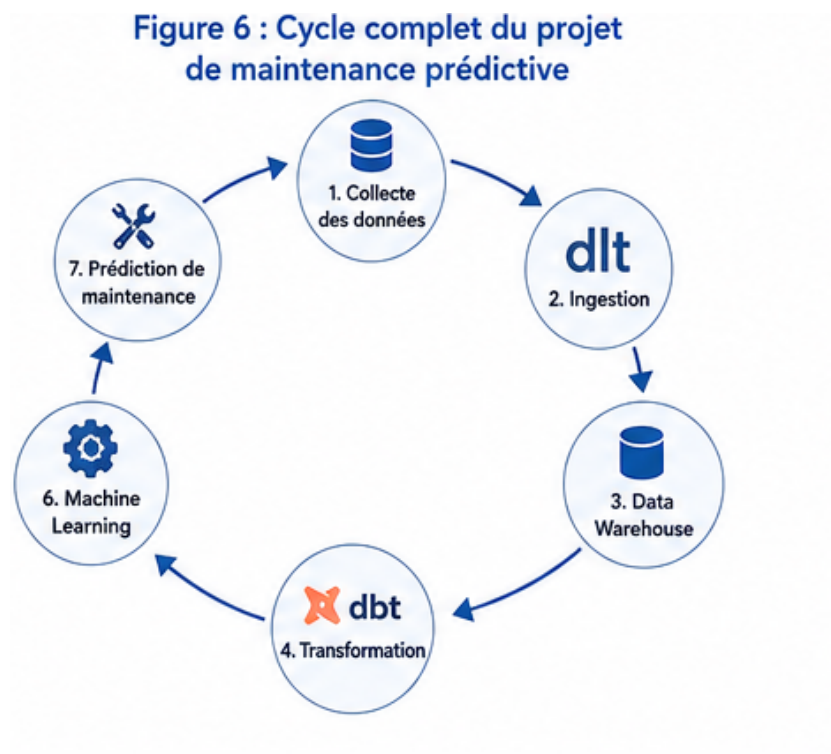


FIGURE 3.11 – Cycle complet du projet de maintenance prédictive.

La figure 3.11 représente sous forme circulaire les grandes étapes du projet, organisées en boucle continue : la collecte des données, l'ingestion via **dlt**, la constitution du Data Warehouse, la transformation avec **dbt**, le Machine Learning et enfin la prédiction de maintenance. Chaque étape est symbolisée par un cercle relié au suivant par une flèche orientée, la disposition circulaire traduisant le caractère itératif du processus : les prédictions produites en fin de cycle sont destinées à alimenter à nouveau la collecte de données lors des itérations futures du projet.

Conclusion

Cette étape a permis de mettre en place une architecture de stockage robuste basée sur DuckDB. Les données sont organisées en quatre couches (**raw**, **clean**, **features** et **ml**), garantissant une séparation claire des différentes phases de préparation.

Les différentes requêtes SQL ainsi que le script de validation ont permis de confirmer la cohérence des données, l'absence de valeurs manquantes et la bonne préparation du jeu de données final destiné aux étapes de transformation avec **dbt** puis à la modélisation Machine Learning. Cette fondation solide constitue un prérequis indispensable à la fiabilité de l'ensemble des analyses et des modèles développés dans les chapitres suivants, l'exactitude des conclusions tirées en aval du pipeline dépendant directement de la qualité des données consolidées à ce stade.

Chapitre 4: Transformations dbt et qualité des données

Introduction

Ce chapitre présente le travail réalisé en tant qu'Analytics Engineer et Data Quality Engineer au sein du projet de maintenance prédictive industrielle. Il couvre l'ensemble de la chaîne de transformation des données, depuis leur réception à l'état brut jusqu'à la production d'une table finale prête pour le Machine Learning, en passant par la mise en place d'un pipeline dbt structuré, un travail approfondi de nettoyage et d'analyse de la qualité des données, la définition de tests automatisés, la formalisation d'un Data Contract, et la génération d'une documentation technique. Chaque étape est présentée avec la méthodologie suivie, les résultats obtenus, et les décisions prises, systématiquement justifiées par des observations chiffrées plutôt que par intuition.

4.1 Objectif des transformations

Cette partie du projet a pour objectif de transformer les données brutes des capteurs machines, ingérées par le pipeline dlt et stockées dans une base DuckDB, en un jeu de données propre, documenté, testé et fiable, prêt à être utilisé par l'équipe Machine Learning pour l'entraînement d'un modèle de maintenance prédictive. Ce travail s'appuie sur l'outil dbt (*data build tool*), qui permet de structurer les transformations de données en couches successives, chacune ayant une responsabilité clairement définie, tout en intégrant nativement des tests de qualité automatisés, une documentation générée, et un contrat de données garantissant la stabilité du schéma livré aux équipes en aval. Les objectifs spécifiques poursuivis dans ce chapitre sont les suivants :

- Mettre en place un projet dbt connecté à la base DuckDB alimentée par l'équipe Data Engineering.
- Structurer les transformations selon une architecture en couches (raw, staging, cleaned, ml_features), afin d'isoler les responsabilités et de faciliter la maintenance du pipeline.
- Mener un travail exhaustif de data profiling sur l'ensemble des colonnes du jeu de données, afin d'identifier les anomalies avant toute correction.

- Nettoyer les données en appliquant des règles de transformation systématiquement justifiées par des faits chiffrés, et non par intuition.
- Mettre en place des tests de qualité automatisés, couvrant à la fois les aspects structurels et métier.
- Formaliser un Data Contract décrivant précisément le schéma final livré à l'équipe Machine Learning, avec activation d'un contrôle de schéma natif dans dbt.
- Générer une documentation technique navigable, exploitable par les autres membres de l'équipe.

Une attention particulière a été portée, tout au long de ce travail, à la traçabilité et à la justification de chaque décision de nettoyage : aucune donnée n'a été modifiée ou supprimée sans une analyse préalable démontrant, chiffres à l'appui, la pertinence de cette action.

4.2 Architecture dbt

Le projet suit une architecture en couches, communément appelée architecture *medallion*, qui organise les données selon leur niveau de transformation croissant.

`raw.raw_sensor_data` → `stg_machine_data` → `clean_machine_data` → `ml_features`

Description de chaque couche

- **raw.raw_sensor_data** : source externe déclarée dans dbt via un fichier `sources.yml`. Elle correspond aux données brutes ingérées et stockées dans DuckDB par l'équipe Data Engineering. Cette couche n'est jamais modifiée directement.
- **stg_machine_data** : couche de mise en forme technique. Les colonnes sont sélectionnées et renommées si nécessaire, et les colonnes techniques ajoutées automatiquement par l'outil d'ingestion (`_dlt_load_id`, `_dlt_id`) sont retirées, car elles n'ont pas de valeur métier.
- **clean_machine_data** : couche de nettoyage métier, où les règles de qualité identifiées lors du data profiling sont appliquées (voir section 4.4).
- **ml_features** : table finale, matérialisée physiquement en table, destinée à l'entraînement des modèles de Machine Learning.

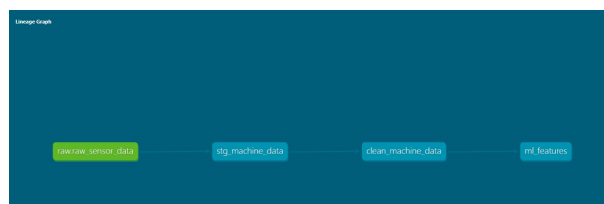


FIGURE 4.1 – Graphe de lineage du pipeline dbt (`raw` → `staging` → `clean` → `ml_features`)

Interprétation : Ce graphe confirme visuellement que le pipeline suit une chaîne de dépendances linéaire et sans rupture : chaque modèle dépend explicitement du précédent via la fonction `ref()` de dbt. L'absence de branches parallèles ou de dépendances circulaires témoigne d'une architecture simple et maîtrisée, adaptée à la taille du projet.

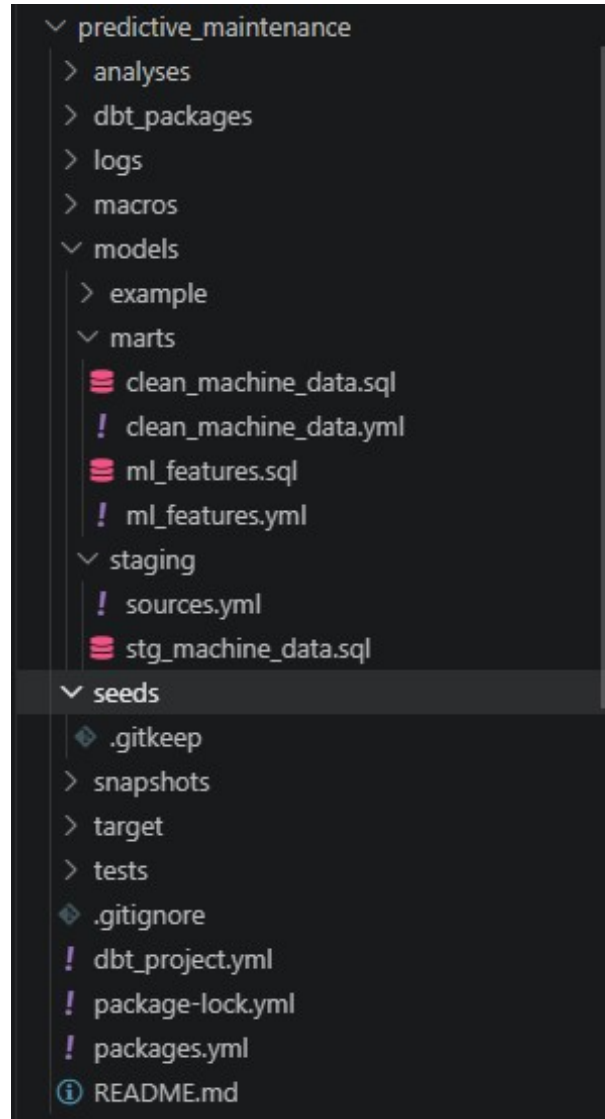


FIGURE 4.2 – Structure du projet dans l'arborescence de fichiers

Interprétation : L'arborescence du projet reflète directement l'architecture en couches décrite ci-dessus : le dossier `models/staging/` contient la déclaration de la source ainsi que le modèle de mise en forme technique, tandis que le dossier `models/marts/` regroupe les modèles orientés métier, accompagnés de leurs fichiers de tests et de documentation respectifs.

4.3 Modèles dbt créés

Le projet dbt a été initialisé à l'aide de la commande `dbt init`, puis connecté à la base DuckDB via le fichier de configuration `profiles.yml`, dont le chemin pointe vers le fichier `maintenance_predictive_pipeline.duckdb`.

```
(new_dbt) PS C:\Users\WRI510\Desktop\projet_alops\predictive_maintenance> dbt debug
22:41:55 profiles.yml file [OK found and valid]
22:41:55 dbt_project.yml file [OK found and valid]
22:41:55 Required dependencies:
22:41:55 - git [OK found]

22:41:55 Connection:
22:41:55 database: maintenance_predictive_pipeline
22:41:55 schema: main
22:41:55 path: C:\Users\WRI510\Desktop\projet_alops\predictive_maintenance-alops-main (2)\predictive_maintenance-alops-main\predictive-
maintenance-alops-main\gestion\maintenance_predictive_pipeline.duckdb
22:41:55 config_options: None
22:41:55 extensions: None
22:41:55 settings: {}
22:41:55 external_root:
22:41:55 use_credential_provider: None
22:41:55 attach: None
22:41:55 filesystems: None
22:41:55 remote: None
22:41:55 plugins: None
22:41:55 disable_transactions: False
22:41:55 Registered adapter: duckdb-1.10.1
22:41:57 Connection test: [OK connection ok]

22:41:57 All checks passed!
```

FIGURE 4.3 – Résultat de la commande `dbt debug`

Interprétation : La commande `dbt debug` exécute une série de vérifications avant tout travail de transformation : validité du fichier `profiles.yml`, validité du fichier `dbt_project.yml`, présence des dépendances requises, et test effectif de connexion à la base de données. Le message final « All checks passed ! » confirme que dbt est correctement configuré.

Modèle `stg_machine_data`

Ce modèle constitue la première étape de transformation. Il sélectionne l'ensemble des colonnes utiles depuis la source brute déclarée et retire les deux colonnes techniques ajoutées automatiquement par l'outil d'ingestion (`_dlt_load_id` et `_dlt_id`), qui ne portent aucune information métier. La colonne `timestamp` a par ailleurs été renommée en `recorded_at`.

```

stg_machine_data.sql X
predictive_maintenance > models > staging > stg_machine_data.sql
1 with source as (
2
3     select * from {{ source('raw', 'raw_sensor_data') }}
4
5 ),
6
7 renamed as (
8
9     select
10         machine_id,
11         timestamp as recorded_at,
12         temperature,
13         vibration,
14         humidity,
15         pressure,
16         energy_consumption,
17         machine_status,
18         anomaly_flag,
19         predicted_remaining_life,
20         failure_type,
21         downtime_risk,
22         maintenance_required
23
24     from source
25
26 )
27
28 select * from renamed

```

FIGURE 4.4 – Code source du modèle `stg_machine_data.sql`

Interprétation : Le code est structuré en deux blocs successifs (*Common Table Expressions*) : un premier bloc `source` qui isole la lecture de la donnée brute via la fonction `source()`, et un second bloc `renamed` qui effectue la sélection et le renommage des colonnes.

Modèle `clean_machine_data`

Ce modèle applique la première et unique règle de nettoyage métier identifiée lors du travail de data profiling détaillé en section 4.4 : l'exclusion des observations présentant une valeur de vibration négative, physiquement impossible.

```

clean_machine_data.sql X
predictive_maintenance > models > marts > clean_machine_data.sql
1 ~ with staged as (
2
3     select * from {{ ref('stg_machine_data') }}
4
5 ),
6
7 ~ cleaned as (
8
9     select *
10    from staged
11
12    -- Suppression des observations présentant une vibration négative.
13    -- Analyse exploratoire : 37 lignes sur 100 000 (0,04 %), dont 6 lignes
14    -- avec maintenance_required = 1 (soit seulement 0,03 % des 19 697 cas
15    -- positifs du dataset - impact négligeable sur le volume disponible pour le ML).
16    -- Ces valeurs étant physiquement impossibles (une vibration ne peut pas
17    -- être négative) et leur proportion étant négligeable, les observations
18    -- sont supprimées afin de garantir la qualité des données, plutôt que
19    -- d'inventer une correction (ex: valeur absolue) qui ne serait pas justifiable.
20    where vibration >= 0
21
22 )
23
24 select * from cleaned

```

FIGURE 4.5 – Code source du modèle `clean_machine_data.sql`

Interprétation : Le commentaire intégré au code SQL précise que l'analyse exploratoire a mis en évidence 37 lignes concernées sur 100 000 (soit 0,04 % du jeu de données), dont seulement 6 lignes correspondant à des cas de maintenance requise.

Modèle ml_features

Ce modèle constitue la table finale du pipeline. Il est explicitement matérialisé en table physique via la configuration `config(materialized='table')`, plutôt qu'en vue.

```
ml_features.sql X
predictive_maintenance > models > marts > ml_features.sql
1 {{ config(materialized='table') }}
2 with cleaned as (
3     select * from {{ ref('clean_machine_data') }}
4 ),
5
6 ),
7
8 final as (
9     select
10         machine_id,
11         recorded_at,
12         temperature,
13         vibration,
14         humidity,
15         pressure,
16         energy_consumption,
17         machine_status,
18         anomaly_flag,
19         failure_type,
20
21         -- Score de risque d'arrêt (0 à 1). À noter : quasi-redondant avec
22         -- anomaly_flag d'après l'analyse exploratoire (8912/8914 valeurs à 1.0
23         -- correspondent exactement aux cas anomaly_flag = 1). Conservé ici
24         -- pour laisser le choix à l'équipe ML de l'utiliser ou non.
25         downtime_risk,
26
27         -- Durée de vie restante estimée. À noter : l'analyse exploratoire
28         -- montre une moyenne quasi identique (233-235) quelle que soit la
29         -- valeur de machine_status, suggérant une faible valeur prédictive
30         -- réelle pour cette variable dans ce dataset.
31         predicted_remaining_life,
32
```

FIGURE 4.6 – Code source du modèle ml_features.sql

Interprétation : Ce modèle sélectionne l'ensemble des colonnes issues de `clean_machine_data`, sans appliquer de filtre supplémentaire. Le choix a été fait de conserver l'intégralité des colonnes, y compris celles dont la valeur prédictive est apparue limitée, plutôt que de les retirer unilatéralement.

```
(venv_dbt) PS C:\Users\XPRISTO\Desktop\projet_ml_ops\predictive_maintenance> dbt run
22:36:15 Running with dbt-1.11.12
22:36:16 Registered adapter: duckdb=1.10.1
22:36:18 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.predictive_maintenance.example
22:36:18 Found 3 models, 18 data tests, 1 source, 601 macros
22:36:18 Concurrency: 1 threads (target='dev')
22:36:18
22:36:18 1 of 3 OK created sql view model main.stg_machine_data ..... [OK in 0.30s]
22:36:19 2 of 3 START sql view model main.clean_machine_data ..... [RUN]
22:36:19 2 of 3 OK created sql view model main.clean_machine_data ..... [OK in 0.13s]
22:36:19 3 of 3 START sql table model main.ml_features ..... [RUN]
22:36:20 3 of 3 OK created sql table model main.ml_features ..... [OK in 0.70s]
22:36:20
22:36:20 Finished running 1 table model, 2 view models in 0 hours 0 minutes and 1.74 seconds (1.74s).
22:36:20 Completed successfully
22:36:20
22:36:20 Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=3
```

FIGURE 4.7 – Résultat de la commande dbt run

Interprétation : L'exécution de la commande `dbt run` construit l'ensemble des modèles du projet dans l'ordre déterminé automatiquement par dbt. Le résultat confirme la construction réussie de chaque modèle, avec la mention explicite du type de matérialisation retenu pour chacun.

4.4 Nettoyage des données

Avant toute décision de transformation, un travail exhaustif de data profiling a été mené sur l'ensemble des 14 colonnes du jeu de données.

4.4.1 Vue d'ensemble des données brutes

```
(venv_dbt) PS C:\Users\VPRISTO\Desktop\projet_alops\predictive_maintenance> dbt show --inline "SELECT * FROM raw.raw_sensor_data" --limit 5
15:46:15 Running with dbt=1.11.12
15:46:16 Registered adapter: duckdb=1.10.1
15:46:18 Found 2 models, 4 data tests, 1 sql operation, 485 macros
15:46:18 Concurrency: 1 threads (target='dev')
15:46:18
Prevailing inline node:
+-----+-----+-----+-----+-----+-----+
| 2025-01-01 01:00:00 | 39 | 78.61 | 28.65 | 79.06 | 3.73 | ... |
| 2025-01-01 01:01:00 | 29 | 68.12 | 57.20 | 35.94 | 3.64 | ... |
| 2025-01-01 01:02:00 | 15 | 98.04 | 50.20 | 72.86 | 1.00 | ... |
| 2025-01-01 01:03:00 | 43 | 90.91 | 37.65 | 30.34 | 3.15 | ... |
| 2025-01-01 01:04:00 | 8 | 72.12 | 40.69 | 50.71 | 2.68 | ... |
```

FIGURE 4.8 – Extrait des données brutes (source raw.raw_sensor_data)

Interprétation : Cet extrait, obtenu via la commande `dbt show` interrogeant directement la source déclarée, permet de confirmer la structure réelle des données, avant toute transformation.

4.4.2 Vérifications structurelles

```
(venv_dbt) PS C:\Users\VPRISTO\Desktop\projet_alops\predictive_maintenance> dbt show --inline "SELECT COUNT(*) AS total, COUNT(DISTINCT machine_id || recorded_at::varchar) AS uniques FROM {{ ref('stg_machine_data') }}"
16:00:11 Running with dbt=1.11.12
16:00:12 Registered adapter: duckdb=1.10.1
16:00:14 Found 3 models, 4 data tests, 1 sql operation, 1 source, 405 macros
16:00:14
Prevailing inline node:
+-----+-----+
| total | uniques |
+-----+-----+
| 100000 | 100000 |
```

FIGURE 4.9 – Vérification de l'absence de doublons

Interprétation : La comparaison entre le nombre total de lignes et le nombre de combinaisons distinctes de `machine_id` et `recorded_at` donne un résultat strictement identique (100 000 dans les deux cas), confirmant qu'aucune ligne n'est dupliquée.

```
(venv_dbt) PS C:\Users\VPRISTO\Desktop\projet_alops\predictive_maintenance> dbt show --inline "SELECT COUNT(*) - COUNT(temperature) AS missing_temp, COUNT(*) - COUNT(vibration) AS missing_vib, COUNT(*) - COUNT(humidity) AS missing_hum, COUNT(*) - COUNT(pressure) AS missing_pressure, COUNT(*) - COUNT(energy_consumption) AS missing_energy FROM {{ ref('stg_machine_data') }}"
16:07:16 Running with dbt=1.11.12
16:07:16 Registered adapter: duckdb=1.10.1
16:07:18 Found 3 models, 4 data tests, 1 sql operation, 1 source, 405 macros
16:07:18
Prevailing inline node:
+-----+-----+-----+-----+-----+
| missing_temp | missing_vib | missing_hum | missing_pressure | missing_energy |
+-----+-----+-----+-----+-----+
| 0 | 0 | 0 | 0 | 0 |
```

FIGURE 4.10 – Vérification des valeurs manquantes

Interprétation : Le calcul de la différence entre le nombre total de lignes et le nombre de valeurs non nulles pour chaque colonne numérique clé retourne zéro pour l'ensemble des colonnes testées.

4.4.3 Anomalie détectée : vibrations négatives

```
(venv_dbt) PS C:\Users\VPRISTO\Desktop\projet_alops\predictive_maintenance> dbt show --inline "SELECT COUNT(*) AS total_lignes, COUNT(CASE WHEN vibration < 0 THEN 1 ELSE 0 END) AS lignes_vibration_negatives, ROUND(CAST(COUNT(CASE WHEN vibration < 0 THEN 1 ELSE 0 END) / COUNT(*) AS percentage FROM {{ ref('stg_machine_data') }}"
16:14:22 Running with dbt=1.11.12
16:14:23 Registered adapter: duckdb=1.10.1
16:14:24 Found 3 models, 4 data tests, 1 sql operation, 1 source, 405 macros
16:14:24 Concurrency: 1 threads (target='dev')
16:14:24
Prevailing inline node:
+-----+-----+-----+
| total_lignes | lignes_vibration_negatives | pourcentage |
+-----+-----+-----+
| 100000 | 37 | 0.04 |
```

FIGURE 4.11 – Quantification de l'anomalie de vibration négative

Interprétation : Cette requête dénombre précisément les lignes concernées par cette anomalie : 37 lignes sur 100 000, soit 0,04 % du jeu de données.

```

C:\Users\PRISTO\Desktop\projet_alops\predictive_maintenance> dbt show --inline "SELECT maintenance_required, COUNT(*) AS nb FROM (( ref('stg_machine_data') )) WHERE vibration < 0 GROUP BY maintenance_required"
16:19:17 Running with dbt=1.11.12
16:19:18 Registered adapter: duckdb=1.10.1
16:19:19 Found 3 models, 4 data tests, 1 sql operation, 1 source, 485 macros
16:19:19
16:19:19
Prevailing inline node:
| maintenance_required | nb |
|-----|-----|
| 0 | 31 |
| 1 | 6 |

```

FIGURE 4.12 – Répartition de la variable cible parmi les lignes concernées par l'anomalie

Interprétation : Sur les 37 lignes présentant une vibration négative, seules 6 correspondent à des cas où `maintenance_required` vaut 1, soit 0,03 % des 19 697 cas positifs du jeu de données. Trois stratégies de traitement ont été envisagées : la correction par valeur absolue, le remplacement par NULL, ou la suppression des lignes. La stratégie retenue a été la suppression des lignes concernées, qui permet de garantir la qualité du jeu de données sans fabriquer de donnée non justifiable.

```

C:\Users\PRISTO\Desktop\projet_alops\predictive_maintenance> dbt show --inline "SELECT COUNT(*) AS total_lignes FROM (( ref('clean_machine_data') ))"
16:26:42 Running with dbt=1.11.12
16:26:43 Registered adapter: duckdb=1.10.1
16:26:45 Found 4 models, 4 data tests, 1 sql operation, 1 source, 485 macros
16:26:45 Concurrency: 1 threads (target='dev')
16:26:45
16:26:45
Prevailing inline node:
| total_lignes |
|-----|
| 99963 |

```

FIGURE 4.13 – Volume total après application du filtre de nettoyage

Interprétation : Après exécution du modèle `clean_machine_data`, le nombre total de lignes de la table est de 99 963, soit exactement 100 000 moins les 37 lignes exclues.

```

C:\Users\PRISTO\Desktop\projet_alops\predictive_maintenance> dbt show --inline "SELECT COUNT(*) AS lignes_negatives FROM (( ref('clean_machine_data') )) WHERE vibration < 0"
16:27:11 Running with dbt=1.11.12
16:27:12 Registered adapter: duckdb=1.10.1
16:27:14 Found 4 models, 4 data tests, 1 sql operation, 1 source, 485 macros
16:27:14
16:27:14 Concurrency: 1 threads (target='dev')
16:27:14
Prevailing inline node:
| lignes_negatives |
|-----|
| 0 |

```

FIGURE 4.14 – Confirmation de l'absence de vibration négative après nettoyage

Interprétation : Une seconde vérification, portant spécifiquement sur le comptage des lignes où la vibration reste négative après nettoyage, retourne un résultat de zéro.

4.4.4 Analyse des autres variables numériques

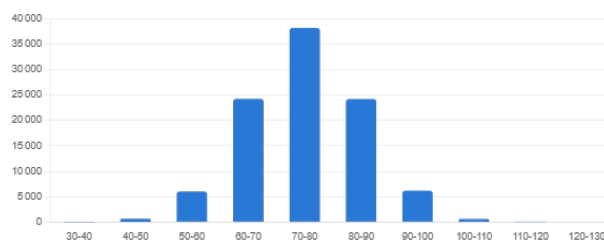


FIGURE 4.15 – Distribution de la variable température par tranches de 10 degrés

Interprétation : L’histogramme obtenu présente une forme de distribution en cloche, centrée entre 70 et 90 degrés, cohérente avec un phénomène physique naturel de mesure de température. Aucune anomalie nécessitant une correction n’a été détectée.

4.4.5 Analyse des variables catégorielles et binaires

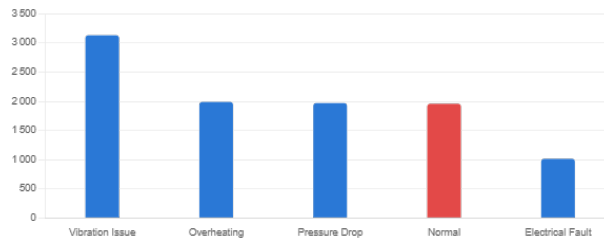


FIGURE 4.16 – Répartition des types de panne (failure_type) parmi les machines en état de panne (machine_status = 2)

Interprétation : Ce graphique révèle une incohérence logique dans le jeu de données : parmi les machines dont l’état est explicitement marqué comme étant en panne, une part importante (1 956 observations sur 10 053, soit environ 19 %) porte néanmoins la mention failure_type = 'Normal'.

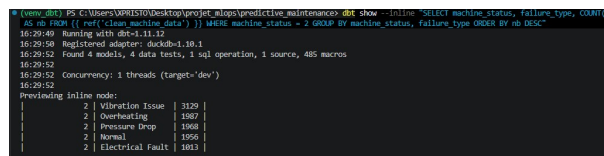


FIGURE 4.17 – Détail chiffré de l’incohérence entre failure_type et machine_status

Interprétation : Cette vue détaillée confirme la répartition exacte des cinq catégories de failure_type parmi les machines en panne, et permet de mesurer précisément l’ampleur de l’incohérence relevée.

4.4.6 Analyse de downtime_risk et predicted_remaining_life

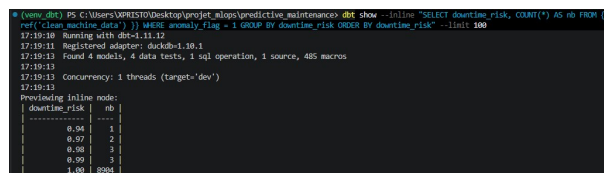


FIGURE 4.18 – Distribution de downtime_risk parmi les observations où anomaly_flag = 1

Interprétation : L'analyse révèle que `downtime_risk` est quasiment binaire dans sa distribution : sur les 8 914 lignes où `anomaly_flag` vaut 1, 8 904 présentent une valeur de `downtime_risk` exactement égale à 1,00.

```
(mvp>) PS C:\Users\WRI1270\Desktop\project_allops\predictive_maintenance> dbt show --inline "SELECT machine_status, ROUND(avg(predicted_re
maining_life), 1) AS avg_life, MIN(predicted_remaining_life) AS min_life, MAX(predicted_remaining_life) AS max_life FROM {{ ref('clean_machi
ne_data') }} GROUP BY machine_status ORDER BY machine_status"
17:21:57 Running with dbt=1.1.12
17:21:58 Registered adapter: duckdb=1.10.1
17:21:58 Found 4 models, 4 data tests, 1 sql operation, 1 source, 485 macros
17:21:59 Concurrency: 1 threads (target='dev')
17:21:59
Providing inline node:
| machine_status | avg_life | min_life | max_life |
|-----|-----|-----|-----|
| 0 | 234.5 | 1 | 499 |
| 1 | 234.4 | 1 | 499 |
| 2 | 233.1 | 1 | 499 |
```

FIGURE 4.19 – Moyenne de `predicted_remaining_life` selon l'état de la machine

Interprétation : La moyenne de `predicted_remaining_life` est quasiment identique selon l'état de la machine (234,5 / 234,4 / 233,1), suggérant une faible valeur prédictive intrinsèque dans ce jeu de données.

4.4.7 Remarques sur le jeu de données

Le jeu de données utilisé dans ce projet étant un jeu de données synthétique, disponible publiquement sur la plateforme Kaggle, et les incohérences observées n'étant documentées dans aucune source officielle, la décision méthodologique a été prise de ne procéder à aucune correction arbitraire de ces observations. Elles ont été exclusivement documentées, afin que l'équipe Machine Learning dispose de l'ensemble des informations nécessaires pour décider de l'usage qu'elle souhaite faire de ces variables.

4.5 Tests qualité

Des tests de qualité automatisés ont été mis en place à l'aide des fonctionnalités natives de dbt, complétées par le package externe `dbt_utils`. Ces tests se répartissent en tests structurels (`not_null`, unicité) et tests métier (plages de valeurs, valeurs catégorielles autorisées).

4.5.1 Tests sur le modèle clean_machine_data

```
! clean_machine_data.yml X
predictive_maintenance > models > marts > ! clean_machine_data.yml
1 version: 2
2
3 models:
4   - name: clean_machine_data
5     description: "Données nettoyées des capteurs machines, après suppression des vibrations négatives"
6     columns:
7       - name: machine_id
8         description: "Identifiant de la machine."
9         data_tests:
10           - not_null
11
12       - name: recorded_at
13         description: "Horodatage de la mesure."
14         data_tests:
15           - not_null
16
17       - name: vibration
18         description: "Niveau de vibration mesuré (doit être positif ou nul)."
19         data_tests:
20           - dbt_utils.expression_is_true:
21             arguments:
22               expression: ">= 0"
23
24       - name: temperature
25         description: "Température mesurée."
26         data_tests:
27           - dbt_utils.expression_is_true:
28             arguments:
29               expression: "between 0 and 200"
30
31       - name: humidity
32         description: "Humidité mesurée, en pourcentage."
```

FIGURE 4.20 – Fichier de configuration des tests clean_machine_data.yml

Interprétation : Ce fichier YAML documente, pour chaque colonne du modèle, sa description et les tests de qualité associés, notamment un test personnalisé vérifiant que la vibration est toujours positive, et un test limitant les valeurs de machine_status à 0, 1 et 2.

```
21:49:02 1 of 13 START test accepted_values_clean_machine_data_anomaly_flag_0_1 ..... [RUN]
21:49:02 1 of 13 PASS accepted_values_clean_machine_data_anomaly_flag_0_1 ..... [PASS in 0.24s]
21:49:02 2 of 13 START test accepted_values_clean_machine_data_failure_type_Normal_Overheating_PressureDrop_Vibration_Issue_Electrical
1:Fault ..... [RUN]
21:49:03 2 of 13 PASS accepted_values_clean_machine_data_failure_type_Normal_Overheating_PressureDrop_Vibration_Issue_Electrical Fault
t ..... [PASS in 0.11s]
21:49:03 3 of 13 START test accepted_values_clean_machine_data_machine_status_0_1_2 ..... [RUN]
21:49:03 3 of 13 PASS accepted_values_clean_machine_data_machine_status_0_1_2 ..... [PASS in 0.12s]
21:49:03 4 of 13 START test accepted_values_clean_machine_data_maintenance_required_0_1 ..... [RUN]
21:49:03 4 of 13 PASS accepted_values_clean_machine_data_maintenance_required_0_1 ..... [PASS in 0.10s]
21:49:03 5 of 13 START test dbt_utils.expression_is_true_clean_machine_data_downtime_risk_between_0_and_1 ..... [RUN]
21:49:03 5 of 13 PASS dbt_utils.expression_is_true_clean_machine_data_downtime_risk_between_0_and_1 ..... [PASS in 0.10s]
21:49:03 6 of 13 START test dbt_utils.expression_is_true_clean_machine_data_humidity_between_0_and_100 ..... [RUN]
21:49:03 6 of 13 PASS dbt_utils.expression_is_true_clean_machine_data_humidity_between_0_and_100 ..... [PASS in 0.08s]
21:49:03 7 of 13 START test dbt_utils.expression_is_true_clean_machine_data_predicted_remaining_life_0 ..... [RUN]
21:49:03 7 of 13 PASS dbt_utils.expression_is_true_clean_machine_data_predicted_remaining_life_0 ..... [PASS in 0.08s]
21:49:03 8 of 13 START test dbt_utils.expression_is_true_clean_machine_data_pressure_0 ..... [RUN]
21:49:03 8 of 13 PASS dbt_utils.expression_is_true_clean_machine_data_pressure_0 ..... [PASS in 0.10s]
21:49:03 9 of 13 START test dbt_utils.expression_is_true_clean_machine_data_temperature_between_0_and_200 ..... [RUN]
21:49:03 9 of 13 PASS dbt_utils.expression_is_true_clean_machine_data_temperature_between_0_and_200 ..... [PASS in 0.11s]
21:49:03 10 of 13 START test dbt_utils.expression_is_true_clean_machine_data_vibration_0 ..... [RUN]
21:49:03 10 of 13 PASS dbt_utils.expression_is_true_clean_machine_data_vibration_0 ..... [PASS in 0.09s]
21:49:04 11 of 13 PASS dbt_utils.unique_combination_of_columns_clean_machine_data_machine_id_recorded_at ..... [PASS in 0.12s]
21:49:04 12 of 13 START test not_null_clean_machine_data_machine_id ..... [RUN]
21:49:04 12 of 13 PASS not_null_clean_machine_data_machine_id ..... [PASS in 0.09s]
21:49:04 13 of 13 START test not_null_clean_machine_data_recorded_at ..... [RUN]
21:49:04 13 of 13 PASS not_null_clean_machine_data_recorded_at ..... [PASS in 0.09s]
21:49:04 Finished running 13 data tests in 0 hours 0 minutes and 2.07 seconds (2.07s).
21:49:04 Completed successfully
21:49:04 Done. PASS=13 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=13
```

FIGURE 4.21 – Résultat de l'exécution des tests sur clean_machine_data

Interprétation : L'exécution de la commande `dbt test -select clean_machine_data` confirme que l'ensemble des 13 tests définis passent avec succès (PASS=13, ERROR=0).

4.5.2 Tests sur le modèle ml_features

```
1 ml_features.yml X
predictive_maintenance > models > marts > / ml_features.yml
2
3 version: 2
4
5 models:
6   - name: ml_features
7     description: "Table finale prête pour le Machine Learning. Contient les données nettoyées et documentée"
8     config:
9       contract:
10         enforced: true
11     columns:
12       - name: machine_id
13         description: "Identifiant de la machine."
14         data_type: bigint
15         data_tests:
16           - not_null
17       - name: recorded_at
18         description: "Horodatage de la mesure, avec fuseau horaire."
19         data_type: timestamp with time zone
20         data_tests:
21           - not_null
22       - name: temperature
23         description: "Température mesurée par le capteur (plage observée : 35 à 122)."
24         data_type: double
25       - name: vibration
26         description: "Niveau de vibration mesuré. Valeurs négatives déjà exclues en amont (règle métier : \
27         data_type: double
28       - name: humidity
29         description: "Humidité relative mesurée, en pourcentage (0 à 100)."
```

FIGURE 4.22 – Fichier de configuration des tests et du contrat ml_features.yml

Interprétation : Ce fichier intègre l'activation du contrat de données natif de dbt (contract: enforced: true), qui impose une vérification stricte de la correspondance entre les types de colonnes déclarés et les types réels produits par la requête SQL.

```
• (venv_dbt) PS C:\Users\XPRISTO\Desktop\projet_mlops\predictive_maintenance> dbt test --select ml_features
22:08:31 Running with dbt=1.11.12
22:08:33 Registered adapter: duckdb=1.10.1
22:08:37 Found 5 models, 22 data tests, 1 source, 601 macros
22:08:37 Concurrency: 1 threads (target='dev')
22:08:37
22:08:38 1 of 5 START test accepted values ml_features_maintenance_required_0_1 ..... [RUN]
22:08:38 1 of 5 PASS accepted values ml_features_maintenance_required_0_1 ..... [PASS in 0.19s]
22:08:38 2 of 5 START test dbt_utils.unique_combination_of_columns ml_features_machine_id_recorded_at ..... [RUN]
22:08:39 2 of 5 PASS dbt_utils.unique_combination_of_columns ml_features_machine_id_recorded_at ..... [PASS in 0.13s]
22:08:39 4 of 5 START test not_null ml_features_maintenance_required ..... [RUN]
22:08:39 4 of 5 PASS not_null ml_features_maintenance_required ..... [PASS in 0.09s]
22:08:39 5 of 5 START test not_null ml_features_recorded_at ..... [RUN]
22:08:39 5 of 5 PASS not_null ml_features_recorded_at ..... [PASS in 0.07s]
22:08:39
22:08:39 Finished running 5 data tests in 0 hours 0 minutes and 1.59 seconds (1.59s).
22:08:39 Completed successfully
22:08:39
22:08:39 Done. PASS=5 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=5
```

FIGURE 4.23 – Résultat de l'exécution des tests sur ml_features

Interprétation : L'exécution de la commande dbt test --select ml_features confirme que l'ensemble des 5 tests définis passent avec succès.

4.5.3 Vue d'ensemble des résultats de tests

```
(venv_dbt) PS C:\Users\VPRISTO\Desktop\projet_alops\predictive_maintenance> dbt test
23:05:46 4 of 18 PASS accepted_values_clean_machine_data_maintenance_required_0_1 ..... [PASS in 0.09s]
23:05:46 5 of 18 START test_accepted_values_ml_features_maintenance_required_0_1 ..... [RUN]
23:05:46 5 of 18 PASS accepted_values_ml_features_maintenance_required_0_1 ..... [PASS in 0.08s]
23:05:46 6 of 18 START test_dbt_utils_expression_is_true_clean_machine_data_downtime_risk_between_0_and_1 [RUN]
23:05:46 6 of 18 PASS dbt_utils_expression_is_true_clean_machine_data_downtime_risk_between_0_and_1 [PASS in 0.12s]
23:05:46 7 of 18 START test_dbt_utils_expression_is_true_clean_machine_data_humidity_between_0_and_100 [RUN]
23:05:46 7 of 18 PASS dbt_utils_expression_is_true_clean_machine_data_humidity_between_0_and_100 [PASS in 0.13s]
23:05:46 8 of 18 START test_dbt_utils_expression_is_true_clean_machine_data_predicted_remaining_life_0 [RUN]
23:05:46 8 of 18 PASS dbt_utils_expression_is_true_clean_machine_data_predicted_remaining_life_0 [PASS in 0.11s]
23:05:46 9 of 18 START test_dbt_utils_expression_is_true_clean_machine_data_pressure_0 [RUN]
23:05:46 9 of 18 PASS dbt_utils_expression_is_true_clean_machine_data_pressure_0 ..... [PASS in 0.10s]
23:05:46 10 of 18 START test_dbt_utils_expression_is_true_clean_machine_data_temperature_between_0_and_200 [RUN]
23:05:46 10 of 18 PASS dbt_utils_expression_is_true_clean_machine_data_temperature_between_0_and_200 [PASS in 0.10s]
23:05:46 11 of 18 START test_dbt_utils_expression_is_true_clean_machine_data_vibration_0 [RUN]
23:05:47 11 of 18 PASS dbt_utils_expression_is_true_clean_machine_data_vibration_0 .... [PASS in 0.08s]
23:05:47 12 of 18 START test_dbt_utils_unique_combination_of_columns_clean_machine_data_machine_id_recorded_at [RUN]
23:05:47 12 of 18 PASS dbt_utils_unique_combination_of_columns_clean_machine_data_machine_id_recorded_at [PASS in 0.18s]
23:05:47 13 of 18 START test_dbt_utils_unique_combination_of_columns_ml_features_machine_id_recorded_at [RUN]
23:05:47 13 of 18 PASS dbt_utils_unique_combination_of_columns_ml_features_machine_id_recorded_at [PASS in 0.16s]
23:05:47 14 of 18 START test_not_null_clean_machine_data_machine_id ..... [RUN]
23:05:47 14 of 18 PASS not_null_clean_machine_data_machine_id ..... [PASS in 0.12s]
23:05:47 15 of 18 START test_not_null_clean_machine_data_recorded_at ..... [RUN]
23:05:47 15 of 18 PASS not_null_clean_machine_data_recorded_at ..... [PASS in 0.08s]
23:05:47 16 of 18 START test_not_null_ml_features_machine_id ..... [RUN]
23:05:47 16 of 18 PASS not_null_ml_features_machine_id ..... [PASS in 0.10s]
23:05:47 17 of 18 START test_not_null_ml_features_maintenance_required ..... [RUN]
23:05:47 17 of 18 PASS not_null_ml_features_maintenance_required ..... [PASS in 0.07s]
23:05:47 18 of 18 START test_not_null_ml_features_recorded_at ..... [RUN]
23:05:47 18 of 18 PASS not_null_ml_features_recorded_at ..... [PASS in 0.15s]
23:05:48
23:05:48 Finished running 18 data tests in 0 hours 0 minutes and 3.40 seconds (3.40s).
23:05:48 Completed successfully
23:05:48 Done. PASS=18 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=18
```

FIGURE 4.24 – Résultat global de l'ensemble des tests du projet

Interprétation : L'exécution de la commande `dbt test`, sans restriction de sélection, lance l'intégralité des 18 tests définis sur le projet et confirme leur réussite totale.

4.6 Data Contract

Un Data Contract a été formalisé pour la table `ml_features`, décrivant précisément le schéma garanti à l'équipe Machine Learning, implémenté techniquement via `contract: enforced: true`.

Colonne	Type	Description	Règle de validation
machine_id	BIGINT	Identifiant unique de la machine	NOT NULL
recorded_at	TIMESTAMP WITH TIME ZONE	Horodatage de la mesure	NOT NULL ; unicité avec (machine_id, recorded_at)
temperature	DOUBLE	Température mesurée par le capteur (°C)	Valeur comprise entre 0 et 200
vibration	DOUBLE	Niveau de vibration mesuré	Valeur ≥ 0
humidity	DOUBLE	Humidité relative (%)	Valeur comprise entre 0 et 100
pressure	DOUBLE	Pression mesurée	Valeur > 0
energy_consumption	DOUBLE	Consommation énergétique instantanée	Type numérique (DOUBLE)
machine_status	BIGINT	État opérationnel de la machine	Valeurs autorisées : 0, 1, 2
anomaly_flag	BIGINT	Indicateur de détection d'anomalie	Valeurs autorisées : 0, 1
failure_type	VARCHAR	Type de panne détectée ou état normal	Normal, Overheating, Pressure Drop, Vibration Issue, Electrical Fault
downtime_risk	DOUBLE	Score estimé du risque d'arrêt	Valeur comprise entre 0 et 1
predicted_remaining_life	BIGINT	Durée de vie restante estimée	Valeur ≥ 0
maintenance_required	BIGINT	Variable cible : maintenance requise ou non	NOT NULL ; valeurs autorisées : 0, 1

TABLE 4.1 – Data Contract de la table `ml_features`

4.7 Documentation des données

La documentation technique du projet a été générée automatiquement à l'aide de la commande `dbt docs generate`.

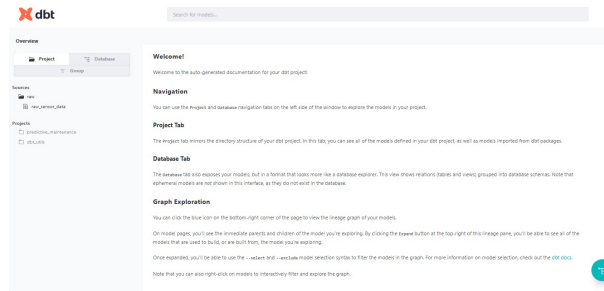


FIGURE 4.25 – Page d'accueil de la documentation dbt

Interprétation : Cette page constitue le point d'entrée de la documentation, présentant l'arborescence complète du projet dans le menu de navigation.

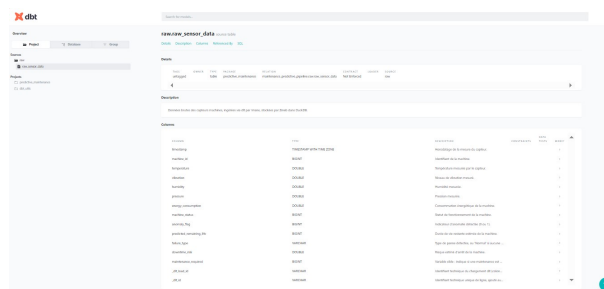


FIGURE 4.26 – Page de documentation de la source `raw.raw_sensor_data`

Interprétation : Cette page présente le détail de la source brute déclarée : sa description générale, ainsi que la liste exhaustive de ses colonnes.

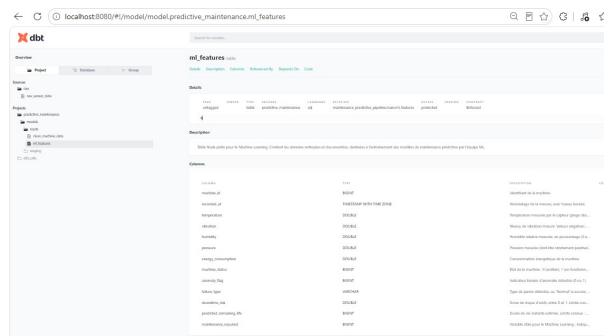


FIGURE 4.27 – Page de documentation du modèle `ml_features`

Interprétation : Cette page présente le détail complet de la table finale : sa description, son type de matérialisation, et la mention explicite « Contract : Enforced », confirmant que le contrat de données est bien activé.

4.8 Résultats des tests

Les résultats obtenus peuvent être synthétisés comme suit :

- Un projet dbt fonctionnel, comprenant 1 source déclarée et 3 modèles de transformation.
- Un data profiling exhaustif portant sur les 14 colonnes, ayant permis d'identifier une anomalie corrigée et plusieurs particularités documentées sans correction arbitraire.
- 18 tests de qualité automatisés, tous passés avec succès (PASS=18, ERROR=0).
- Un Data Contract formalisé et techniquement activé.
- Une documentation technique générée automatiquement et navigable.
- Une table finale `ml_features` comprenant 99 963 observations.

```

(venv_dbt) PS C:\Users\JVRISTO\Desktop\projet_ml_ops\predictive_maintenance> dbt test
23:05:46 4 of 18 PASS accepted values clean_machine_data.maintenance_required_0_1 ..... [PASS in 0.09s]
23:05:46 5 of 18 START test_accepted_values_ml_features.maintenance_required_0_1 ..... [RUN]
23:05:46 5 of 18 PASS accepted values_ml_features.maintenance_required_0_1 ..... [PASS in 0.08s]
23:05:46 6 of 18 START test_dbt_utils.expression_is_true_clean_machine_data.downtime_risk_between_0_and_1 [RUN]
23:05:46 6 of 18 PASS dbt_utils.expression_is_true_clean_machine_data.downtime_risk_between_0_and_1 [PASS in 0.12s]
23:05:46 7 of 18 START test_dbt_utils.expression_is_true_clean_machine_data.humidity_between_0_and_100 [RUN]
23:05:46 7 of 18 PASS dbt_utils.expression_is_true_clean_machine_data.humidity_between_0_and_100 [PASS in 0.13s]
23:05:46 8 of 18 START test_dbt_utils.expression_is_true_clean_machine_data.predicted_remaining_life_0 [RUN]
23:05:46 8 of 18 PASS dbt_utils.expression_is_true_clean_machine_data.predicted_remaining_life_0 [PASS in 0.11s]
23:05:46 9 of 18 START test_dbt_utils.expression_is_true_clean_machine_data.pressure_0 [RUN]
23:05:46 9 of 18 PASS dbt_utils.expression_is_true_clean_machine_data.pressure_0 ..... [PASS in 0.10s]
23:05:46 10 of 18 START test_dbt_utils.expression_is_true_clean_machine_data.temperature_between_0_and_200 [RUN]
23:05:46 10 of 18 PASS dbt_utils.expression_is_true_clean_machine_data.temperature_between_0_and_200 [PASS in 0.10s]
23:05:46 11 of 18 START test_dbt_utils.expression_is_true_clean_machine_data.vibration_0 [RUN]
23:05:47 11 of 18 PASS dbt_utils.expression_is_true_clean_machine_data.vibration_0 .... [PASS in 0.08s]
23:05:47 12 of 18 START test_dbt_utils.unique_combination_of_columns_clean_machine_data.machine_id_recorded_at [RUN]
23:05:47 12 of 18 PASS dbt_utils.unique_combination_of_columns_clean_machine_data.machine_id_recorded_at [PASS in 0.18s]
23:05:47 13 of 18 START test_dbt_utils.unique_combination_of_columns_ml_features.machine_id_recorded_at [RUN]
23:05:47 13 of 18 PASS dbt_utils.unique_combination_of_columns_ml_features.machine_id_recorded_at [PASS in 0.16s]
23:05:47 14 of 18 START test_not_null_clean_machine_data.machine_id ..... [RUN]
23:05:47 14 of 18 PASS not_null_clean_machine_data.machine_id ..... [PASS in 0.12s]
23:05:47 15 of 18 START test_not_null_clean_machine_data.recorded_at ..... [RUN]
23:05:47 15 of 18 PASS not_null_clean_machine_data.recorded_at ..... [PASS in 0.08s]
23:05:47 16 of 18 START test_not_null_ml_features.machine_id ..... [RUN]
23:05:47 16 of 18 PASS not_null_ml_features.machine_id ..... [PASS in 0.10s]
23:05:47 17 of 18 START test_not_null_ml_features.maintenance_required ..... [RUN]
23:05:47 17 of 18 PASS not_null_ml_features.maintenance_required ..... [PASS in 0.07s]
23:05:47 18 of 18 START test_not_null_ml_features.recorded_at ..... [RUN]
23:05:47 18 of 18 PASS not_null_ml_features.recorded_at ..... [PASS in 0.15s]
23:05:48
23:05:48 Finished running 18 data tests in 0 hours 0 minutes and 3.40 seconds (3.40s).
23:05:48
23:05:48 Completed successfully
23:05:48
23:05:48 Done. PASS=18 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=18

```

FIGURE 4.28 – Synthèse finale de l'exécution des tests du projet

Interprétation : Ce résultat global constitue la preuve finale, reproductible à tout moment, que l'ensemble du pipeline respecte les règles de qualité définies tout au long de ce chapitre.

Conclusion

Ce chapitre a permis de mettre en place, de bout en bout, un pipeline de transformation de données conforme aux standards de l'ingénierie analytique moderne, depuis la donnée brute jusqu'à une table finale documentée, testée et contractualisée. La démarche suivie — diagnostic par le data profiling, décision de nettoyage justifiée par les chiffres, implémentation dans le code, puis vérification systématique par les tests automatisés — garantit à l'équipe Machine Learning une table de données fiable, sur laquelle les futures évolutions du pipeline pourront s'appuyer sans risque de régression silencieuse. Ce travail constitue également une base solide pour l'intégration de ce pipeline dans l'orchestration automatisée Dagster, et un exemple reproductible de la méthodologie attendue d'un Analytics Engineer et Data Quality Engineer en contexte de projet MLOps/DataOps industriel.

Chapitre 5: Orchestration avec Dagster

Introduction

Ce chapitre présente la mise en place de l'orchestration du pipeline de données du projet de maintenance prédictive industrielle, à l'aide de l'outil Dagster. Après avoir développé séparément les briques d'ingestion, de transformation et de préparation des données, il était nécessaire de les faire communiquer entre elles de manière fiable et automatisée. Nous détaillons dans ce chapitre le rôle de Dagster dans l'architecture globale du projet, la manière dont chaque étape du pipeline a été modélisée sous forme d'assets, les dépendances établies entre ces étapes, ainsi que les résultats obtenus lors de l'exécution complète du pipeline. Nous présentons également l'organisation du travail d'équipe selon la méthodologie Agile mise en place tout au long du projet.

5.1 Rôle de Dagster dans le projet

Dans le cadre du projet de maintenance prédictive industrielle, le pipeline de données mobilise plusieurs technologies distinctes réparties entre différents membres de l'équipe : l'ingestion des données de capteurs avec dlt, le stockage et les transformations avec DuckDB et dbt, puis l'entraînement d'un modèle de Machine Learning. Chacune de ces étapes a été développée de manière indépendante, ce qui pose la question de leur enchaînement fiable, reproductible et supervisable.

Dagster a été retenu comme outil d'orchestration DataOps pour répondre à ce besoin. Il permet de définir chaque étape du pipeline comme un *asset* (une donnée produite), de déclarer explicitement les dépendances entre ces assets, et d'exécuter l'ensemble du pipeline — ou une partie seulement — depuis une interface web centralisée. Dagster prend également en charge la planification des exécutions, la capture des journaux d'exécution (logs) et la détection automatique des échecs.

Notre rôle a consisté à installer et configurer Dagster, à modéliser chaque étape du pipeline existant sous forme d'assets, à vérifier la cohérence des dépendances entre ces étapes, puis à exécuter et superviser le pipeline complet.

5.2 Architecture du pipeline orchestré

Le pipeline orchestré suit la chaîne de traitement suivante, correspondant à la répartition des tâches au sein de l'équipe :

- **Ingestion (Imane)** — extraction du fichier CSV des capteurs et chargement dans le schéma `raw` de DuckDB via `dlt` ;
- **Transformation (Sara)** — modèles dbt en couches successives : staging (`stg_machine_data`), nettoyage (`clean_machine_data`), puis table finale (`ml_features`) ;
- **Vérification (Amina)** — contrôle de la disponibilité et de la qualité des données finales avant leur mise à disposition pour le Machine Learning ;
- **Entraînement (Maroua)** — déclenchement du script d'entraînement du modèle prédictif à partir de la table `ml_features`.

La figure 5.1 présente la vue globale du lineage des assets telle qu'affichée par l'interface Dagster, organisée par groupes fonctionnels : `ingestion` (1 asset), `default` (3 assets dbt) et `ml` (2 assets). La flèche entre les groupes `default` et `ml` matérialise la dépendance : les modèles dbt doivent être matérialisés avec succès avant que la vérification finale et l'entraînement puissent démarrer.

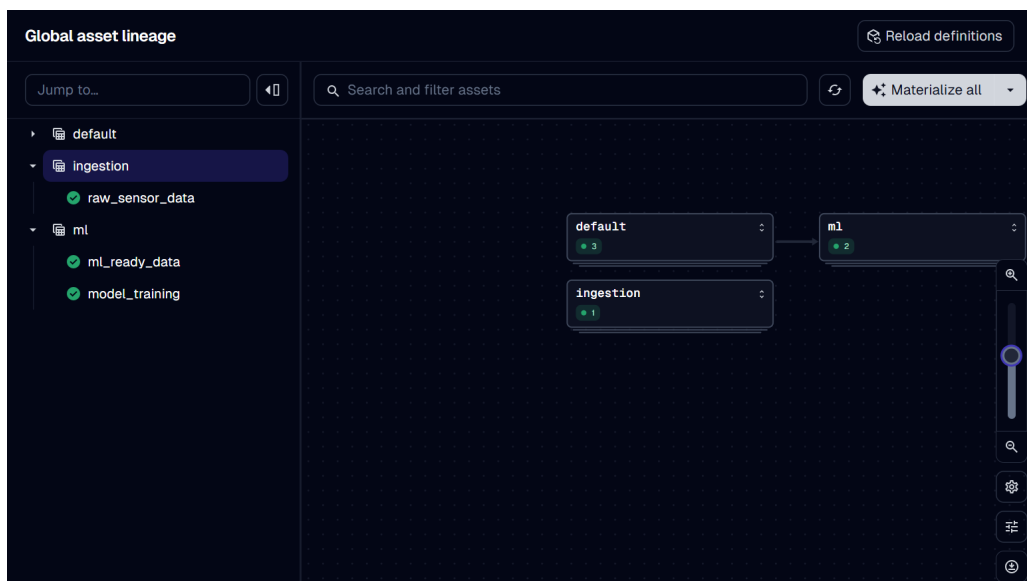


FIGURE 5.1 – Vue globale du lineage des assets dans Dagster

5.3 Description des assets et jobs

Chaque étape du pipeline a été modélisée comme un asset Dagster distinct. Le tableau 5.1 résume les six assets définis, l'étape qu'ils orchestrent et le membre de l'équipe à l'origine du code sous-jacent.

Asset Dagster	Étape orchestrée	Rôle / responsable	Groupe
<code>raw_sensor_data</code>	Ingestion des données brutes des capteurs via dlt	Script d’Imane, orchestré par Amina	ingestion
<code>stg_machine_data</code>	Standardisation des données brutes (couche staging dbt)	Modèle dbt de Sara, orchestré par Amina	default (dbt)
<code>clean_machine_data</code>	Nettoyage des données (suppression des valeurs physiquement impossibles)	Modèle dbt de Sara, orchestré par Amina	default (dbt)
<code>ml_features</code>	Construction de la table finale prête pour le Machine Learning	Modèle dbt de Sara, orchestré par Amina	default (dbt)
<code>ml_ready_data</code>	Vérification de la disponibilité et de la qualité des données finales	Amina	ml
<code>model_training</code>	Déclenchement de l’entraînement du modèle prédictif	Amina (orchestration) / Maroua (script)	ml

TABLE 5.1 – Assets Dagster du pipeline orchestré

Les trois assets dbt (`stg_machine_data`, `clean_machine_data`, `ml_features`) sont générés automatiquement par Dagster à partir du manifeste du projet dbt de Sara, grâce à l’intégration officielle `dagster-dbt`. Cette intégration exécute la commande `dbt build`, qui combine les transformations (`dbt run`) et les tests de qualité des données (`dbt test`) en une seule étape supervisée.

5.4 Dépendances entre les étapes

Les dépendances entre assets sont déclarées explicitement dans le code et vérifiées automatiquement par Dagster avant chaque exécution :

- `raw_sensor_data` ne dépend d’aucune autre étape : c’est le point d’entrée du pipeline ;
- `stg_machine_data` dépend de `raw_sensor_data` — cette dépendance est établie automatiquement par Dagster, car le nom de la source déclarée dans le projet dbt (`raw_sensor_data`) correspond exactement au nom de l’asset d’ingestion ;

- `clean_machine_data` dépend de `stg_machine_data`, et `ml_features` dépend de `clean_machine_data`, conformément à la logique des modèles dbt ;
- `ml_ready_data` dépend de `ml_features` : la vérification ne peut avoir lieu que si la table finale a été construite avec succès ;
- `model_training` dépend de `ml_ready_data` : l'entraînement n'est déclenché que si les données ont été validées.

Cette déclaration explicite des dépendances garantit qu'aucune étape ne peut s'exécuter avant que ses prérequis ne soient satisfaits, et permet à Dagster de déterminer automatiquement l'ordre d'exécution du pipeline complet.

5.5 Exécution du pipeline

Le pipeline a été exécuté dans son ensemble depuis l'interface Dagster (fonctionnalité *Materialize all*). L'exécution complète — ingestion des données, transformations dbt, exécution des tests de qualité, vérification finale — s'est déroulée avec succès, comme le montre la figure 5.2 : les six assets et les dix-huit tests de qualité (*checks*) associés aux modèles dbt ont été validés.

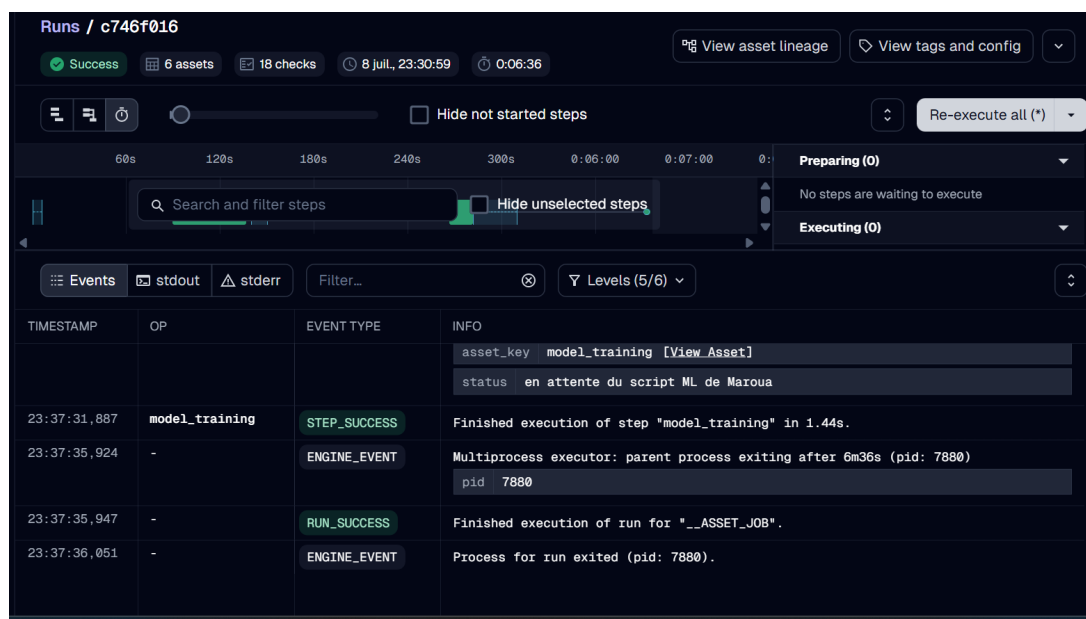


FIGURE 5.2 – Détail d'une exécution réussie du pipeline (6 assets, 18 checks, statut Success)

L'historique des exécutions, consultable depuis l'onglet *Runs* de l'interface, permet de conserver une traçabilité complète de chaque lancement du pipeline : identifiant de run, statut, durée et déclencheur (figure 5.3).

ID	Target	Launched by	Status	Created at	Duration
c746f016 View all tags (2)	6 assets 18 checks	Manual...	Success	8 juil., 23:29	0:06:36

FIGURE 5.3 – Historique des exécutions du pipeline

5.6 Logs et suivi des erreurs

Dagster capture automatiquement les journaux d'exécution (logs) de chaque étape du pipeline, consultables individuellement depuis la page de détail d'un run. Ces logs comprennent notamment :

- les messages applicatifs émis par le code (par exemple, le nombre de lignes chargées par le pipeline `dlt`) ;
- les événements techniques Dagster (démarrage et fin d'étape, statut de matérialisation) ;
- la sortie standard (`stdout`) et les éventuelles erreurs (`stderr`) de chaque commande exécutée, notamment les commandes `dbt`.

En cas d'échec d'une étape, Dagster interrompt automatiquement l'exécution des étapes qui en dépendent et signale l'échec dans l'interface, ce qui facilite l'identification rapide de l'origine du problème sans avoir à examiner l'ensemble du pipeline.

Conclusion

L'orchestration mise en place avec Dagster a permis de transformer un ensemble de scripts indépendants — ingestion, transformations `dbt`, préparation des données pour le Machine Learning — en un pipeline de données unique, fiable et supervisable depuis une interface centralisée. La déclaration explicite des dépendances entre les différentes étapes garantit un enchaînement cohérent du traitement, et la capture automatique des logs facilite l'identification rapide de toute anomalie survenant lors d'une exécution.

Ce travail constitue une base solide pour la suite du projet : dès que le script d'entraînement du modèle prédictif sera finalisé par l'équipe, il pourra être intégré directement

dans le pipeline existant, sans remise en question de l'architecture mise en place. L'orchestration développée ouvre également la voie à une automatisation complète du pipeline, par exemple via une planification périodique des exécutions, ce qui rapprocherait le projet d'un véritable système de maintenance prédictive opérationnel en production.

Chapitre 6: Machine Learning et prédiction des pannes

Introduction

Dans ce chapitre, nous présentons la partie Machine Learning du projet de maintenance prédictive. L'objectif principal est de construire un modèle capable de prédire si une machine nécessite une intervention de maintenance à partir des données industrielles préparées dans les étapes précédentes du pipeline.

Cette partie s'appuie sur la table finale `ml_features`, issue de l'ingestion, du stockage et des transformations. Le travail réalisé consiste à préparer les données pour l'apprentissage, détecter et supprimer les variables pouvant causer une fuite d'information, tester plusieurs modèles de classification, optimiser le seuil de décision et interpréter les résultats obtenus.

L'objectif n'est pas seulement d'obtenir un score élevé, mais surtout d'obtenir un modèle fiable, réaliste et défendable dans un contexte industriel.

6.1 Objectif du modèle

L'objectif de cette partie est de développer un modèle de Machine Learning capable de prédire si une machine nécessite une maintenance, à partir des données issues du pipeline DataOps. La variable cible utilisée est `maintenance_required`, qui prend la valeur 1 lorsqu'une intervention de maintenance est nécessaire, et 0 sinon.

Cette étape représente la partie intelligente du projet, car elle transforme les données nettoyées et préparées en un modèle prédictif exploitable. L'objectif métier est d'aider les équipes industrielles à anticiper les pannes, réduire les arrêts non planifiés et améliorer la disponibilité des machines.

6.2 Données utilisées

Les données utilisées proviennent de la table finale `ml_features`, générée après les étapes d'ingestion, de stockage et de transformation. Cette table contient les mesures des capteurs industriels ainsi que certaines informations liées à l'état des machines.

Le dataset final contient environ 99 963 observations après nettoyage. La variable cible est déséquilibrée : environ 19,7% des lignes correspondent à des cas où une maintenance

est requise.

6.3 Analyse du risque de fuite d'information

Lors des premiers essais, certains modèles obtenaient des performances très élevées, avec notamment une précision égale à 1.0000. Ce résultat semblait trop parfait et donc suspect.

Une analyse des variables a montré que certaines colonnes étaient trop directement liées à la cible. Par exemple, lorsque `anomaly_flag = 1`, la machine nécessitait presque toujours une maintenance. De même, certaines valeurs de `machine_status` et `failure_type` donnaient presque directement l'information sur la panne.

Ces variables créaient donc une fuite d'information, car elles donnaient au modèle une information qui ne serait pas forcément disponible avant la décision de maintenance.

Les colonnes suivantes ont donc été retirées de l'entraînement :

- `anomaly_flag`
- `machine_status`
- `failure_type`
- `downtime_risk`
- `predicted_remaining_life`

Cette décision permet d'obtenir une évaluation plus réaliste et plus défendable du modèle.

6.4 Préparation des données

Après suppression des variables de fuite, les variables utilisées pour l'entraînement sont principalement les mesures capteurs et quelques informations temporelles :

- `machine_id`
- `temperature`
- `vibration`
- `humidity`
- `pressure`
- `energy_consumption`
- `recorded_hour`
- `recorded_dayofweek`

Les valeurs temporelles ont été extraites à partir de la colonne de date afin de donner au modèle des informations sur l'heure et le jour d'observation.

Les variables numériques ont été imputées avec la médiane puis normalisées. Les variables catégorielles éventuelles ont été encodées avec un encodage One-Hot.

6.5 Feature Engineering temporel

Afin d'améliorer la performance du modèle sans réintroduire de fuite d'information, nous avons ajouté des variables temporelles par machine. L'idée est de ne pas analyser uniquement une observation isolée, mais aussi l'évolution récente des capteurs.

Pour chaque machine et pour les principaux capteurs, nous avons généré les variables suivantes :

- `lag_1` : valeur précédente du capteur ;
- `diff_1` : différence entre la valeur actuelle et la valeur précédente ;
- `roll5_mean` : moyenne glissante sur les 5 dernières mesures ;
- `roll5_std` : écart-type glissant sur les 5 dernières mesures ;
- `roll15_mean` : moyenne glissante sur les 15 dernières mesures.

Ces variables sont calculées uniquement à partir des mesures passées de chaque machine. Elles ne contiennent donc pas d'information future et ne créent pas de fuite d'information.

6.6 Séparation des données

Pour obtenir une évaluation plus fiable, les données ont été séparées en trois parties :

- 60% pour l'entraînement ;
- 20% pour la validation ;
- 20% pour le test final.

L'ensemble de validation a été utilisé pour choisir le meilleur seuil de décision. L'ensemble de test final a été conservé séparé afin d'évaluer le modèle sur des données non utilisées pendant l'optimisation.

La séparation a été faite de manière stratifiée afin de conserver la même proportion de cas positifs et négatifs dans chaque sous-ensemble.

6.7 Modèles testés

Plusieurs modèles de classification ont été testés :

- Régression Logistique ;
- Arbre de Décision ;
- Random Forest ;
- Gradient Boosting.

Chaque modèle a été intégré dans un pipeline `scikit-learn` contenant le prétraitement des données et le classifieur.

6.8 Optimisation du seuil de décision

Par défaut, un modèle de classification utilise souvent un seuil de 0.5 pour convertir une probabilité en classe. Cependant, dans un problème de maintenance prédictive, ce seuil n'est pas toujours optimal.

Nous avons donc testé plusieurs seuils entre 0.05 et 0.95 sur l'ensemble de validation. Le seuil final a été choisi pour améliorer le F1-score, tout en évitant un modèle trop conservateur donnant une précision artificiellement parfaite.

Le meilleur seuil retenu est :

$$\text{seuil} = 0.18$$

6.9 Résultats obtenus

Après suppression des variables de fuite, ajout des variables temporelles et optimisation du seuil, le meilleur modèle obtenu est le **Gradient Boosting**.

Métrique	Valeur
Accuracy	0.8909
Precision	0.9531
Recall	0.4693
F1-score	0.6289
ROC-AUC	0.7369
Seuil de décision	0.18

TABLE 6.1 – Performances finales du meilleur modèle

6.10 Comparaison des modèles

Modèle	Accuracy	Precision	Recall	F1-score	ROC-AUC
Gradient Boosting	0.8909	0.9531	0.4693	0.6289	0.7369
Decision Tree	0.8895	0.9404	0.4688	0.6257	0.7362
Random Forest	0.8881	0.9250	0.4700	0.6233	0.7408
Logistic Regression	0.7972	0.4844	0.4601	0.4719	0.6992

TABLE 6.2 – Comparaison des modèles testés

Le modèle Gradient Boosting a été retenu car il offre le meilleur compromis global entre précision, recall et F1-score. Même si Random Forest obtient un ROC-AUC légèrement supérieur, le Gradient Boosting présente le meilleur F1-score, qui est une métrique importante dans un problème déséquilibré.

6.11 Interprétation des résultats

La précision finale de 0.9531 signifie que lorsque le modèle déclenche une alerte de maintenance, celle-ci est généralement fiable. Autrement dit, le modèle produit peu de fausses alertes.

Cependant, le recall de 0.4693 montre que le modèle ne détecte qu'environ 47% des vrais cas de maintenance. Cela signifie qu'il manque encore une partie importante des machines nécessitant réellement une intervention.

Le F1-score de 0.6289 indique un compromis moyen entre précision et recall. Le ROC-AUC de 0.7369 montre que le modèle apprend un vrai signal dans les données, mais que ce signal reste limité.

6.12 Importance des variables

L'analyse de l'importance des variables montre que les deux facteurs les plus influents sont :

- la température ;
- la vibration.

Ces résultats sont cohérents avec le contexte industriel, car une température élevée ou une vibration anormale peuvent indiquer un dysfonctionnement mécanique ou un risque de panne.

Les variables temporelles ajoutées, comme les moyennes glissantes et les variations récentes, contribuent également au modèle, mais avec une importance plus faible. Elles permettent toutefois d'améliorer légèrement les performances par rapport au modèle utilisant uniquement les valeurs instantanées.

6.13 Limites du modèle

Les résultats obtenus sont réalistes et défendables, mais le modèle n'est pas encore suffisamment robuste pour une utilisation industrielle réelle.

La principale limite est le recall relativement faible. Dans un contexte industriel, manquer une panne peut avoir un coût important. Il serait donc préférable d'améliorer la détection des cas positifs, même si cela entraîne une légère baisse de la précision.

Pour améliorer le modèle, il faudrait disposer de données plus riches, par exemple :

- un historique plus long par machine ;
- la date de la dernière maintenance ;
- l'âge de la machine ;
- le type de machine ;
- la charge de production ;
- des fenêtres temporelles plus longues ;
- des données réelles d'intervention maintenance.

6.14 Conclusion

Dans cette partie, nous avons construit un modèle de prédiction de maintenance en suivant une démarche progressive. Les premiers résultats étaient trop élevés à cause d'une fuite d'information. Après analyse, les variables problématiques ont été supprimées afin d'obtenir une évaluation plus fiable.

Nous avons ensuite amélioré le modèle grâce au feature engineering temporel et à l'optimisation du seuil de décision. Le modèle final retenu est un Gradient Boosting avec un F1-score de 0.6289 et un ROC-AUC de 0.7369.

Le modèle est donc capable de détecter une partie des cas de maintenance, avec des alertes généralement fiables. Cependant, son recall reste limité, ce qui montre que des données plus riches seraient nécessaires pour obtenir un modèle réellement robuste dans un contexte industriel.

Conclusion

Dans cette partie, nous avons développé un modèle de Machine Learning pour la prédiction des besoins de maintenance. Plusieurs modèles ont été testés, notamment la régression logistique, l'arbre de décision, Random Forest et Gradient Boosting.

Les premiers résultats obtenus étaient trop élevés, ce qui a permis d'identifier un problème de fuite d'information. Certaines variables donnaient presque directement la réponse au modèle, comme `anomaly_flag`, `machine_status`, `failure_type`, `downtime_risk` et `predicted_remaining_life`. Ces variables ont donc été supprimées afin d'obtenir une évaluation plus réaliste.

Pour améliorer le modèle, nous avons ajouté des variables temporelles par machine, comme les valeurs précédentes, les variations et les moyennes glissantes des capteurs. Le meilleur modèle final est Gradient Boosting, avec un F1-score de 0.6289 et un ROC-AUC de 0.7369.

Ces résultats montrent que le modèle apprend un signal réel dans les données, mais que sa capacité de détection reste limitée. Dans un contexte industriel réel, il serait nécessaire d'enrichir les données avec plus d'historique machine et des informations de maintenance afin d'obtenir un modèle plus robuste.

Chapitre 7: MLflow, versionnement et monitoring

7.1 Rôle de MLflow dans le projet

Dans le cadre du pipeline MLOps du projet, MLflow est l'outil utilisé pour assurer le suivi, la comparaison et la gestion du cycle de vie des modèles de Machine Learning entraînés par Maroua (voir chapitre précédent). Sans un tel outil, chaque entraînement de modèle nécessiterait un suivi manuel des paramètres, des métriques et des fichiers de modèles, ce qui devient rapidement source d'erreurs et de perte d'information.

MLflow répond à trois besoins précis dans ce projet :

- **Experiment Tracking** : conserver un historique structuré de chaque entraînement (paramètres utilisés, métriques obtenues) ;
- **Model Registry** : enregistrer officiellement le meilleur modèle identifié, avec un système de versionnement ;
- **Monitoring** : permettre un suivi des performances dans le temps et une détection de dérive des données.

Le script `train_model.py`, développé initialement par Maroua, a été enrichi pour intégrer le tracking MLflow autour de la logique d'entraînement déjà en place, sans modifier la méthodologie de préparation des données ni de sélection de seuil. La comparaison détaillée des quatre modèles testés (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting) ayant déjà été présentée au chapitre précédent (Table 6.2), ce chapitre se concentre sur la manière dont ces résultats sont tracés, comparés et exploités via MLflow, ainsi que sur le monitoring mis en place.

7.2 Experiment Tracking

Chaque modèle testé dans `train_model.py` est encapsulé dans un run MLflow distinct grâce à `mlflow.start_run()`. Cela permet de conserver une trace indépendante et comparable pour chacun des quatre algorithmes évalués, sans avoir à gérer manuellement des fichiers de résultats séparés.

L'expérience MLflow a été nommée `predictive_maintenance`. Au total, 5 runs ont été enregistrés : les 4 modèles testés avec succès, ainsi qu'une première tentative ayant échoué en raison d'une incompatibilité du format de sauvegarde `skops` avec certains types

de données internes (`numpy.dtype`). Cette erreur a été corrigée en spécifiant explicitement le format de sérialisation `pickle` lors de l'enregistrement du modèle.

The screenshot shows the MLflow interface for the 'predictive_maintenance' experiment. The 'Runs' tab is active, displaying a table of 5 runs. The table columns are Run Name, Created, Dataset, Duration, Source, and Models. The first four runs are successful (green status icons) and the last one is failed (red status icon).

Run Name	Created	Dataset	Duration	Source	Models
gradient_boosting	16 hours ago	-	1.2min	train_mo...	predictive_maint...
random_forest	16 hours ago	-	20.8s	train_mo...	model
decision_tree	16 hours ago	-	10.7s	train_mo...	model
logistic_regression	16 hours ago	-	17.4s	train_mo...	model
logistic_regression	16 hours ago	-	10.2s	train_mo...	model

FIGURE 7.1 – Liste des runs enregistrés dans l'expérience `predictive_maintenance`

La figure 7.1 montre les 5 runs enregistrés dans MLflow, chacun correspondant à une tentative d'entraînement. On y retrouve les 4 modèles finaux (`gradient_boosting`, `random_forest`, `decision_tree`, `logistic_regression`) marqués comme terminés avec succès, ainsi qu'un premier run `logistic_regression` en échec, correspondant à l'erreur de sérialisation mentionnée ci-dessus. Ce suivi automatique permet de retrouver à tout moment l'historique complet des tentatives, y compris les échecs, ce qu'un simple fichier de résultats ne permettrait pas de conserver aussi clairement.

7.3 Paramètres suivis

Pour chaque run, les hyperparamètres du modèle sont automatiquement extraits et enregistrés via `mlflow.log_param()`. Pour le modèle Gradient Boosting retenu comme meilleur modèle, 21 paramètres ont été suivis, incluant notamment le type de modèle (`model_type`) ainsi que les hyperparamètres propres à l'algorithme.

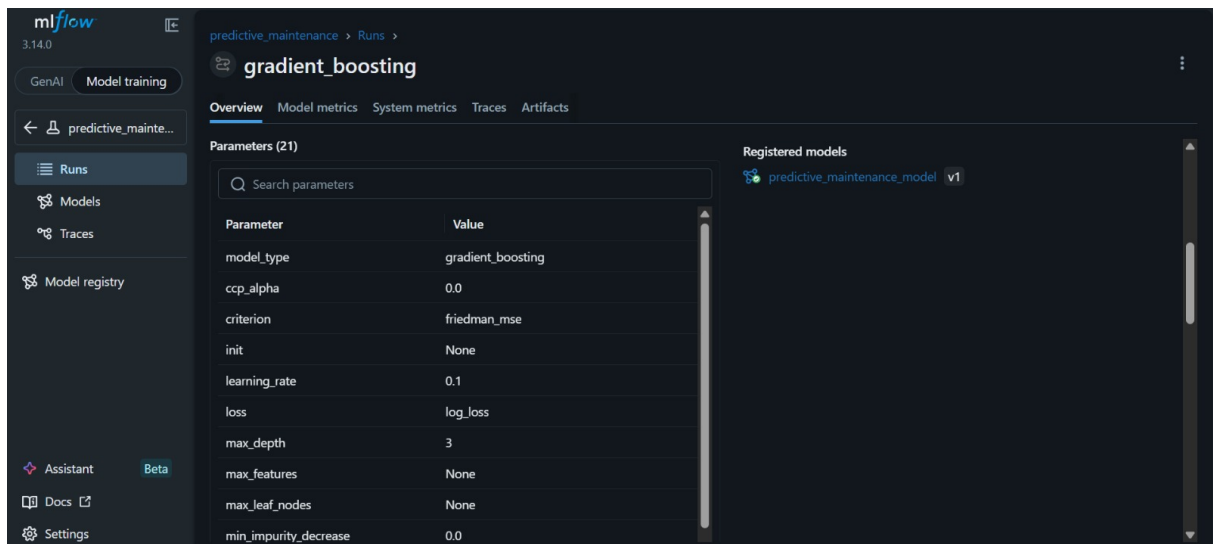


FIGURE 7.2 – Paramètres enregistrés pour le run `gradient_boosting`

La figure 7.2 liste l'ensemble des paramètres associés au run `gradient_boosting`. Cette traçabilité est essentielle en contexte MLOps : elle permet de savoir exactement quelle configuration a produit quel résultat, et de reproduire un entraînement à l'identique si nécessaire. Sans cet enregistrement automatique, il faudrait documenter manuellement chaque configuration testée, avec un risque élevé d'erreur ou d'oubli.

7.4 Métriques suivies

Sept métriques sont suivies pour chaque run, calculées sur le jeu de test (données jamais utilisées pendant l'entraînement ni pour le choix du seuil de décision) : `accuracy`, `precision`, `recall`, `f1_score`, `roc_auc`, `threshold` et `validation_f1_score`. Ces métriques correspondent à celles détaillées dans le chapitre précédent (Table 6.2), mais sont ici stockées de manière structurée et interrogeable via l'API MLflow plutôt que dans un simple fichier CSV.

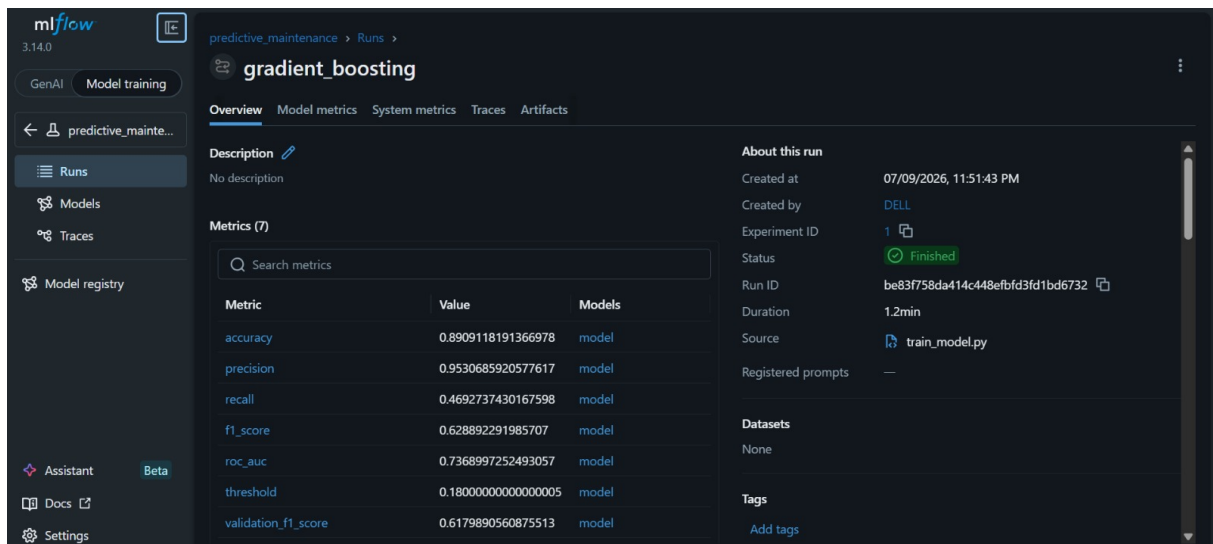


FIGURE 7.3 – Métriques enregistrées pour le run `gradient_boosting`

La figure 7.3 confirme les résultats du modèle Gradient Boosting (accuracy de 0.8909, f1-score de 0.6289), déjà présentés et interprétés au chapitre précédent. La valeur ajoutée de cet enregistrement dans MLflow est de rendre ces métriques directement comparables entre elles via l'interface, sans avoir à ouvrir plusieurs fichiers, et de conserver un lien direct entre chaque métrique et le run exact qui l'a produite.

7.5 Artefacts sauvegardés

Pour chaque run, MLflow sauvegarde automatiquement, via `mlflow.sklearn.log_model()`, le modèle entraîné ainsi que les fichiers nécessaires à sa réutilisation : `model.pkl` (le modèle sérialisé, incluant le préprocesseur), `MLmodel` (fichier de configuration), `conda.yaml` / `python_env.yaml` (environnement Python), et `requirements.txt` (dépendances).

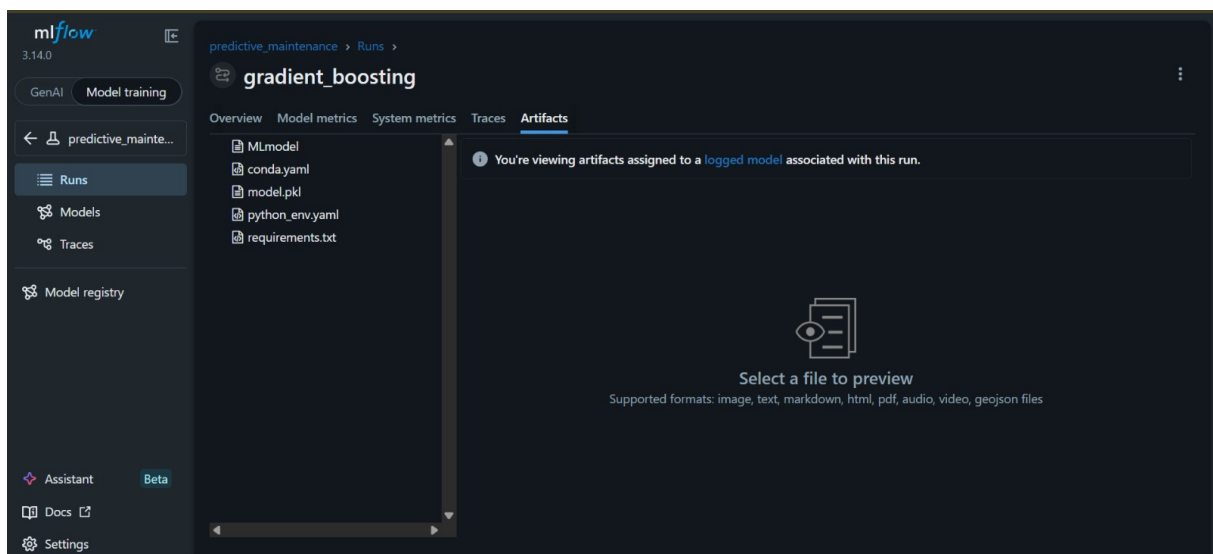


FIGURE 7.4 – Artefacts sauvegardés pour le run `gradient_boosting`

La figure 7.4 montre l'ensemble des fichiers générés automatiquement par MLflow pour le run retenu. Ces artefacts sont directement exploitables pour le déploiement du modèle : le fichier `model.pkl` peut être chargé indépendamment de MLflow, tandis que les fichiers d'environnement garantissent que le modèle sera exécuté avec les mêmes versions de bibliothèques que lors de l'entraînement. Ces fichiers seront utilisés par la partie déploiement du projet (FastAPI et Docker, chapitre suivant).

7.6 Model Registry

Une fois le meilleur modèle identifié par tri décroissant des runs (f1-score, puis ROC-AUC, puis recall en cas d'égalité), son run MLflow correspondant est utilisé pour l'enregistrer officiellement dans le Model Registry via `mlflow.register_model()`, sous le nom `predictive_maintenance_model`.

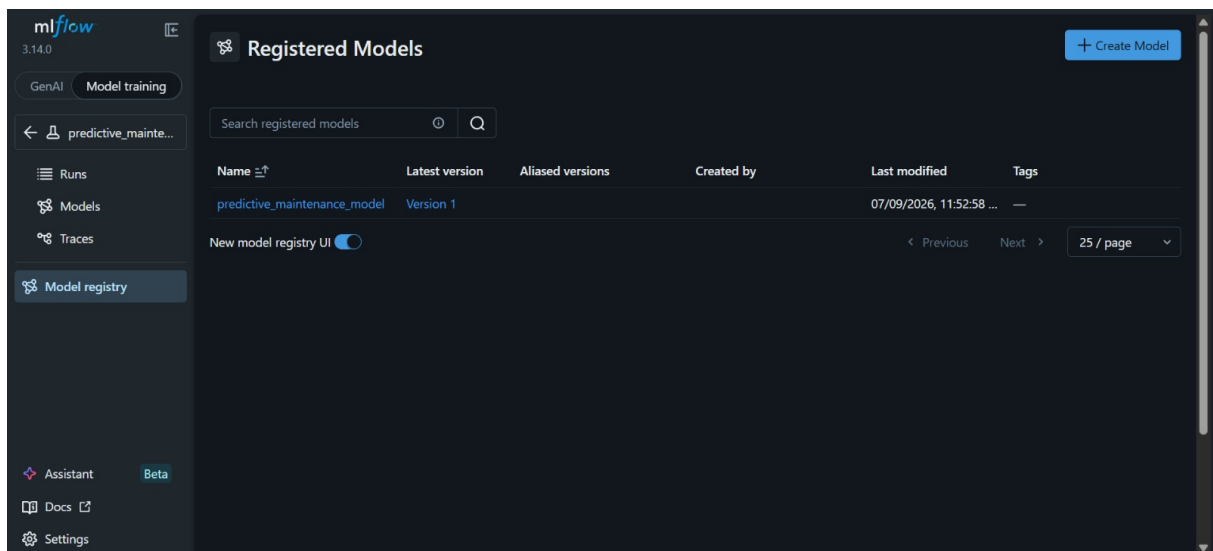


FIGURE 7.5 – Modèle enregistré dans le Model Registry

La figure 7.5 montre la vue d'ensemble du Model Registry, avec le modèle `predictive_maintenance_model` enregistré en version 1. Ce mécanisme distingue clairement les runs d'expérimentation (nombreux, dont certains sont des essais ou des échecs) du modèle officiellement validé et destiné à être utilisé en production, ce qui évite toute ambiguïté sur la version du modèle à déployer.

7.7 Versionnement du modèle

Le Model Registry attribue automatiquement un numéro de version à chaque nouvel enregistrement d'un modèle sous un même nom. Lors du premier entraînement complet, le modèle `predictive_maintenance_model` a été enregistré en **version 1**, à partir du run `gradient_boosting`.

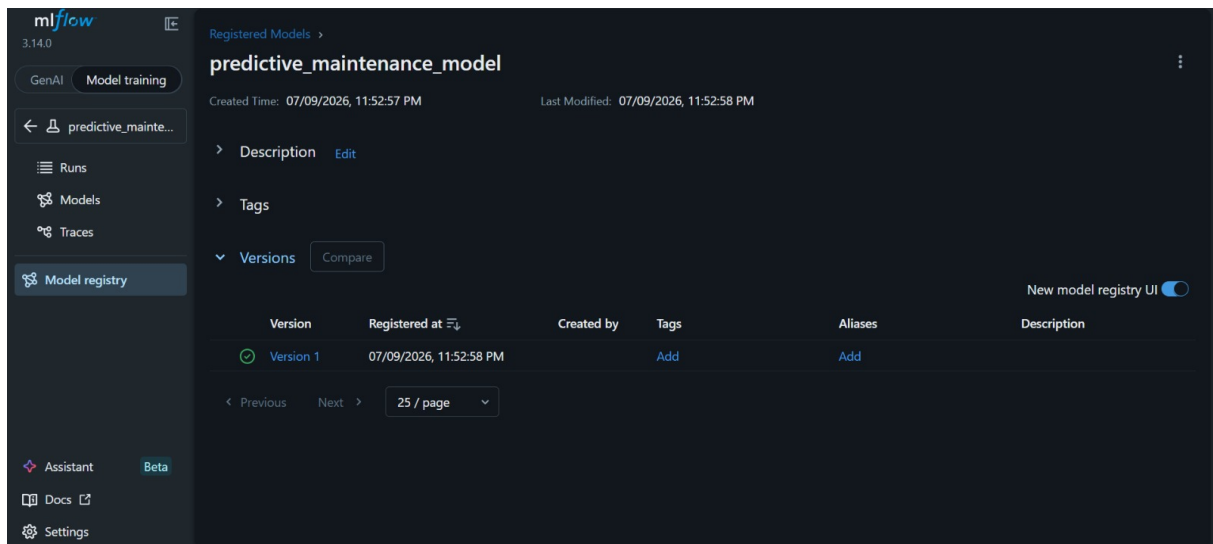


FIGURE 7.6 – Détail de la version 1 du modèle enregistré

La figure 7.6 détaille la version 1 du modèle, avec sa date d'enregistrement et son run source (`gradient_boosting`). Ce mécanisme de versionnement permettra, lors de futurs ré-entraînements avec de nouvelles données collectées en production, de conserver un historique complet de toutes les versions du modèle et de pouvoir revenir à une version antérieure en cas de besoin, sans perdre la traçabilité du modèle actuellement en production.

7.8 Monitoring ML

Un script indépendant, `monitoring.py`, a été développé pour assurer un suivi simple des performances du modèle dans le temps. Ce script interroge l'historique des runs enregistrés dans MLflow via l'API `MLflowClient`, et compare l'évolution de l'accuracy entre le premier et le dernier run enregistré.

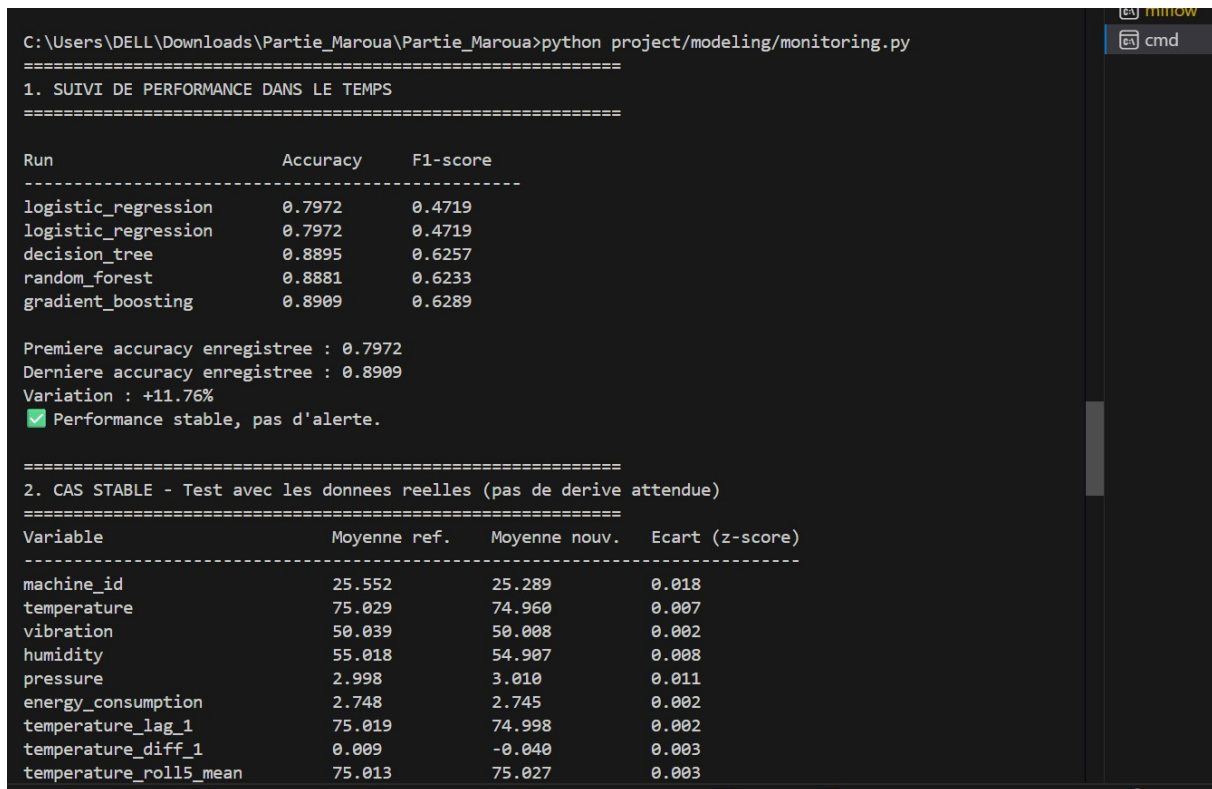


FIGURE 7.7 – Suivi de l'évolution de l'accuracy à travers les runs enregistrés

La figure 7.7 montre que l'accuracy a évolué de 0.7972 (premier modèle testé, Logistic Regression) à 0.8909 (dernier modèle testé, Gradient Boosting), soit une amélioration de **+11.76%**. Le seuil d'alerte étant fixé à une baisse de plus de 5%, aucune alerte n'a été déclenchée ici, ce qui est cohérent puisque les modèles se sont améliorés au fil des tests plutôt que dégradés. Ce mécanisme deviendrait particulièrement utile lors de ré-entraînements futurs du modèle en production, où une baisse de performance pourrait, elle, être détectée automatiquement.

7.9 Détection simple de dérive

7.9.1 Méthodologie

La détection de dérive des données (*data drift*) vise à identifier si la distribution des nouvelles données diffère significativement de celle utilisée pour l'entraînement du modèle. En l'absence de données de production réelles collectées après le déploiement, deux scénarios complémentaires ont été mis en place pour démontrer et valider ce mécanisme.

La mesure de dérive est calculée pour chaque variable numérique sous la forme d'un **z-score** :

$$z = \frac{|\mu_{\text{nouveau}} - \mu_{\text{référence}}|}{\sigma_{\text{référence}}}$$

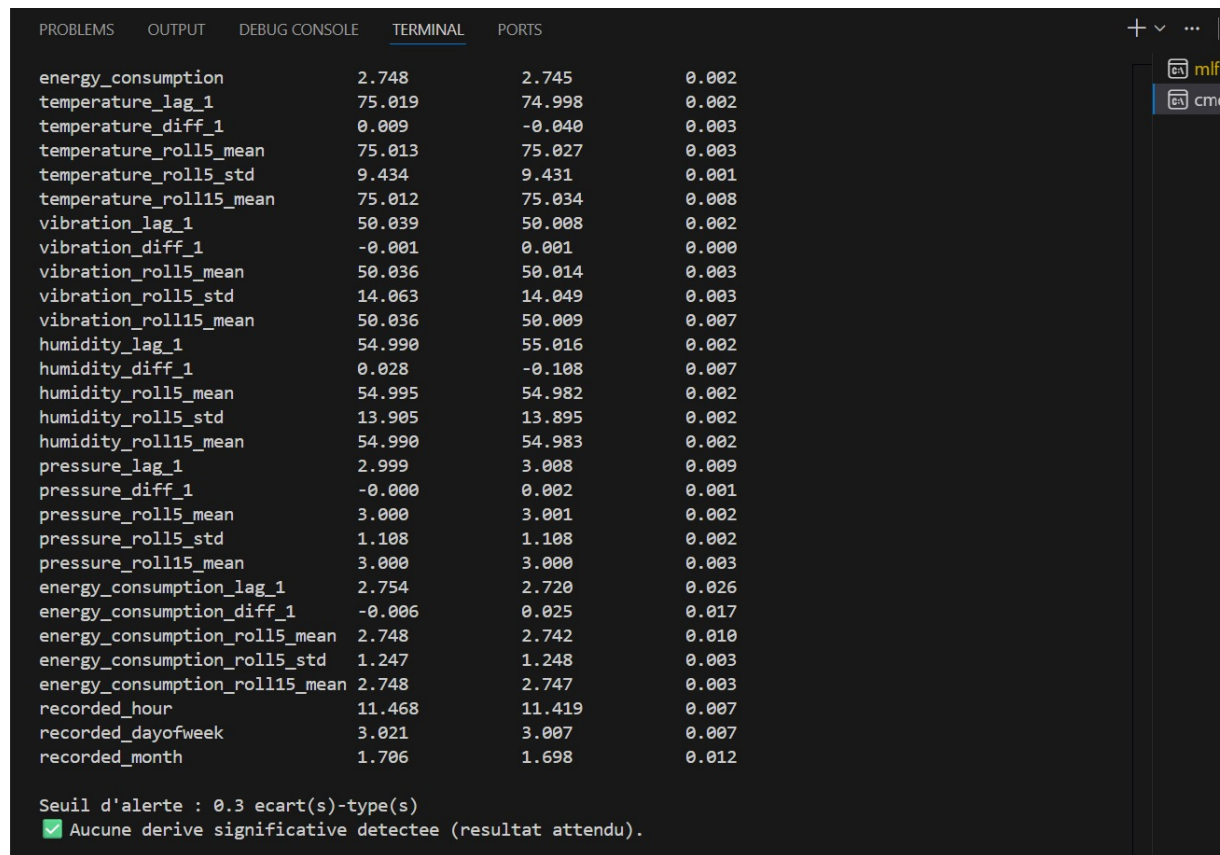
où $\mu_{\text{référence}}$ et $\sigma_{\text{référence}}$ sont respectivement la moyenne et l'écart-type de la variable dans

les données de référence (entraînement), et μ_{nouveau} la moyenne de la variable dans les nouvelles données comparées. Un seuil d'alerte de 0.3 écart-type a été retenu.

Cette approche a été préférée à un simple calcul de variation en pourcentage. Une première version du script, basée sur le pourcentage de variation, générait de nombreux faux positifs pour les variables dont la moyenne est proche de zéro (notamment les variables de type différence temporelle, *_diff_1), où de très légers écarts absolus produisaient des variations relatives supérieures à 500%, sans qu'il y ait de dérive réelle. Le passage à une mesure en écarts-types a permis de corriger ce biais mathématique.

7.9.2 Cas stable

Le premier scénario compare les données d'entraînement au jeu de test réel (données jamais vues par le modèle, ni pour l'entraînement, ni pour la calibration du seuil de décision).



Variable	2.748	2.745	0.002
energy_consumption	2.748	2.745	0.002
temperature_lag_1	75.019	74.998	0.002
temperature_diff_1	0.009	-0.040	0.003
temperature_roll15_mean	75.013	75.027	0.003
temperature_roll15_std	9.434	9.431	0.001
temperature_roll15_mean	75.012	75.034	0.008
vibration_lag_1	50.039	50.008	0.002
vibration_diff_1	-0.001	0.001	0.000
vibration_roll15_mean	50.036	50.014	0.003
vibration_roll15_std	14.063	14.049	0.003
vibration_roll15_mean	50.036	50.009	0.007
humidity_lag_1	54.990	55.016	0.002
humidity_diff_1	0.028	-0.108	0.007
humidity_roll15_mean	54.995	54.982	0.002
humidity_roll15_std	13.905	13.895	0.002
humidity_roll15_mean	54.990	54.983	0.002
pressure_lag_1	2.999	3.008	0.009
pressure_diff_1	-0.000	0.002	0.001
pressure_roll15_mean	3.000	3.001	0.002
pressure_roll15_std	1.108	1.108	0.002
pressure_roll15_mean	3.000	3.000	0.003
energy_consumption_lag_1	2.754	2.720	0.026
energy_consumption_diff_1	-0.006	0.025	0.017
energy_consumption_roll15_mean	2.748	2.742	0.010
energy_consumption_roll15_std	1.247	1.248	0.003
energy_consumption_roll15_mean	2.748	2.747	0.003
recorded_hour	11.468	11.419	0.007
recorded_dayofweek	3.021	3.007	0.007
recorded_month	1.706	1.698	0.012

Seuil d'alerte : 0.3 écart(s)-type(s)
☒ Aucune derive significative detectee (resultat attendu).

FIGURE 7.8 – Résultat de la détection de dérive — cas stable (aucune alerte)

La figure 7.8 montre qu'aucune dérive significative n'a été détectée sur l'ensemble des 33 variables numériques analysées : tous les z-scores restent proches de 0 et largement inférieurs au seuil d'alerte de 0.3. Ce résultat est celui attendu, puisque les données d'entraînement et de test proviennent du même jeu de données source et présentent donc des distributions statistiquement similaires.

7.9.3 Cas simulé

Afin de valider que le mécanisme de détection réagit correctement en présence d'une véritable dérive, un second scénario a été construit en modifiant artificiellement une copie des données : la température a été augmentée de 30% et la vibration de 40%, afin de reproduire un scénario réaliste de début de surchauffe d'une machine industrielle.

```
=====
3. CAS SIMULE - Donnees volontairement modifiees (derive attendue)
=====
Scenario : temperature +30%, vibration +40% (ex. debut de surchauffe)
```

Variable	Moyenne ref.	Moyenne nouv.	Ecart (z-score)	
machine_id	25.552	25.461	0.006	
temperature	75.029	97.699	2.263	⚠ DERIVE
vibration	50.039	70.045	1.341	⚠ DERIVE
humidity	55.018	54.904	0.008	
pressure	2.998	3.002	0.003	
energy_consumption	2.748	2.749	0.001	
temperature_lag_1	75.019	75.084	0.006	
temperature_diff_1	0.009	0.065	0.004	
temperature_roll5_mean	75.013	75.004	0.002	
temperature_roll5_std	9.434	9.431	0.001	
temperature_roll15_mean	75.012	75.040	0.010	
vibration_lag_1	50.039	49.912	0.008	
vibration_diff_1	-0.001	0.120	0.006	
vibration_roll5_mean	50.036	50.047	0.002	
vibration_roll5_std	14.063	14.132	0.014	
vibration_roll15_mean	50.036	50.039	0.001	
humidity_lag_1	54.990	54.953	0.003	
humidity_diff_1	0.028	-0.048	0.004	
humidity_roll5_mean	54.995	54.961	0.005	
humidity_roll5_std	13.905	13.900	0.001	
humidity_roll15_mean	54.990	54.974	0.004	
pressure_lag_1	2.999	3.013	0.013	
pressure_diff_1	-0.000	-0.011	0.006	
pressure_roll5_mean	3.000	3.003	0.005	
pressure_roll5_std	1.108	1.106	0.005	

FIGURE 7.9 – Résultat de la détection de dérive — cas simulé (température +30%, vibration +40%)

La figure 7.9 montre que le système a correctement identifié les deux variables modifiées, avec des z-scores respectifs de **2.263** (température) et **1.341** (vibration), toutes deux nettement supérieures au seuil d'alerte de 0.3. Aucune des 31 autres variables n'a déclenché de fausse alerte. Ce résultat démontre que le mécanisme ne se contente pas de rester silencieux par défaut : il est capable de détecter précisément les variables réellement concernées par un changement de distribution, sans bruit sur les variables non affectées.

7.9.4 Synthèse et limites

Cette double validation (cas stable et cas simulé) permet de démontrer que le mécanisme de détection de dérive fonctionne correctement dans les deux directions attendues : il reste silencieux en l'absence de changement réel des données, et il détecte précisément une dérive lorsqu'elle est introduite.

Il convient toutefois de noter une limite méthodologique assumée : dans un contexte de production réelle, la comparaison devrait s'effectuer entre les données d'entraînement et des données effectivement collectées après le déploiement du modèle, sur une période donnée. En l'absence de telles données à ce stade du projet, la validation présentée ici repose sur un sous-ensemble du jeu de données original (cas stable) et sur une version de ce même sous-ensemble modifiée artificiellement (cas simulé). Cette approche permet de valider le fonctionnement technique du mécanisme, mais devra être complétée par un suivi sur des données réelles une fois le modèle déployé en production.

7.10 Conclusion

L'intégration de MLflow au pipeline de Machine Learning développé par Maroua a permis de structurer le suivi des expérimentations, de tracer précisément les paramètres et métriques de chaque modèle testé, et d'enregistrer officiellement le meilleur modèle (Gradient Boosting) dans un Model Registry versionné. Un mécanisme de monitoring simple, couvrant à la fois le suivi de performance dans le temps et la détection de dérive des données, a été mis en place et validé selon deux scénarios complémentaires : un cas stable confirmant l'absence de fausses alertes, et un cas simulé confirmant la capacité du système à détecter une dérive réelle. L'ensemble des artefacts produits (modèle sérialisé, métriques, rapport de dérive) est prêt à être exploité par la partie déploiement du projet, présentée dans le chapitre suivant.

Chapitre 8: FastAPI, Docker, CI/CD et déploiement

8.1 Objectif du déploiement

L'objectif de cette dernière partie technique du projet est de transformer le modèle de prédiction de pannes, entraîné par Maroua et suivi via MLflow par Rachida (voir chapitres précédents), en un *service* directement utilisable par une application tierce ou un outil de supervision industrielle. Concrètement, il s'agit de construire une API REST stable, testée automatiquement, conteneurisée avec Docker, intégrée à un pipeline CI/CD, puis déployée sur l'infrastructure fournie par l'équipe pédagogique du module.

Cette chaîne complète – de l'exposition du modèle jusqu'à sa mise en production – correspond au rôle de DevOps / API Engineer au sein de l'équipe : le modèle produit par la partie Machine Learning n'a de valeur opérationnelle que s'il peut être interrogé de façon fiable par un système externe, avec des garanties de disponibilité et de reproductibilité.

8.2 Architecture de l'API FastAPI

L'API a été développée avec le framework **FastAPI**, choisi pour sa rapidité de mise en œuvre, sa validation automatique des données via Pydantic, et la génération native d'une documentation interactive (Swagger UI). Le projet est organisé de la manière suivante :

- `app/main.py` – points d'entrée de l'API (`/health`, `/predict`);
- `app/model_service.py` – chargement du modèle et calcul des features en ligne;
- `app/schemas.py` – contrats de données (requête et réponse) via Pydantic;
- `app/config.py` – configuration via variables d'environnement;
- `models/best_model.joblib` – modèle fourni par Maroua (pipeline scikit-learn, seuil de décision, liste des features);
- `tests/test_api.py` – suite de tests automatisés.

Le fichier `best_model.joblib` est un dictionnaire contenant le pipeline scikit-learn complet (prétraitement et classifieur `GradientBoostingClassifier`), le seuil de décision optimal (0,18, retenu pour maximiser le F1-score sous contrainte de précision) ainsi que la liste exacte des 34 colonnes attendues en entrée.

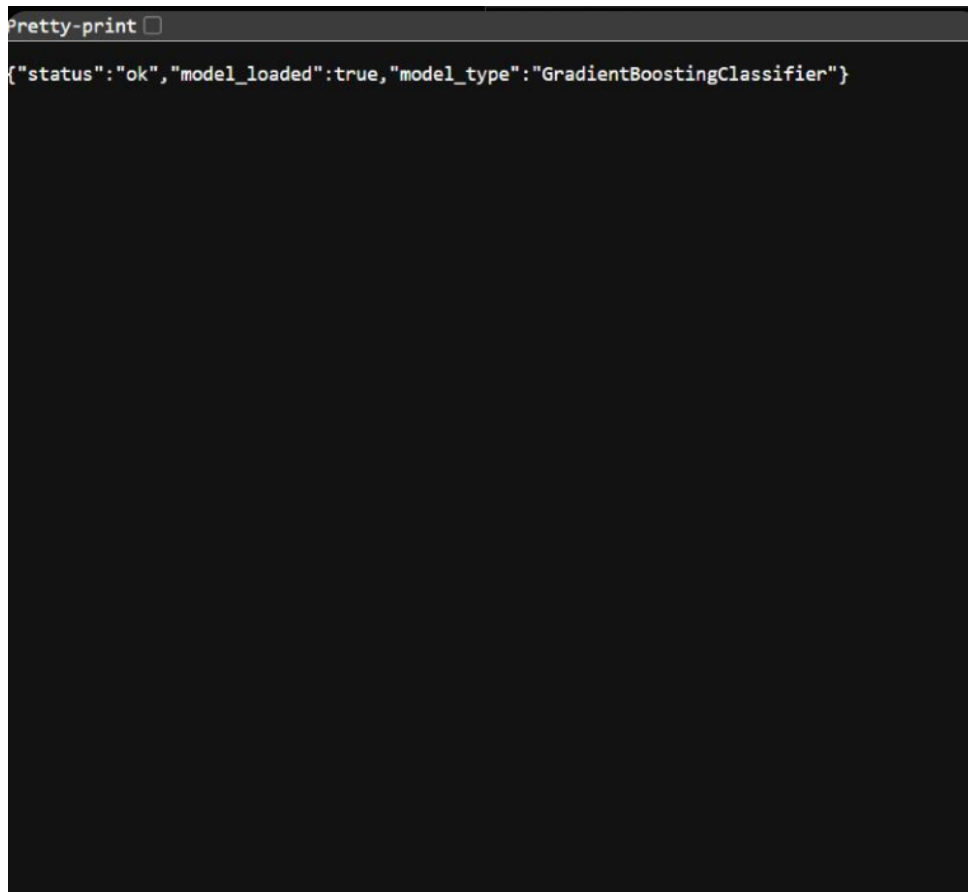
Une difficulté technique importante a été identifiée dès le chargement du modèle : celui-ci a été entraîné avec `scikit-learn==1.6.1`. Le charger avec une version plus récente

(1.9.0, disponible par défaut sur la machine de développement) provoque une erreur de désérialisation (`ModuleNotFoundError: No module named '_loss'`), causée par une ré-organisation interne de scikit-learn entre ces deux versions. Le fichier `requirements.txt` de l'API fixe donc précisément les mêmes versions que celles utilisées à l'entraînement (`scikit-learn==1.6.1`, `pandas==2.2.3`, `numpy==2.2.5`, `scipy==1.15.2`), garantissant la reproductibilité du service.

Autre point d'attention : le modèle attend, en plus des cinq valeurs capteurs brutes (température, vibration, humidité, pression, consommation énergétique), des *features glissantes* calculées par machine (valeur précédente, différence, moyenne et écart-type mobiles sur 5 et 15 relevés). Une seule requête HTTP ne peut pas contenir cet historique. Le module `model_service.py` maintient donc un petit historique en mémoire par `machine_id` (15 relevés, paramétrable via `HISTORY_SIZE`) et reproduit exactement les transformations utilisées lors de l'entraînement, afin que les prédictions en production soient cohérentes avec celles obtenues pendant l'évaluation du modèle.

8.3 Endpoint GET /health

Ce endpoint permet de vérifier que le service est démarré et que le modèle est correctement chargé en mémoire – un prérequis indispensable avant d'accepter des requêtes de prédiction.

A screenshot of a terminal window with a dark background. The title bar at the top says "Pretty-print" followed by a small square icon. The terminal displays a single line of JSON text: {"status":"ok","model_loaded":true,"model_type":"GradientBoostingClassifier"}.

```
Pretty-print ☐  
{ "status": "ok", "model_loaded": true, "model_type": "GradientBoostingClassifier" }
```

FIGURE 8.1 – Réponse de l'endpoint GET /health

La figure 8.1 montre la réponse retournée : `status: ok`, `model_loaded: true` et le type de modèle chargé (`GradientBoostingClassifier`). Ce endpoint est utilisé à la fois par le HEALTHCHECK Docker et par le pipeline CI/CD pour valider qu'un déploiement s'est correctement déroulé.

8.4 Endpoint POST /predict

Ce endpoint reçoit un relevé capteur d'une machine et retourne la probabilité de panne ainsi qu'une décision binaire fondée sur le seuil optimal du modèle :

```
{  
  "machine_id": 39,  
  "temperature": 85.2,  
  "vibration": 4.1,  
  "humidity": 55,  
  "pressure": 102,  
  "energy_consumption": 20  
}
```

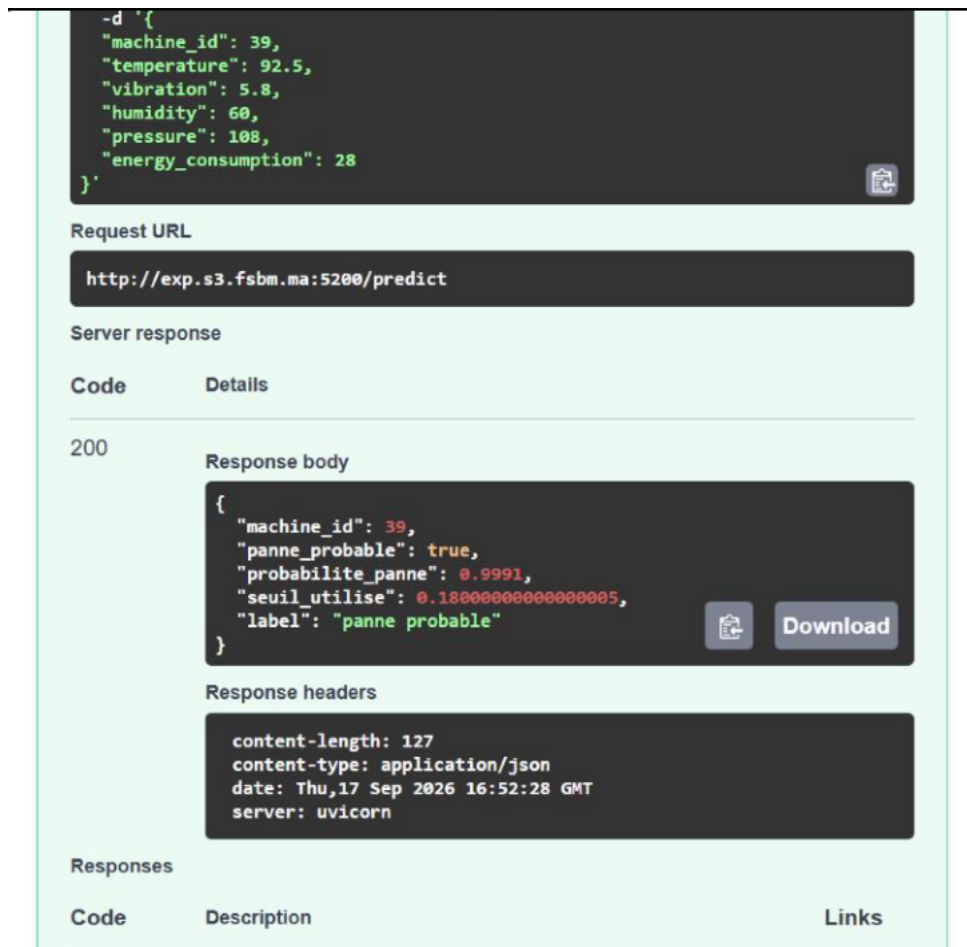


FIGURE 8.2 – Test de l’endpoint POST /predict via Swagger UI

La figure 8.2 montre un appel réel exécuté depuis Swagger UI (*Try it out* → *Execute*), avec une réponse 200 contenant `panne_probable`, `probabilite_panne` et le `seuil_utilise`. La validation des entrées (types, champs obligatoires) est gérée automatiquement par Pydantic : une requête incomplète retourne un code 422 avec le détail du champ manquant, sans qu’il soit nécessaire de coder cette vérification manuellement.

8.5 Tests de l’API

Une suite de tests automatisés (`pytest`) a été développée pour valider le comportement du service de façon reproductible, indépendamment de toute vérification manuelle :

- GET /health retourne `status=ok` et `model_loaded=true` ;
- POST /predict retourne une prédiction cohérente (probabilité entre 0 et 1, label attendu) ;
- plusieurs appels successifs pour une même machine construisent l’historique sans erreur ;
- une requête incomplète retourne bien un code 422.

L'ensemble des quatre tests passe avec succès en local. Ce comportement a également été confirmé manuellement via Swagger UI et en ligne de commande, avec des résultats identiques.

8.6 Conteneurisation Docker

Le `Dockerfile` part d'une image `python:3.11-slim`, installe les dépendances figées, copie le code et le modèle, puis démarre `uvicorn` sur le port 8000. Un `HEALTHCHECK` interroge `/health` toutes les 30 secondes, permettant à l'orchestrateur (Docker, ou la plateforme de déploiement) de détecter automatiquement un service dégradé.

Le build et l'exécution de l'image ont été vérifiés en local : la commande `docker build` se termine sans erreur, le conteneur démarre et passe à l'état `healthy`, et les deux endpoints répondent correctement depuis l'intérieur du conteneur.

8.7 Docker Compose

Le fichier `docker-compose.yml` construit l'image et démarre le service avec les variables d'environnement nécessaires (`MODEL_PATH`, `HISTORY_SIZE`), redémarre automatiquement le conteneur en cas d'échec (`restart: unless-stopped`), et expose le port du service via une variable `API_PORT`, ce qui permet d'adapter le port publié à l'infrastructure cible sans modifier le code. Ce choix s'est avéré déterminant lors du déploiement final (voir section 8.9), la plateforme fournie imposant une plage de ports spécifique par groupe.

8.8 Pipeline CI/CD avec GitHub Actions

Le code de l'API est hébergé sur un dépôt Git dédié (`predictops-mlops-api`), avec un workflow GitHub Actions (`.github/workflows/ci-cd.yml`) déclenché à chaque *push* ou *pull request*. Il comprend deux *jobs* exécutés séquentiellement :

1. **test** – installation des dépendances, analyse statique du code (`ruff`), puis exécution de la suite `pytest` ;
2. **build** – construction de l'image Docker, puis démarrage d'un conteneur éphémère avec vérification que `/health` répond correctement avant de valider le pipeline.

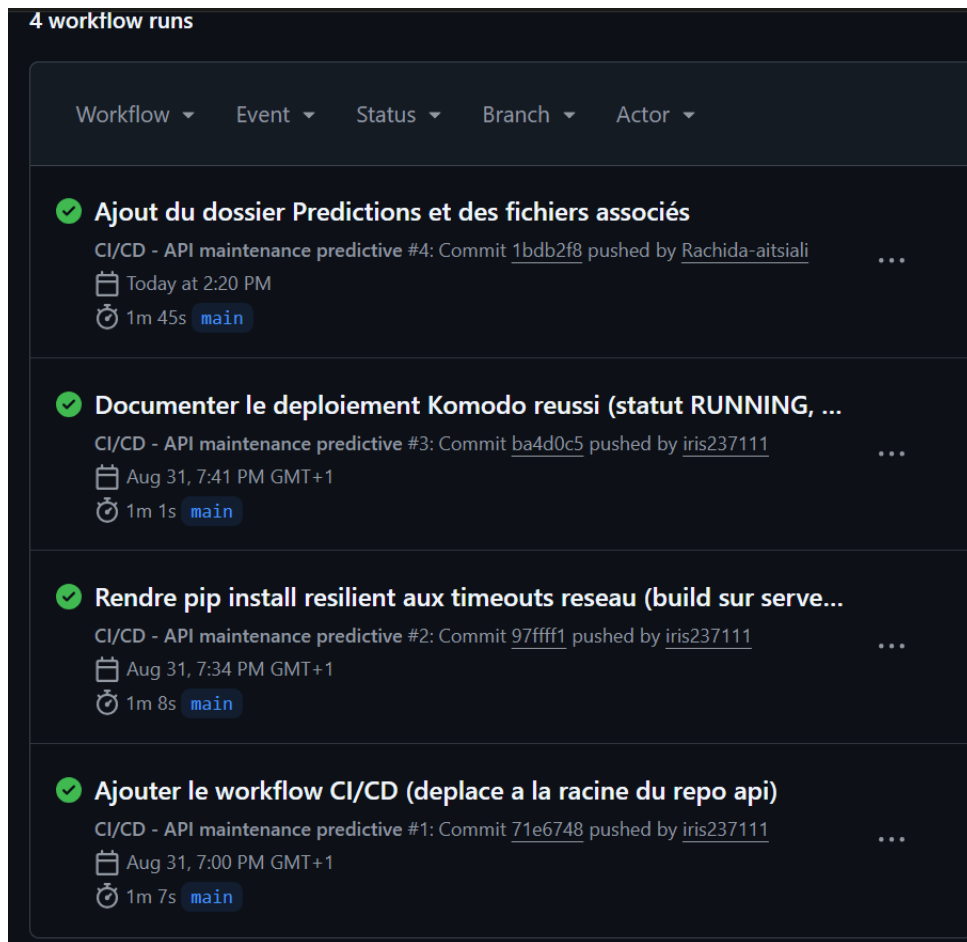


FIGURE 8.3 – Pipeline CI/CD GitHub Actions en succès (jobs `test` et `build`)

La figure 8.3 montre les deux jobs validés en succès. Ce pipeline garantit qu’aucune régression fonctionnelle (tests) ni aucune erreur de construction de l’image ne peut être fusionnée sans être détectée automatiquement, ce qui est essentiel dans un contexte de travail en équipe.

8.9 Déploiement sur l’infrastructure Komodo

Contrairement à un déploiement sur un service cloud public classique (AWS, Azure, ...), l’infrastructure fournie par le professeur pour ce module repose sur **Komodo**, une plateforme self-hosted de gestion et de déploiement de conteneurs, installée sur un serveur partagé entre tous les groupes du cours. Chaque groupe dispose d’un *Stack* dédié (le nôtre, `predictops`) avec des permissions limitées (*Write*, *Inspect*, *Logs*, *Terminal*, sans droits d’administration sur le serveur), ainsi que d’une plage de ports réservée (5200-5299) afin d’éviter les conflits entre les services des différents groupes.

Le Stack a été configuré pour cloner directement le dépôt Git de l’API (`iris237111/predictops-ml` branche `main`) et exécuter `docker compose up -build` à chaque déploiement, ce qui correspond exactement au `Dockerfile` et au `docker-compose.yml` déjà validés en local –

sans nécessiter de registre d'images intermédiaire. Le port publié a été fixé à 5200 (variable `API_PORT`), dans la plage attribuée au groupe.

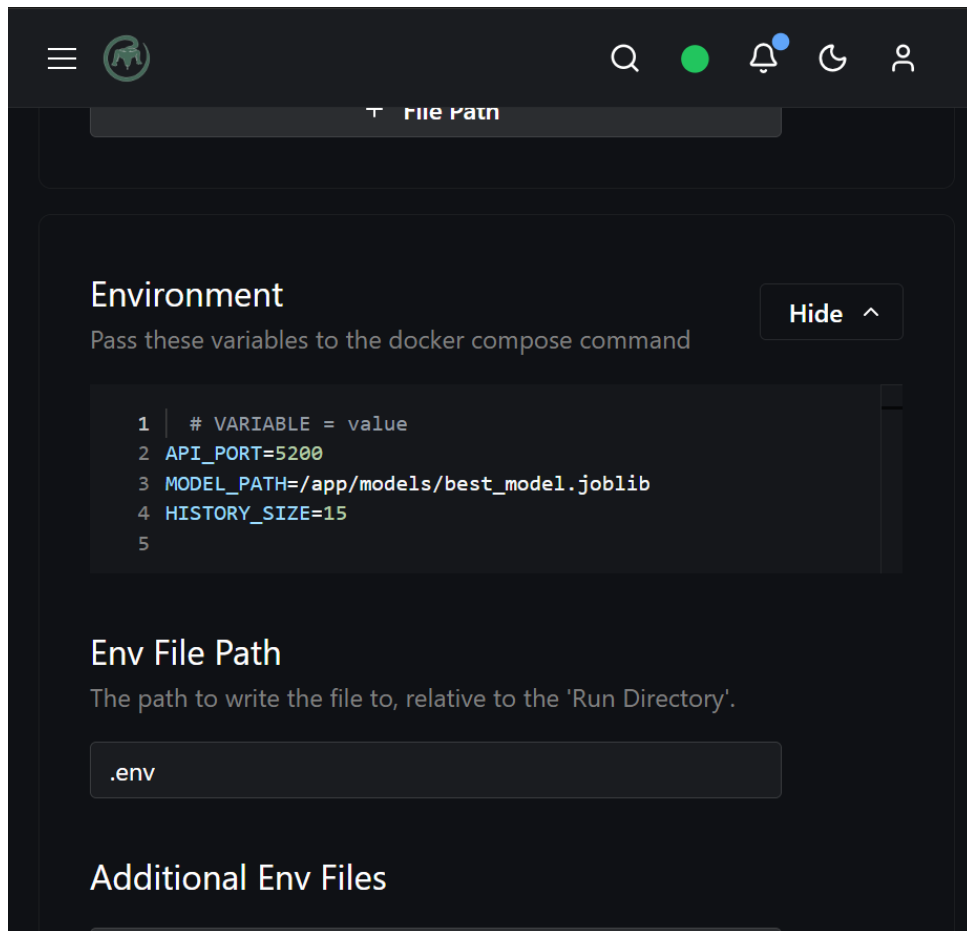


FIGURE 8.4 – Configuration du Stack Komodo – source Git et variables d'environnement

La figure 8.4 montre la configuration du Stack : dépôt source, branche, et les trois variables d'environnement transmises au `docker-compose.yml` (`API_PORT`, `MODEL_PATH`, `HISTORY_SIZE`).

Le premier déploiement a échoué : le journal d'exécution a révélé une erreur réseau lors de la phase de construction de l'image (`pip._vendor.urllib3.exceptions.ReadTimeoutError`, un *timeout* lors du téléchargement d'une dépendance depuis `files.pythonhosted.org`), liée à la lenteur ou à l'instabilité du réseau sortant du serveur partagé, et non à une erreur de code. Ce problème a été corrigé en rendant la commande d'installation des dépendances plus tolérante aux coupures réseau, via les options `-default-timeout=120` et `-retries 5` ajoutées à `pip install` dans le `Dockerfile`.

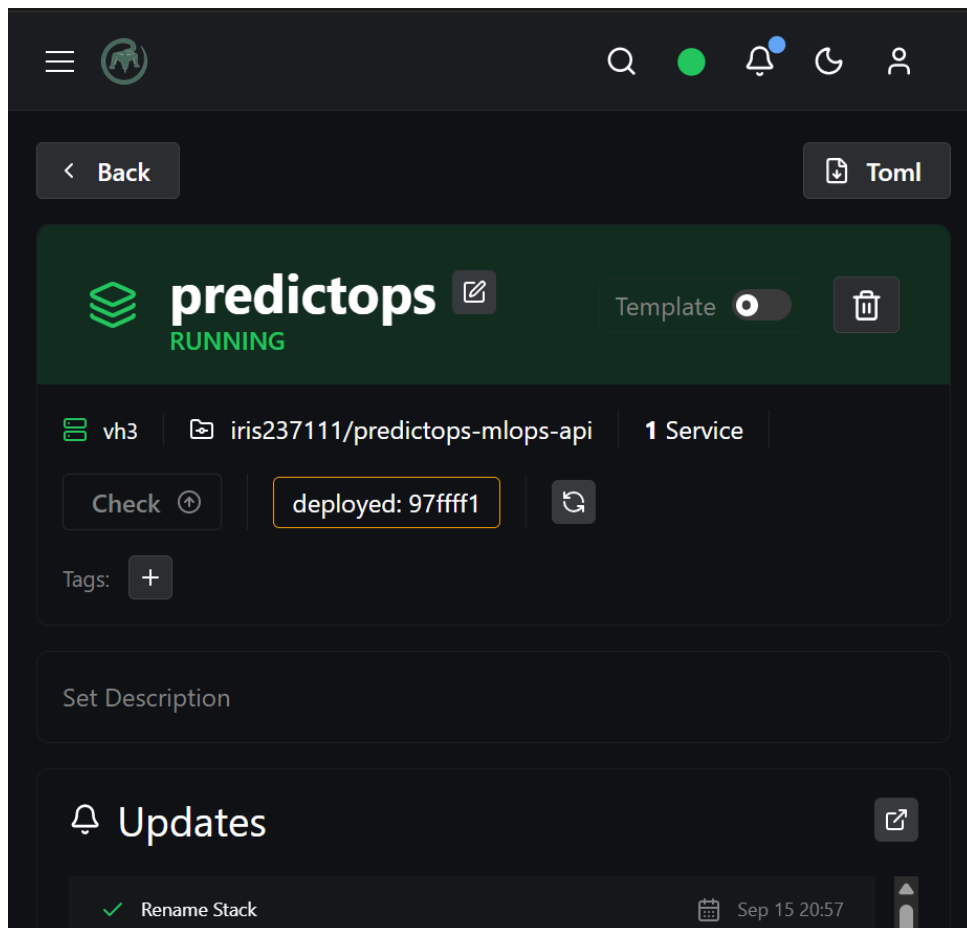


FIGURE 8.5 – Stack Komodo à l'état **RUNNING** après correction

Après cette correction, le déploiement s'est terminé avec succès et le Stack est passé à l'état **RUNNING** (figure 8.5), avec le commit correspondant au correctif effectivement déployé. L'API a ensuite été vérifiée comme accessible publiquement à l'adresse `http://exp.s3.fsbm.ma:5` confirmant que l'ensemble de la chaîne – du code source jusqu'au service exposé sur le serveur partagé – fonctionne de bout en bout.

8.10 Monitoring du service

Le endpoint `/health` sert de sonde de disponibilité et est déjà exploité à trois niveaux : le **HEALTHCHECK** du conteneur Docker, le job **build** du pipeline CI/CD, et la supervision native de Komodo, qui affiche l'état du Stack en temps réel.

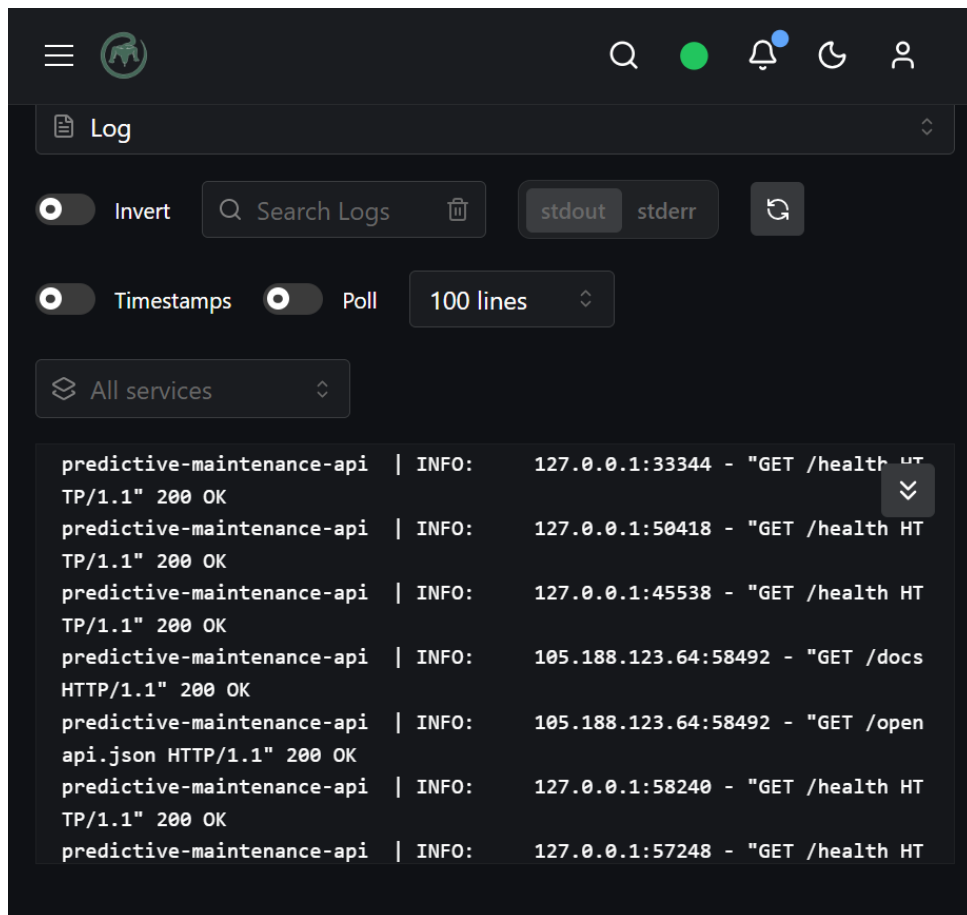


FIGURE 8.6 – Journal Komodo – requêtes de health check internes (HTTP 200)

La figure 8.6 montre l'onglet *Logs* du Stack Komodo, dans lequel apparaissent les requêtes `GET /health` traitées avec succès (`HTTP/1.1" 200`) par le service. Ce même mécanisme pourrait, dans une évolution future, être connecté à un système d'alerte externe ou au monitoring de dérive des données mis en place par Rachida (chapitre 7), afin de couvrir à la fois la disponibilité technique du service et la qualité des prédictions produites dans le temps.

8.11 Conclusion

Ce chapitre a permis de transformer le modèle de prédiction de pannes, entraîné et suivi dans les chapitres précédents, en un service REST complet et opérationnel : une API FastAPI testée automatiquement, conteneurisée avec Docker, intégrée à un pipeline CI/CD GitHub Actions, et effectivement déployée sur l'infrastructure Komodo fournie pour le module. Les deux difficultés techniques rencontrées – l'incompatibilité de version de scikit-learn lors du chargement du modèle, et l'instabilité réseau du serveur partagé lors de la construction de l'image – ont chacune été diagnostiquées précisément et corrigées, sans compromis sur la reproductibilité du service. L'API est aujourd'hui accessible publi-

quement et prête à être interrogée par un système de supervision industrielle, concrétisant ainsi l'objectif initial du projet : fournir un outil d'aide à la décision pour la maintenance prédictive.

Conclusion Générale

Ce projet a permis de concevoir et de déployer PredictOps, un pipeline complet de maintenance prédictive couvrant l'ensemble du cycle de vie de la donnée industrielle, depuis l'ingestion brute jusqu'à la mise à disposition d'un service de prédiction opérationnel. Chaque étape du pipeline a été construite en appliquant les principes DataOps et MLOps : l'ingestion avec `dlt` a garanti la reproductibilité du chargement des données, le stockage en couches avec `DuckDB` a structuré la donnée depuis son état brut jusqu'aux features exploitables par le modèle, les transformations et tests `dbt` ont assuré la qualité et la gouvernance de la donnée à travers un Data Contract formalisé, et l'orchestration avec `Dagster` a automatisé l'enchaînement de bout en bout de ces étapes.

Sur le plan de la modélisation, plusieurs algorithmes de classification ont été comparés afin de prédire le besoin de maintenance à partir des relevés capteurs, le modèle Gradient Boosting étant retenu pour ses performances. Le suivi des expériences avec `MLflow` a permis de tracer les paramètres, les métriques et les artefacts de chaque run, ainsi que de versionner le modèle final dans un Model Registry, tandis qu'un mécanisme simple de détection de dérive a posé les bases d'un monitoring dans le temps de la qualité des prédictions. Enfin, l'exposition du modèle via une API FastAPI, conteneurisée avec Docker et intégrée à une chaîne CI/CD GitHub Actions, a permis un déploiement effectif sur l'infrastructure Komodo, rendant le service accessible et interrogeable en conditions réelles.

Malgré ces résultats, certaines limites subsistent. Le jeu de données couvre une fenêtre temporelle restreinte (janvier à mars 2025), ce qui limite la capacité du modèle à généraliser à des variations saisonnières plus larges ou à de nouvelles machines. Le mécanisme de détection de dérive mis en place reste par ailleurs volontairement simple et gagnerait à être renforcé par des méthodes statistiques plus robustes ainsi que par une alerte automatisée connectée à un système de supervision externe. Comme perspectives d'amélioration, le pipeline pourrait être enrichi par une ingestion en flux continu plutôt que par lots, par l'ajout de nouvelles sources de données (maintenance historique, interventions manuelles), ainsi que par une industrialisation plus poussée du monitoring de dérive et du réentraînement automatique du modèle.

En définitive, ce projet illustre comment une chaîne DataOps/MLOps rigoureuse et outillée permet de transformer un flux brut de données capteurs en un véritable outil d'aide à la décision pour les équipes de maintenance industrielle, tout en posant les bases d'une amélioration continue du service.